

Retrieval-Augmented Generation for Large Language Models: Foundations, Techniques, and Future Directions

Generated by AutoSurvey

Contents

1	Introduction and Background	3
1.1	Background and Limitations of Large Language Models	3
1.2	Foundations and Motivation of Retrieval-Augmented Generation	4
1.3	Evolution of RAG Paradigms	5
1.4	Survey Scope, Contributions, and Organization	7
2	Core Architectures and Methodological Foundations of RAG	9
2.1	From Naive to Advanced and Modular RAG: A Paradigm Evolution	9
2.2	Integration Mechanisms for Retrievers and Generators	10
2.3	Memory-Augmented Architectures and Knowledge Construction	12
2.4	Design Choices: Black-Box vs. Fine-Tuned Systems and Impact on Performance . .	14
3	Retrieval Strategies, Indexing, and Knowledge Sources	16
3.1	Taxonomy of Retrieval Models in RAG	16
3.2	Query Understanding and Enhancement Techniques	18
3.3	Indexing Structures and Knowledge Source Diversity	20
4	Generation Techniques and Context Integration in RAG	22
4.1	In-Context Learning and Demonstration Selection in RAG	22
4.2	Instruction Tuning and Grounded Response Generation	24
4.3	Context Compression, Fusion Strategies, and Attention Efficiency	27
4.4	Faithfulness, Reasoning under Conflict, and Output Control	29
5	Training, Fine-tuning, and Optimization Strategies	31
5.1	Foundations of Fine-Tuning in RAG: Full, Unsupervised, and Few-Shot Paradigms .	31
5.2	Parameter-Efficient Tuning for RAG: From LoRA to Sparse and Hybrid Methods . .	33
5.3	Training Augmentation, Self-Feedback, and Iterative Refinement in RAG	35
5.4	Optimization, Robustness, and Resource-Aware Fine-Tuning for Practical RAG Deployment	37
6	Robustness, Security, and Knowledge Conflict Management	39
6.1	Knowledge Conflicts Between Parametric Prior and External Evidence	39
6.2	Adversarial Robustness, Noise, and Negative Rejection	41
6.3	Security Threats and Privacy Implications	44
6.3.1	Security Threats: The Expanded Attack Surface	44
6.3.2	Privacy Leakage and the Dual-Edged Role of RAG	44

6.3.3	Defense Strategy: Hardening and Privacy-Preserving Mechanisms	45
6.4	Formal Guarantees and Calibration for Trustworthy RAG	45
7	Efficiency, Scalability, and Practical Deployment	48
7.1	Model Compression and KV-Cache Reduction	48
7.2	System-Level Optimization for Latency, Throughput, and Scaling	50
7.3	Advanced Decoding and Dynamic Execution Strategies	52
7.4	Cost–Accuracy Trade-offs and Lightweight Deployment in Resource-Constrained Environments	54
8	RAG for Multimodal, Multilingual, and Domain-Specific Applications	56
8.1	Multimodal RAG: Architectures and Applications Across Modalities	56
8.2	Multilingual and Cross-Lingual RAG	59
8.3	Domain-Specific RAG in High-Stakes Fields: Healthcare, Legal, and Finance	60
8.3.1	Healthcare: Precision, Clinical Reasoning, and Domain Adaptation	61
8.3.2	Legal and Finance: Knowledge-Critical Reasoning and Structured Knowledge	62
8.3.3	Balancing System Implementation, Human Oversight, and System Evaluation	62
8.4	Cross-Domain Generalization, Personalization, and Human-Centered Interaction . .	63
9	Evaluation, Benchmarks, and Interplay with Other LLM Techniques	65
9.1	Evaluation Metrics and Automated Assessment Frameworks	65
9.2	Comprehensive Benchmarks and Multi-Task Evaluation	67
9.3	Interplay with Model Editing, Long Context Windows, and Continual Learning . . .	70
9.4	Attribution Evaluation, Automated Citation, and Human-Aligned Assessment	72
10	Open Challenges, Ethical Society, and Future Directions	74
10.1	Persistent Grammar of Hallucination and Knowledge Conflict	74
10.2	Security, Privacy, and Ethical Considerations	76
10.3	—	79
10.4	Scalable and Bottom-Line Sustainability	80
10.5	Toward Most-Next-Generation and Dynamic RAG: A Roadmap	82
	References	84

1 Introduction and Background

1.1 Background and Limitations of Large Language Models

The emergence of large language models (LLMs) has fundamentally transformed the field of natural language processing, enabling unprecedented capabilities in text generation, comprehension, and complex reasoning across a diverse array of tasks [1, 2]. Built upon vast amounts of training data and increasingly sophisticated neural architectures, these models have demonstrated strong performance on numerous standard benchmarks, ranging from language modeling and translation to intricate question-answering and multi-step reasoning challenges [3, 4]. Their ability to capture intricate statistical patterns in language allows them to generate text that is not only fluent and contextually appropriate but also often difficult to distinguish from human-written content. This capability has led to their rapid integration into numerous real-world applications, positioning them as versatile tools for a huge number of tasks, such as content creation, code generation, and professional support across both general-purpose and highly specialized fields [5]. However, despite these impressive capabilities, a deeper examination of their architecture and training paradigm reveals fundamental limitations that pose severe challenges, particularly for high-stakes and knowledge-intensive applications. These limitations are critical to understand, as they define the boundaries within which these powerful models can be reliably used.

Among the most significant obstacles to establishing LLM trustworthiness is the phenomenon of “hallucination,” where the model produces outputs that are plausible-sounding yet factually incorrect, nonsensical, or unsupported by available sources [6, 7]. Hallucinations are not a niche or occasional error; they are a systemic deficiency and an innate limitation of the current architecture, being a direct byproduct of how these models learn to emulate the statistical distribution of the text they are trained on [6]. As the model’s output is not constrained to be synonymous with claims for which it has verifiable evidence, a condition known as “evidential closure,” it can generate unattested information as easily as it can generate empirically derived facts [8]. Often, this generation is nondeterministic and can lead to content or ontology hallucinations, where the model either recalls incorrect facts, or introduces and reasons with fictional concepts, entities, or logical frameworks that have no grounding in reality or in the provided context [2]. This occurs across a spectrum of tasks, from simple fact retrieval to complex legal cases, where the model often fails to bias its output towards ground truth data, instead invoking its existing “knowledge” that may be outdated, biased, or entirely fabricated [9, 10].

The issue is further aggravated by the problem of knowledge cutoff and the inherent non-verifiability of stored information. LLMs encode all their knowledge during pre-training, resulting in a static snapshot of data up to a certain point in time. This encoded semantic information cannot be easily updated to reflect new facts or events, leading to a static and potentially outdated knowledge base [11]. When LLMs encounter queries that depend on recent information or require granular details not covered in their training data, they often pattern-match a plausible answer, frequently resulting in hallucinated or unverifiable content. For instance, in financial ecosystems where real-time data is critical, a static parametric model without dynamic updates will provide misleading answers [12].

Furthermore, the architecture’s operational design leads to difficulties in handling knowledge-intensive tasks. While LLMs excel at pattern recognition and retrieval of high-frequency information, they often struggle to identify what internal information is missing, failing to recognize the boundaries of their own knowledge. Experimental evidence shows that LLMs are frequently overconfident in their responses and cannot perceive the full extent of the gaps in their semantic knowledge [13, 14]. This overconfidence is particularly harmful, as users may trust model outputs, including

hallucinations, due to the model’s own certainty. For example, high-end LLMs have been shown to fail drastically when asked to produce specific legal citations, generating appeals to arbitrary legal entities or nonexistent case law, while presenting this fabricated information with high confidence [10, 15]. A single such error can undermine the credibility of an entire response.

The challenges are further intensified by the opaque reasoning mechanisms of LLMs. LLMs are complex mathematical devices with billions of parameters, but the logic that leads to a final output is often hidden, making it difficult to understand why a given answer was chosen or whether that reasoning is trustworthy [16, 17]. Without a transparent rationale, the model cannot establish epistemic credibility, because the output cannot be verified against a causal chain of facts. This opacity is a core challenge for high-stakes domains that require not just correctness but also explainability and a clear chain of reasoning. For example, in law and medicine, it is insufficient for an LLM to simply output a conclusion; it must support that conclusion with sound professional reasoning, such as legal precedent or medical evidence [18, 9]. When the model’s decision-making process is a black-box, users cannot verify or audit its reasoning, preventing trust and adoption in these sensitive fields.

Another consequence of the opaque and text-based nature of these systems is their vulnerability to confirmation bias. Not only do their outputs reflect the statistical regularities in the training data, but they are also susceptible to deceptive prompts and inputs: they can be misled by semantic and logical shortcuts to generate answers that are incorrect, biased, or even deliberately misleading [19, 20]. For instance, LLMs often struggle with contradictory or logically flawed prompts and may acknowledge errors yet fail to correct them [21]. This reinforces the notion that LLMs are not inherently responsible for ensuring their outputs align with verifiable facts and the actual state of the world.

Given these limitations—most notably hallucination, stagnant knowledge, and unverifiable opaque reasoning—there is an urgent need for a paradigm shift from purely parametric memory systems to those augmented with non-parametric, verifiable external evidence [22, 16]. The reliability and correctness of an LLM should therefore not depend solely on its static internal parameters; it must instead be supported by a dynamic ability to access external data, evidence, and logic to generate trustworthy, actionable information. This transition, from a closed-book parametric system to an open-book framework that integrates authoritative external sources, is precisely the central theme of this survey: Retrieval-Augmented Generation (RAG). This paradigm, as detailed in the following sections, holds the potential for mitigating hallucinations, solving the issue of staleness, and enabling auditable generation output—ultimately making LLMs more reliable and trustworthy for real-world, knowledge-intensive applications.

1.2 Foundations and Motivation of Retrieval-Augmented Generation

Large Language Models (LLMs) have demonstrated remarkable capabilities across a wide spectrum of natural language processing tasks, showcasing an impressive ability to store and manipulate factual knowledge within their parameters [22]. However, as established in the previous section, this parametric knowledge is inherently static, creating several fundamental limitations that hinder reliability and applicability in real-world settings. These models are prone to generating convincing yet factually incorrect outputs, a phenomenon known as hallucination, and their knowledge is frozen at the time of training, leading to a temporal degradation of information that makes them susceptible to providing outdated or stale responses [23, 24]. Furthermore, their performance often diminishes on tasks involving long-tail or less-popular knowledge, and the opaque nature of their internal reasoning makes it difficult to verify and trace the provenance of their claims, which is critical for trustworthy decision-making [24, 25].

To overcome these challenges, Retrieval-Augmented Generation (RAG) has emerged as a powerful paradigm that synergistically combines the parametric knowledge of LLMs with non-parametric external knowledge sources [22]. The core principle of RAG is to augment the generation process by retrieving relevant information from an external database—which can range from a large-scale corpus like Wikipedia to domain-specific knowledge bases—and injecting this retrieved context into the prompt for the LLM. By providing the model with direct access to up-to-date and relevant evidence, RAG shifts the system from a purely closed-book model to an open-book one that can ground its outputs in verifiable source material. This foundational approach, as introduced in [22], combines a pre-trained seq2seq model as the parametric memory with a dense vector index as the non-parametric memory, accessed via a pre-trained neural retriever, effectively establishing a general-purpose architecture for external knowledge integration. This shift from purely parametric generation to hybrid parametric–non-parametric systems directly lays the groundwork for the evolutionary paradigms—Naive, Advanced, and Modular—that will be examined in the following section.

The motivation for RAG is driven by its ability to mitigate hallucinations. By incorporating retrieved evidence, RAG systems provide the LLM with a factual basis for its answers, which has been shown to enhance the factual accuracy of outputs and reduce the risk of generating ungrounded content [22, 26, 27]. While hallucinations can still occur if the retrieved evidence is flawed, RAG’s retrieval step allows the model to operate on more reliable, current information, and outputs become more attributable [28, 26].

Another critical motivation is addressing knowledge staleness and enabling low-cost knowledge updates. The parametric memory of a frozen LLM cannot easily adapt to new or evolving information, but RAG provides a solution by maintaining knowledge in a separate, updatable index. This structure makes it practical to integrate domain-specific or private information and update knowledge as new data becomes available, rather than re-training or fine-tuning the model [23, 29]. This ability to dynamically incorporate external sources provides a decided advantage for applications requiring access to private, specialized, or fast-changing organizational data [22]. Comparative studies against fine-tuning reinforce this benefit, since RAG directly leverages both existing and newly introduced knowledge without altering model parameters, offering both flexibility and cost efficiency [30, 25].

Beyond enhancing accuracy and freshness, RAG also contributes to interpretability and traceability. By retrieving cited sources, RAG enables outputs to be traced back to specific documents, making it possible to verify facts and assess the credibility of the generated content [31, 22]. In contrast to opaque parametric recall, this supports a chain of evidence that helps validate model claims. This becomes particularly critical in high-stakes fields like healthcare, where explainability is a precondition for trust and adoption [32].

In summary, RAG offers a balanced integration of parametric and non-parametric knowledge, directly addressing the core limitations of pure parametric models while laying a structured foundation for progressively more sophisticated mechanisms. It represents not only a solution for mitigating hallucinations and knowledge staleness, but also a pathway toward more interpretable, credible, and continuously adaptable models [22, 33]. As such, it is an effective research direction toward building more faithful, flexible, and accountable LLM systems—an evolution that the next section will trace from its foundational architecture to increasingly advanced and modular variants.

1.3 Evolution of RAG Paradigms

The evolution of Retrieval-Augmented Generation (RAG) can be understood as a progression through three distinct yet overlapping paradigms: Naive RAG, Advanced RAG, and Modular RAG. This evolutionary trajectory reflects the field’s response to inherent limitations in earlier designs,

driven by the need for greater robustness, flexibility, and integration efficiency in knowledge-intensive tasks [23]. Each paradigm represents a successive refinement in how the core components—retrieval, generation, and augmentation—are orchestrated to synergistically combine the parametric knowledge of Large Language Models (LLMs) with non-parametric external knowledge sources [23]. As we will explore, this trajectory directly addresses the motivations outlined earlier: mitigating hallucinations, overcoming knowledge staleness, and enhancing interpretability. Each successive paradigm provides a more sophisticated mechanism for grounding outputs in verifiable evidence, thereby advancing the field toward more trustworthy and adaptable systems.

Naive RAG: The Foundational Retrieve-Then-Generate Paradigm

The foundational approach, often termed Naive RAG, adopts a straightforward “retrieve-then-read” architecture. In this paradigm, a user query is first used to retrieve a fixed set of relevant documents or passages from an external knowledge base, typically using a dense retriever like DPR. These retrieved passages are then prepended to the user’s query as context, forming a prompt that is fed into an LLM to generate the final answer [23]. While this simple process marks a significant improvement over purely parametric generation by introducing external context, it is plagued by several critical vulnerabilities. The Naive RAG approach suffers from imprecise retrieval, where the fetched information may be irrelevant or partially relevant, leading to the potential failure to effectively utilize the key information [34]. It also struggles with handling the noise and erroneous information that can be retrieved, a problem highlighted by the observation that retrieved documents often contain flawed information that diminishes the reliability and correctness of generated outputs [35]. Crucially, Naive RAG lacks mechanisms to evaluate the quality of retrieved documents, and the LLM becomes a passive, and often trusting, receptor of the retrieved knowledge, with no capacity to discern its utility [36]. The model’s parametric prior often conflicts with the external evidence, creating a tug-of-war where the model may either fail to update its internal knowledge or be overly susceptible to incorrect information [37]. This passive role of the generator sets the stage for more sophisticated and active designs.

Advanced RAG: Pre- and Post-Retrieval Optimization

To address the primary shortcomings of Naive RAG, Advanced RAG was introduced, incorporating pre-retrieval and post-retrieval optimization techniques [23, 38]. Pre-retrieval optimization focuses on improving the initial retrieval quality, thereby reducing the risk of imprecise or noisy context that plagued the Naive approach. This can involve query rewriting to transform a complex or ambiguous query into a clearer, more context-aware form, leveraging LLMs to generate more effective search queries [39]. Techniques like query expansion and the use of multiple query generation are also common to improve the likelihood of finding pertinent information [23]. This stage aims to fix and refine the query to guarantee that the quality of the retrieval input is high to begin with, essentially implementing a first-pass filter for relevance. In contrast, post-retrieval optimization improves the quality of the retrieved content before it enters the generation model. This involves strategies like re-ranking the retrieved documents to select the most relevant ones up front, reducing the context size to avoid the “lost-in-the-middle” problem [23]. Moreover, prompt compression aims to distill large amounts of potentially noisy retrieved information into a concise, focused format [23]. Notably, some works within this paradigm have developed corrective mechanisms which explicitly inject a retrieval evaluator to assess the overall quality of retrieved documents. If the quality is deemed to low, the process can trigger a web search fallback to provide the generator with a higher-quality context, surpassing the limitations of static and limited corpora [40]. While Advanced RAG improves the effectiveness of information delivery through these pre- and post-processing steps, it remains a mostly static pipeline in the sense that the LLM is still a reader rather than an active participant in the retrieval loop.

Modular RAG: Flexibility, Adaptivity, and Self-Reflection

The latest paradigm, Modular RAG, marks a decisive shift from static pipelines to flexible, adaptive, and even recursive frameworks [23]. Rather than a static linear flow of retrieve-then-generate, Modular RAG introduces the concept that the model itself must actively participate in the retrieval process [36]. This paradigm integrates mechanisms to learn when to retrieve, what to retrieve, and how to use that information through self-reflection and self-feedback. This is a direct evolution from the passive reception in Naive RAG and the external optimization in Advanced RAG. For instance, the philosophy that the model’s intrinsic knowledge can guide retrieval or be complemented is explored in frameworks like Missing Information Guided Retrieve-Extraction-Solving (MIGRES), where LLMs can identify the missing information to craft targeted queries for retrieving specific documents [39]. The Self-RAG framework raises the bar by training a model that can generate a special token to indicate the usefulness of retrieved passages in addition to its own generation, thus making the model aware and adaptive to the retrieved context [41]. The architecture can be dynamically configured, allowing a system to “do not retrieve” if it has sufficient knowledge or to “feedback” and modify the retrieval strategy. ActiveRAG constructs knowledge construction mechanisms by connecting outside knowledge with already-memorized information, turning passive retrieval into an active learning process that directly calibrates the cognition of the LLM [36]. Modular RAG is characterized by the ability to configure the components of retrieval, generation, and augmentation in a way that allows for iterative and adaptive retrieval-generation workflows, making the full system far more powerful [23]. Some refined modular frameworks also iterate over the retrieved output, using self-feedback to improve problem-solving capabilities repeatedly and enhance the overall factuality of a RAG system, such as the Retrieval Augmented Iterative Self-Feedback (RA-ISF) approach [42].

The Tripartite Foundation of RAG: Retrieval, Augmentation, and Generation

Throughout these three paradigms, the field has solidified a tripartite foundation: the retrieval, the generation, and the augmentation components [23]. The retrieval component determines what external knowledge to integrate, varying from retrieving fixed passages in Naive approaches to the more sophisticated retrieval mechanisms used in Advanced and Modular RAG. The augmentation component serves as the mechanism for knowledge injection, encompassing both preprocessing of retrieved data—as seen in Advanced RAG’s re-ranking and compression—and how it is integrated into the prompt given to the generator. And the generation component, often the LLM itself, is no longer static but, in recent forms, is brought to the engagement of being self-reflective and explicitly reasoning with facts and sources, setting new standards for truthful and reliable outputs [41].

1.4 Survey Scope, Contributions, and Organization

This survey provides a comprehensive and systematic examination of Retrieval-Augmented Generation (RAG) for Large Language Models (LLMs), charting its evolution from foundational concepts to state-of-the-art frameworks. Building directly upon the evolutionary trajectory outlined in the preceding section—from Naive RAG through Advanced RAG to Modular RAG—the scope of this work is deliberately broad, encompassing the architectural, methodological, and practical dimensions of RAG systems. We aim to consolidate the dispersed body of research into a coherent narrative that not only describes the current state of the art but also provides a critical analysis of the challenges and opportunities that lie ahead. Our primary contribution is a structured taxonomy that organizes the RAG landscape into the tripartite foundation—retrieval, generation, and augmentation—introduced earlier, which then guides the in-depth analysis presented in each subsequent section. This taxonomy is not merely a static classification; it is designed as a tool for understanding the evolutionary trajectory of RAG architectures. As we have begun to demonstrate, RAG paradigms have progressed from a simple, sequential approach to advanced, modular, and adaptive systems. This evolution

is a central theme of our survey, as we investigate how each new generation of frameworks refines the integration of parametric and non-parametric knowledge. Moreover, to comprehensively map this evolving field, we have adapted our methodological framework in ways similar to systematic standards used in other complex domains [43].

A key contribution of this work is the in-depth examination of the two fundamental components of any RAG system—the retriever and the generator—which serve as the operational pillars of the tripartite foundation. We provide a dedicated taxonomy of retrieval models, contrasting the trade-offs and synergies between sparse lexical methods, dense semantic approaches, and hybrid strategies. This critical analysis extends to the advanced methodologies for query rewriting-and-expanding, and index construction and management, directly addressing the pre-retrieval optimization challenges identified in the Advanced RAG paradigm. Conversely, on the generation side, we delve into the methods for integrating retrieved context. This includes inspecting prompting strategies for in-context learning, and the more intricate advancements in grounded response generation, which ensures the final claim is not just fluent but also factually verifiable. We also address the practical challenges of integration, such as mitigating the “lost-in-the-middle” problem and the complexities of handling conflicting information, both of which are central to the post-retrieval optimization and self-reflective mechanisms discussed in the previous section. Our coverage of these challenges is analogous to the separation of different conceptual dimensions in other analytical domains [44].

Building upon this foundational analysis, our survey dedicates significant attention to the practical and applied aspects of RAG, moving from principles to deployment-ready considerations. This encompasses the crucial phase of training and adaptation, where we detail various optimization strategies, from parameter-efficient methods like LoRA to reinforcement learning-based approaches. Given that design decisions often involve adapting pretrained models for domain-specific tasks, which have been discussed in the literature as a “fine-tuning” paradigm, we explore the trade-offs of this paradigm in comparison to the in-context learning methods highlighted in our generation analysis [45]. We also address critical deployment obstacles, with a detailed analysis of latency, memory constraints, and cost in the context of real-world systems, which can span multimodal, multilingual, and domain-specific contexts [46].

Relatedly, the increasing reliance on AI-driven systems has brought issues of robustness, security, and trustworthiness to the forefront of RAG research. This critical examination includes analyzing the threat of adversarial attacks, dealing with knowledge conflicts from multiple sources, and ensuring factuality. Dedicated attention is given to the integration of uncertainty awareness, temporal awareness, and the traceability of generated claims. This discussion directly extends the self-reflective and adaptive mechanisms emphasized earlier, addressing how RAG systems can remain reliable in dynamic and conflicting information environments. Here, we can discuss issues ranging from the security of the retrieval knowledge base to the need for transparency in source attribution, and techniques for detecting and resolving conflicting knowledge.

Our survey also strives to comprehend the strengths and limitations of RAG across a variety of domains—from the high-stakes environments of healthcare, legal, and finance to the rapidly evolving phases of multimodal and multilingual [47]. We explore the interaction of RAG with other machine learning paradigms, such as long-context windows, continual learning, and model editing, to delineate its unique and complementary role within the wider methodological landscape. This exploration situates RAG not as an isolated technology but as an integral component of the contemporary AI ecosystem.

Finally, our survey adopts a forward-looking stance. By carefully identifying persistent challenges, we lay a roadmap for future research. This includes potential strategies for achieving more dynamic, self-adaptive RAG systems, with a particular focus on the capacity for interpretability and advanced self-reflection. Moreover, we encourage further investigation into sustainable, scalable solutions

to reconcile the growing demand for intelligent systems with computational budgets and energy constraints, echoing the desire for more efficient and accessible large-scale analysis seen in other domains [48]. With these contributions, we aim to equip the community not only with a map that helps navigate the existing state of the art, but also with a robust foundation for future innovation.

2 Core Architectures and Methodological Foundations of RAG

2.1 From Naive to Advanced and Modular RAG: A Paradigm Evolution

The evolution of Retrieval-Augmented Generation (RAG) architectures represents a fundamental shift in how large language models (LLMs) interact with external knowledge, moving from simple retrieve-then-generate pipelines to sophisticated, adaptive systems that can dynamically reason about when and how to retrieve and integrate information [23]. This progression, commonly categorized into Naive, Advanced, and Modular RAG paradigms, reflects the growing understanding of the complex interplay between parametric memory (the static knowledge within LLM weights) and non-parametric memory (the dynamic, external knowledge sources) [23, 22]. Each paradigm anticipates and addresses specific limitations of the previous, steadily pushing the boundaries of what is achievable in knowledge-intensive tasks like open-domain question answering and fact verification [23].

The foundational **Naive RAG paradigm**, often regarded as the “retrieve-then-generate” approach, establishes the core architecture upon which all subsequent advancements are built [23]. In this framework, the system operates in a linear and fixed workflow: an input query is first used to retrieve a set of relevant documents or passages from a static external knowledge base, and these retrieved documents are then concatenated with the original query to form a prompt. This prompt is finally passed to a pre-trained LLM, which uses the context to generate an answer, thereby grounding responses in external knowledge [22]. The seminal work introducing the RAG model operationalized this by combining a pre-trained seq2seq model as the parametric memory with a dense vector index of Wikipedia, accessed via a pre-trained neural retriever, demonstrating state-of-the-art results on open-domain QA tasks and showing that RAG models can generate more specific, diverse, and factual language than a parametric-only baseline [22]. However, while Naive RAG is effective, it suffers from several critical limitations. Its static nature fails to adapt to complex multi-hop queries that require iterative step-by-step reasoning [39, 49]. Furthermore, the quality of the generated response is tightly coupled to the quality of the retrieval, and it can fail when retrieval retrieves noisy, large, or contradictory information, leaving the LLM unable to process the context effectively [40, 50]. The lack of adaptability and robustness in Naive RAG against irrelevant information and simple query variations has been well documented, leading to the development of more sophisticated approaches [50, 51].

To overcome the limitations of the Naive paradigm, the field advanced to the **Advanced RAG paradigm**, which introduces a suite of optimization techniques centered around the retrieval process [23, 38]. Advanced RAG adopts a data-centric approach by moving the optimization into explicit pre-retrieval and post-retrieval stages, a categorization that enables fine-grained control over the quality and utility of the context given to the LLM [23]. The pre-retrieval stage focuses on improving the searchability and quality of the knowledge base itself. This involves methods like query rewriting, expansion, and pseudo-relevance feedback to align the user’s underlying intent with the available data [29]. It can also include sophisticated indexing techniques, such as representing the corpus in finer granularities like propositions as opposed to larger chunks, or leveraging graph structures to better navigate complex relationships between entities in a corpus [52, 23]. The post-retrieval stage focuses on refining the retrieved knowledge, analyzing the context of the retrieved documents to

mitigate the “lost-in-the-middle” problem, where the LLM needs to attend to all relevant information across a long context, and switching to filter out irrelevant passages in order to avoid the degradation in performance that comes with including unnecessary information in the generation prompt [27, 23]. Techniques include decompression, reranking, and the reordering of chunks in the context to place the most promising information where the model’s attention is most effective, such as at the beginning and end of the context window [23]. By splitting the pipeline into these distinct stages, Advanced RAG leads to a more flexible system. For example, it is a typical RAG setup to have two-stage retrieval (initial retrieval followed by a dedicated reranking model) as a standard means of improving the quality of the retrieved text before it is injected into the generative model [53].

The latest stage in the field is the **Modular RAG paradigm**, a flexible and component-based framework that represents a conceptual leap forward [23]. Rather than functioning as a strict linear pipeline, Modular RAG supports the iterative and adaptive control of individual components, allowing for the interchangeable use of modules and allowing for iterative retrieval-generation workflows, and self-corrective mechanisms [23]. This design enables more sophisticated patterns like iterative retrieval, where the model can plan for a multi-step retrieval process, retrieve knowledge, and then decide to retrieve more based on the initial results, improving reasoning and multi-hop capabilities [23]. Ironically, the modular flexibility allows for the inclusion of critically active components like a retrieval evaluator. This evaluator, a lightweight model, can assess the overall quality of retrieved documents and trigger appropriate knowledge retrieval actions; in some cases, the system can decide whether to rely on previous retrieval results or to switch to more robust search engines [40]. Furthermore, within modular RAG, to ensure that the model is not forced to incorporate unhelpful information, an approach that introduces the term “self-reflection” in the RAG context, such as Self-RAG and ActiveRAG, creates models that can adaptively decide when retrieval is necessary and can even critique their own generation using special tokens, thereby significantly enhancing downstream task performance [41, 36]. This shift from a system with rigid retrieval on virtually every input to one that can actively control when to retrieve knowledge from the external database is central to modular design, making the system more versatile and user-facing by avoiding the potential generation of unhelpful responses [41, 23].

In summary, the transition from Naive to Advanced to Modular RAG marks a profound evolution in the pursuit of grounding LLMs. While Naive RAG established the core principle of retrieve-then-generate, Advanced RAG strengthened this by optimizing the retrieval and output stages within a strict linear process. Modular RAG marks the next fundamental step forward, moving from a static pipeline to a dynamic and adaptable system that can self-improve, critically evaluate its own process, and determine the best strategy on a case-by-case basis, ultimately aligning further with the future of AI systems that are more intelligent, self-aware, and trustworthy [23].

2.2 Integration Mechanisms for Retrievers and Generators

The integration of retriever and generator components represents the defining architectural challenge in Retrieval-Augmented Generation (RAG) systems. Building upon the evolutionary trajectory from Naive to Advanced to Modular RAG paradigms discussed earlier, the manner in which these two modules interact—whether through simple concatenation, sophisticated attention mechanisms, or learned policies—fundamentally shapes the system’s ability to leverage external knowledge effectively while maintaining coherent generation. This subsection examines the core architectural strategies for coupling these components, tracing the evolution from foundational models to contemporary hybrid designs, and analyzes the critical design choice between treating large language models (LLMs) as frozen black-boxes versus fine-tuning them for enhanced adaptability.

At the foundational level, landmark models such as REALM, RETRO, and ATLAS established

the blueprint for integrating retrieval with generation. While these models are not explicitly provided in the current corpus, their architectural paradigms serve as the essential backdrop for understanding modern RAG designs. The [23] provides a comprehensive categorization of these paradigms, tracing the progression from Naive RAG—which simply concatenates retrieved documents with the query as context—to Advanced and Modular RAG frameworks. A key distinction in integration mechanisms lies in the fusion strategy employed: late fusion, where the retrieval results are used to condition the generator’s output distribution; and early fusion, where the retrieved information is integrated into the model’s internal representations.

A prominent example of early fusion is presented in [54], which introduces a cross-attention-based mechanism. Rather than simply prepending retrieved chunks to the input, a differentiable encoder processes the retrieved neighbors and injects their representations into the decoder through a dedicated chunked cross-attention layer. This architecture minimizes the modification of the underlying autoregressive model, allowing the knowledge to be integrated more directly into the token prediction process. The success of this approach demonstrates that deep, cross-attentional early fusion can dramatically improve performance, although subsequent analysis within the survey literature suggests that many contemporary systems, particularly those built upon existing large language models, move away from such tightly coupled modifications due to computational overhead and complexity [23].

For the exploration of late fusion strategies, the distinction between sequence-level and token-level marginalization provides a fundamental framework for controlling token-level uncertainty. While the specific paper detailing these variants is not included in the provided corpus, the broader discussion of RAG architecture in [23] highlights the trade-offs associated with these approaches. In sequence-level fusion, the generator weights the entire retrieved document per output sequence; in token-level fusion, each token is generated by marginalizing over all the passages retrieved. Token-level fusion offers more granular control but is computationally more intensive.

More recent integration approaches adopt a modular perspective, where the retriever and generator are coupled even more loosely, often through additional adaptive modules that determine *when* and *how* retrieval should influence generation. Self-RAG, presented in [41], introduces an innovative integration scheme via reflection tokens. Here, a single language model is trained to output critical reflection tokens that signal whether retrieval is needed, whether retrieved passages are relevant, and whether the final generation is factually consistent. The integration is self-aware: the generator learns to pay more attention to retrieved contexts conditionally, only when it deems them necessary, thus reducing the negative impact of irrelevant or noisy information. Self-RAG significantly outperforms the standard RAG baseline by allowing the model to adapt on-demand, rather than using a fixed number of passages for every query, underscoring that the integration is no longer a tiling of passage text but a dynamic, context-aware gate.

Similarly, ActiveRAG, introduced in [36], shifts the LLM’s role from a passive knowledge receptor to an active learner. Its novelty in integration also includes a self-reflection mechanism that links associations between retrieved external passages and the LLM’s own memorized knowledge, acting as an auxiliary memory that calibrates the intrinsic model behavior, a concept that will be further explored in the subsequent discussion on memory architectures. It builds on the observation that a model’s own prior knowledge and the retrieved context form a tension. The [55] paper introduces a bridge model as a third module that connects the retriever and the LLM, acknowledging that a presumably “good” retrieval result doesn’t reflect the value the LLM will derive from it. By chaining supervised and reinforcement learning, the bridge model creates a combined suffix, directly addressing the semantic-relevance vs. model-utility preference gap which is a divergence between optimizing human-friendly relevance judgments and assembling a sequence that the LLM can best process.

The architectural choice between treating LLMs as frozen black-boxes versus fine-tuning components has become a dominant consideration. Fine-tuning the entire pipeline is not only expensive, but requires millions of dollars and thousands of GPUs. However, this architecture can unlock significant potential for more nuanced integration. UniMS-RAG [56] unifies source selection, retrieval, and generation into a single token sequence, enabling the effective integration of multiple heterogeneous sources. This demonstrates a significant benefit of training-based integration, but it also requires access to the model weights. For systems where such access is unavailable, approaches like PRCA [57] treat the LLM as a black-box with frozen parameters, but insert a separate trainable adapter between the retriever output and the generator’s input. In this study, a pluggable and reward-driven contextual adapter is then used to refine and select the retrieved information in a spatial and temporal order before the generator processes it. The work in [55] also emphasizes this sequence, showing that fine-tuning a bridge model remains beneficial even when the LLM is only available as a closed-source API. Consequently, full-term performance gains are achieved without any updates to the LLM itself.

The proper choice of integrating mechanisms depends on the model and system constraints. As [58] shows, encoder-decoder and decoder-only models benefit from similar retrieval-augmentation setups, but with crucial differences; they can lose performance when too many documents are retrieved. This insight indicates that the coupling strategy is not independent of the model’s parametric memory. Also, a properly designed integration requires an understanding of whether the model tends to rely on internal priors or on the externally retrieved evidence, a theme further developed in the following subsection on memory [37]. Moreover, extensive experiments across models show that the optimization of the system is best supported by having different training objectives that suit the respective components [25]. A system can significantly benefit from having the architecture and integration mechanisms support techniques for handling conflicts such as those between parametric and non-parametric knowledge, which shows the power of treating the integration as a modular ecosystem.

Ultimately, this integration aims to build a single pipeline for robust reasoning. The context of this sets up the discussion on memory and generation strategies in the following sections, where a modular component will become progressively more common. These current choices are leading the way to future multi-agent, self-reflective systems.

2.3 Memory-Augmented Architectures and Knowledge Construction

Memory plays a pivotal role in bridging the gap between the static, parametric knowledge of large language models (LLMs) and the dynamic, non-parametric nature of external knowledge sources. While the original Retrieval-Augmented Generation (RAG) framework introduced the concept of a dense vector index as an explicit non-parametric memory [22], the architectures have since evolved significantly. This evolution has moved beyond simple key-value stores to incorporate more complex memory structures, auxiliary rationale memories, and self-reflective mechanisms that actively calibrate a model’s intrinsic cognition, fundamentally transforming how knowledge is acquired, retained, and utilized. Understanding this evolution is essential, as the design of memory directly influences the architectural integration strategies discussed earlier and sets the stage for the black-box versus fine-tuning trade-offs examined in the following subsection.

Traditional memory-augmented architectures primarily rely on external memory structures, such as dense vector indices and knowledge trees, which serve as the foundational layer for many RAG systems. These structures store vast amounts of information in a format that is efficiently searchable by neural retrievers. However, a key limitation of these systems is their static nature; they do not inherently learn or improve from their past successes or failures. In stark contrast to human memory,

which continually updates and refines itself, frozen LLMs paired with a static index do not acquire new knowledge from past interactions [59]. For instance, the concept of explicit read-and-write memory, as proposed in MemLLM, has been introduced to enhance LLMs by integrating a structured and explicit memory module. This allows for dynamic interaction with the memory, which improves the model’s interpretability and its ability to manage stored knowledge effectively, addressing some of the inherent limitations of parametric memorization [33]. Similarly, the notion of a working memory module for decision-making, inspired by human cognition, has been shown to improve training efficiency and generalization by storing, blending, and retrieving information for various downstream tasks [60]. These contrasting approaches—from static retrieval indices to dynamic, structured memory modules—highlight a key trajectory in RAG: creating models that are not just vectors [61] but are capable of a more nuanced interaction between external data and internal reasoning.

A crucial advancement in memory-augmented architectures is linking external evidence with the parametric memory of the LLM. The core challenge here is effectively calibrating the intrinsic cognition of the model, ensuring that it can integrate new, retrieved facts without disrupting its existing knowledge structure. This is particularly important in active learning scenarios where the model must go beyond being a “passive knowledge receptor” and instead actively associate new information with previously acquired knowledge. The ActiveRAG framework addresses this by utilizing the concept of the Knowledge Construction mechanism to develop a deeper understanding of external knowledge. By connecting external data to existing knowledge, ActiveRAG shifts LLMs from passive retrieval to active learning, enabling a more profound assimilation of information [36]. This process of linking new evidence to the model’s internal knowledge, however, is not without risk. A failure to integrate this new information properly can lead to a tug-of-war between the model’s internal prior and the retrieved information, as demonstrated in studies showing that when a document contradicts the LLM’s internal knowledge, the model may either correctly update its answer or be misled [37]; [28, 62, 63]. This highlights the inherent tension between what the model knows and what it encounters, and the need for sophisticated mechanisms to mediate such conflicts.

Another significant area of innovation involves auxiliary rationale memory and self-reflective mechanisms to refine retrieved information and calibrate the model’s intrinsic cognition. The primary concept here is to learn from reasoning processes, not just from the final answers. For example, ARM-RAG proposes learning from historical successes by storing and retrieving entire reasoning chains, demonstrably improving performance on grade-school math problems [59]. Similarly, the concept of an “Artificial Neuron” extends this idea by creating a dynamic feedback loop where incorrect responses are refined and stored in a structured memory, allowing the LLM to reference past interactions and apply learned reasoning strategies to new problems [64]. These approaches highlight the value of storing not just raw facts, but the multi-step solution pathways and rationales themselves.

Self-reflection takes this a step further by making the model the evaluator of the information it retrieves and generates. Recall the discussion of integration mechanisms in the previous section: Self-RAG, introduced there, also exemplifies this memory-based shift. The model is trained to determine when to retrieve, how to generate, and then critique its own output and the retrieved passages using special reflection tokens. This self-reflective ability makes the model controllable and enables it to adapt its behavior to various tasks, significantly improving factual accuracy and citation generation [41]. Similarly, the Self-BioRAG framework applies this concept to a dedicated domain. It was trained not only to retrieve medical documents but also to adaptively retrieve documents if needed, and self-reflect on its generated responses, demonstrating that domain-specific self-reflection can significantly enhance reasoning in high-stakes fields [65]. These self-reflective mechanisms enrich the RAG model from a simple retriever-generator pair into a more holistic and

reliable system. Ultimately, these frameworks, both at the rationale and self-reflection level, serve to not only store explicit memories but also actively shape the model’s thoughts. The next generation of memory systems is more than just a repository of facts; it is a cognitive partner that aids the model in integrating external evidence with its internal knowledge to reach a reliable answer.

In summary, the role of memory in RAG systems has evolved from a simple static index to a sophisticated, self-aware component of the architecture. The progression from dynamic “knowledge construction” mechanisms [36] to the formal introduction of self-reflective memory [41] reveals a clear trend toward systems that can learn, not just retrieve. As the following discussion on black-box and fine-tuned systems will show, the design choices regarding memory management are inherently tied to how the system can be trained. Whether a system treats its LLM as a frozen component or fine-tunes it, the memory integration strategy—especially its adaptivity and self-reflectiveness—remains a dominant factor in system performance. The future of RAG lies in fulfilling the promise of these memory-augmented architectures, moving beyond simple knowledge look-up to a truly integrated system that evolves and learns.

2.4 Design Choices: Black-Box vs. Fine-Tuned Systems and Impact on Performance

Building on the evolutionary trajectory of memory architectures discussed in the previous subsection—where the shift from static indices to self-reflective mechanisms has redefined how knowledge is stored and utilized—the next critical design decision concerns the trainability of the system’s core components. The architecture of a RAG system is fundamentally shaped by whether the LLM and retrieval components are treated as frozen, black-box entities or if they are fine-tuned in a coordinated manner. This choice has profound implications for system performance, adaptability, computational cost, and overall effectiveness of the augmentation pipeline, and it is inherently intertwined with the memory strategies described earlier. For instance, the dynamic, self-reflective memory modules discussed previously often require the system to have access to model gradients or activations for training, while simpler static memory indices can operate entirely within a black-box setting. The dichotomy between black-box and fine-tuned approaches is not merely a technical detail but a strategic decision that defines the system’s potential and limitations, directly influencing the trade-offs between architectural sophistication and deployment feasibility.

The black-box approach, as the name suggests, involves using pre-trained models and retrieval APIs without modifying their internal parameters. This strategy is largely motivated by practicality and resource constraints; the extraordinarily large number of parameters in modern LLMs makes full fine-tuning an impractical and costly endeavor for many applications [66]. Furthermore, model providers often offer LLMs exclusively through APIs without exposing their weights for gradient-based updates [57]. In this paradigm, which works naturally with the static, external memory indices discussed earlier, the system’s architecture focuses on optimizing the interaction between the frozen components. For instance, research shows that a fine-tuned embedding model—even when the LLM itself is frozen—can still be integrated as a trainable auxiliary model to enhance the retrieval stage of a black-box system [67]. This suggests that the black-box constraint primarily applies to the generator and the core retrieval model but can leave the door open for targeted optimization of the retrieval interface—a form of tuning that complements the fixed memory index rather than altering the model’s intrinsic cognition.

When the LLM is treated as a frozen entity, the success of the RAG pipeline depends heavily on the quality of the retrieved and processed information, a dynamic that parallels the reliance on well-structured memory in earlier architectures. Methods therefore must be developed to bridge the gap between the retriever’s output and the LLM’s input requirements. The “reward gap” that exists between these components often results from the retriever not being tuned for LLM-specific

downstream utility [66]. In a black-box system, this gap can be addressed by carefully filtering the retrieved content, ordering the information, or restructuring it to better fit the input format of the LLM, rather than by updating the LLM itself. For example, the use of a self-knowledge recognizer and a sub-document-level token reducer can enable the LLM to be more efficient by only receiving the necessary factual tokens [66]. Similarly, a pluggable, trainable reward-driven contextual adapter can filter and reorder retrieved information to significantly improve a RAG system’s performance without altering the generator [57]. These methods demonstrate that in a black-box setting, the surrounding engineering and optimization of the interface between retrieval and generation—much like the careful design of memory structures—connects significantly to overall performance.

In contrast, the fine-tuning paradigm involves co-adapting the generator and/or the retriever for a specific task, a process that aligns closely with the dynamic, self-reflective memory mechanisms discussed earlier. This approach allows the model to learn a coordinated strategy to bridge a particular “preference gap” between the retriever and the LLM [55]. A framework that chains together supervised and reinforcement learning to train a bridge model that optimizes the connection between a retriever and a decoder-only LLM has been shown to lead to substantial gains in both question-answering and personalized generation tasks [55]. This co-adaptation can involve fine-tuning both the retriever and the generator on task-specific data, much like the “knowledge construction” mechanisms in ActiveRAG that calibrate the model’s intrinsic cognition with external evidence.

The depth of fine-tuning can be further categorized into full-parameter fine-tuning versus parameter-efficient fine-tuning (PEFT). While full fine-tuning of both the retriever and generator can be quite costly, with models like Llama-2 requiring significant GPU resources, PEFT techniques such as LoRA and QLoRA have been developed to scale down computational and memory requirements while maintaining competitive performance [68, 69]. The flexibility and resource awareness of PEFT make possible the adaptation of these systems to constrained on-premises or single-GPU environments, a real-world requirement for many enterprise applications [70]. This resource-conscious approach complements memory-aware architectures that aim to reduce the computational overhead of storing and retrieving large amounts of information, enabling fine-tuning to be a viable path even in environments where full parameter updates are impossible.

The performance outcomes of fine-tuning a model also heavily depend on its goal and the task at hand, echoing the earlier discussion of how memory strategies must be calibrated to the specific knowledge conflict scenarios. When injecting a new domain, fine-tuning can strengthen model parametric memory, while RAG can enrich the context, and the success of each can vary depending on the task. Fine-tuned models often have increased success on the most or least popular groups of entities, while RAG performs better on average [25]. In other words, the choice depends on the domain type and knowledge landscape. For example, in high-stakes financial question-answering, a carefully structured system combining a fine-tuned embedding model with a fine-tuned LLM shows greater accuracy than generic models, while adding reasoning iterations can get closer to human-expert quality [71]. This variability underscores that the selection of a training paradigm and its associated memory design must be informed by the expected knowledge distribution and conflict patterns.

One significant drawback of fine-tuning approaches is that the process of fine-tuning for RAG demands a great deal of quality data generation and fine-tuning continuous hyperparameter grid search. Creating an effective set of in-domain questions and answers to use for model tuning often relies on synthetic data generation from the LLM, which itself needs validation to ensure good quality [72]. Additionally, unsupervised fine-tuning may not be useful for injecting factual knowledge into the LLM, as models struggle to learn new facts this way [30]. This introduces a challenge that contrasts with the memory systems that can be updated asynchronously, as fine-tuning requires a simultaneous, instance-wise data strategy to maintain relevance.

The impact of these design choices on overall RAG system performance is best described as “task-dependent,” a theme that connects directly to the discussion of knowledge conflicts in the previous subsection. RAG can address the “tug-of-war” between internal and retrieved knowledge, but certain models can suffer from “over-reliance” on the retrieved context, causing them to ignore their own parametric memory and potentially cite incorrect information. This sensitivity between what the model knows and what retrieval provides is highly nuanced, and the best design choice can vary depending on the domain and deployment needs [37, 72]. The role of memory strategy in these systems—static, dynamic, or self-reflective—acts as a moderator of this tug-of-war, with self-reflective mechanisms offering the potential to mediate between internal knowledge and external evidence in a more controlled fashion.

The decision between black-box and fine-tuned approaches also affects a system’s robustness to domain-specific knowledge bases. In the black-box setting, the system entirely relies on the retrieval phase to identify the relevant tokens, so the quality of indexing and retrieval becomes the primary determinant of performance—an extension of the importance of the memory structure itself. In contrast, fine-tuning makes the retriever and generator *aware* of the specific domain’s terminology, style, and structure, which can improve semantic similarity matching [29, 73]. The use of fine-tuning for the retriever adds significant quality and accuracy improvements that cannot be achieved with just fine-tuning the LLM [71]. This also extends to enterprise applications; for instance, an application using hierarchical structures, like an enterprise internal document set, demonstrates that fine-tuning a model and combining it with domain-specific ranking can outperform generic approaches used only for retrieval-augmented generation [68].

In summary, the choice between black-box and fine-tuned systems is not a “one-size-fits-all” decision; it is intrinsically linked to the memory architecture—whether static, dynamic, or self-reflective—adopted in the system. For environments with high data privacy and computational constraints, the black-box approach, paired with static memory indices and interface engineering, might be the only viable route; it reduces compute overhead and does not require updating model weights [66, 57]. Conversely, when high quality and domain adaptation are top priorities and computational resources are available, fine-tuning strategies—especially with cost-effective PEFT methods—offer a mechanism to co-adapt and synchronize inner components with the dynamic, self-reflective memory systems to meet the task’s preference gap and foster domain expertise [55, 68]. The analysis of these trade-offs, intertwined with memory system choices, is critical for balancing the design, cost, and performance triangle in real-world RAG deployments [72]. As we proceed, this understanding of the dual dimensions—both memory and training rationale—will remain central to evaluating downstream RAG variants and applications.

3 Retrieval Strategies, Indexing, and Knowledge Sources

3.1 Taxonomy of Retrieval Models in RAG

The retrieval component constitutes the foundational pillar of Retrieval-Augmented Generation (RAG) systems, determining the quality and relevance of the external knowledge that conditions the generative model. However, as highlighted in the preceding discussion, the effectiveness of this component is intrinsically dependent on the quality of the query formulation. Even the most sophisticated retrieval architecture cannot compensate for a poorly articulated or ambiguous query, underscoring the symbiotic relationship between query understanding and the retrieval mechanisms explored herein. A comprehensive taxonomy of retrieval models is essential for understanding the landscape of possible approaches. This subsection systematically categorizes these models, contrasting sparse and dense families, and exploring hybrid and multi-vector strategies, while

analyzing the inherent trade-offs that dictate their suitability for diverse contexts.

The most traditional approach to retrieval involves sparse, lexical methods, with BM25 serving as the archetypal representative. These models operate on exact term matching between the query and the document, creating sparse vector representations where the dimensionality is defined by the vocabulary’s size. The primary advantage of these methods lies in their efficiency and interpretability. They can be efficiently indexed and searched using inverted indexes with off-the-shelf systems, and their logic—based purely on term “overlap” and frequency—is straightforward to understand [74]. However, their reliance on exact “matching makes them particularly susceptible to the vocabulary mismatch problem”, where surfaces are semantically related but do not share lexical tokens. This \serious limitation reinforces the importance of the query expansion and rewriting techniques discussed in the preceding subsection, which attempt to bridge this lexical gap. To counter this inherent weakness, methods like [75] investigate learning augmentations and re-weightings to improve BM25’s performance while retaining its speed advantage, effectively incorporating a learned querying component directly into the classical retrieval pipeline.

Given the limitations of exact lexical matching, the advent of pre-trained language models (PLMs) introduced dense retrieval (DR) methods that capture the deeper semantic intent of both queries and documents. Dense techniques, such as the Dense Passage Retriever (DPR) [76], create low-dimensional, continuous vector representations for queries and documents, computed the semantic similarities even in the absence of lexical overlap. This directly addresses the vocabulary mismatch problem, a key motivation for LLM-based query enhancement. However, [77] highlights the challenges of training such models, proposing a technique that performs full retrieval without sampling, mitigating imbalanced negative sampling that can degrade performance. “This risk is particularly acute in RAG pipelines where the generator depends the retriever’s recall, making robust mini-batch training critical.” While potentially more effective at capturing intent, dense retrievers face significant efficiency challenges, as they require the construction of approximate nearest neighbor (ANN) indexes, which trade retrieval performance for speed [78]. Their limited ability to generalize to unseen domains, often resulting in significant performance degradation, further underscores the need for robust query understanding and contextualization as described in the prior part of this framework [79].

To bridge the semantic power of neural models with the efficiency of classical lexical methods, *learned sparse* models have emerged as an effective middle ground, particularly useful in scenarios where computational resources are constrained. SPLADE [80] is a primary example, showing how sparse representations with high sparsity can achieve state-of-the-art results while remaining compatible with inverted indexes. A unified framework for learned sparse retrieval [81] analyzes that including document term weighting is central to a method’s effectiveness, and that query expansion can have a cancellation effect with document expansion, sometimes impacting efficiency without improving effectiveness, effectively by both retrieval rooms and query-enhancement modules in their control design. Further advancing this approach, Multimodal LSR works, such as [82], introduce techniques to handle high-dimensional co-activation and semantic deviations.

Another significant family is that of multi-representational or multi-vector models, which rose to prominence with ColBERT [83]. These models are architected for extraordinarily fine-grained interactions, using contextualized token embeddings of queries and documents to achieve term-level precision. In a specific analysis of dense retrieval families [84], the effectiveness of these multi-vector inputs is highlighted, showing how they often outperform single-representation models for complex information needs, although at a significant cost in storage and retrieval latency [85]. To combat these resource heavy requirements, models such as [86] have been developed to reduce computational load while maintaining accuracy. Furthermore, work like [87] and [88] improves practical usage by converting token embeddings into sparse lexical fields, enabling the use of standard inverted indexes

to significantly cut storage and query time costs.

Given the distinct strengths of lexical and semantic approaches on their own, retrieval enhanced by hybrid strategies aims to combine the best of both worlds. A unified paradigm can be seen in [89], which merges dense and sparse representations within one model. Many systems achieve new robustness by explicitly employing a hybrid framework with out-of-domain datasets [90], showing that combining them can lead to improved adaptability. For instance, the approach of [91] demonstrates that a hybrid model combining lexical and dense approaches significantly improves retrieval performance, showing their complementary nature, especially in specialized domains. Another hybrid direction of note involves converting sparse vectors into a dense representation via slicing [92]. This elegant approach allows a single model to generate both sparse and dense representations, seamlessly integrating lexical and semantic retrieval modes.

Evaluation considerations also differ across these retrieval families. [93] provides a framework for comparing systems beyond quality metrics, including storage costs, query latency, and system readiness and, crucially, how this readiness interacts with the query-enhancement techniques upstream. Index storage is a constant bottleneck for dense and multi-vector retrievers, leading to methods that shrink the footprint, such as [94], which reduces the number of stored vectors per document to maintain effectiveness with a reduced memory footprint. A unified evaluation toolkit like [95] underscores that efficacy must be evaluated on both in-domain and zero-shot tasks to fully assess a model’s ability to generalize. Choosing the right retrieval family ultimately requires a holistic consideration of cost-performance trade-offs in the context of the entire RAG pipeline. The work on [96] notes that a static choice between sparse, dense, or hybrid approaches may not be optimal; depending on the query type, latency budget, and available resources, an adaptive selection strategy—potentially influenced by the query’s inherent ambiguity or complexity—can yield superior outcomes. This insight, in turn, connects to the need for intelligent and flexible query planning and formulation, directly setting the stage for the strategies explored in the following subsection on query understanding and enhancement.

3.2 Query Understanding and Enhancement Techniques

Query understanding and enhancement constitute a critical preliminary step in Retrieval-Augmented Generation (RAG) pipelines, as the quality of the initial retrieval fundamentally constrains the information available to the generator—no amount of downstream sophistication can compensate for a poorly formulated query. This stage acts as the essential bridge between the user’s often ambiguous or under-specified information need and the retrieval mechanisms described in the previous subsection, directly influencing whether sparse, dense, or hybrid approaches can operate at their full potential. The core challenge lies in closing the semantic and lexical gap between the query and the vast, heterogeneous documents within a knowledge base. Historically, Pseudo-Relevance Feedback (PRF) served as a foundational method, but the emergence of Large Language Models (LLMs)—with their capacity for knowledge memorization and multi-step reasoning—has fundamentally altered the landscape. This evolution has shifted query enhancement from purely statistical term-weighting toward generative, intent-aware, and even planning-based formulations, enabling a more dynamic and interpretable understanding that anticipates retrieval needs at a level previously unattainable.

While PRF remains a widely used and effective anchor, modern research has systematically addressed its inherent fragility. The central assumption of PRF—that the top-ranked documents are relevant—is brittle; when initial results are off-target, the borrowed terms can steer the query in misleading directions. This is precisely why a sophisticated strategy for selecting and weighting the feedback signal is essential, as even the most well-designed retriever is powerless in the face of a

corrupted query. Recent solutions therefore focus on two fronts: refining the quality of the feedback set and reinventing the reformulation strategy itself. For instance, pseudo-relevance feedback for multiple representation dense retrieval leverages KMeans-based clustering and IDF weighting on dense embeddings to generate representative feedback terms, avoiding overreliance on the [97] token and improving mean average precision in dense retrieval settings [98]. Furthermore, the Loss-over-Loss (LoL) framework reformulates the classic PRF loss by comparing the effects of different revisions of the same query during training, penalizing revisions that increase feedback signal without improving performance to prevent severe query drift [99]. In parallel, hybrid injection methods such as BlendFilter deliberately blend external retrieval knowledge with the parametric memory of the LLM itself, providing a means to enrich short and content-sparse queries [100]. These strategies make query refinement robust enough to withstand the flaws of the feedback loop itself, ensuring that the subsequent retrieval and indexing stages receive actionable signal.

Beyond refining PRF, the ML paradigm has opened the door to a fundamentally different approach to expansion, leveraging the generative capabilities of LLMs to produce inherently more semantic and concept-dense terms. Techniques such as Zero-shot and Chain-of-Thought (CoT) prompting allow LLMs to decompose a query step-by-step, providing a larger and more contextually rich pool of expansion terms [101]. In contrast to PRF, which draws from the potentially noisy entire documents, generative relevance feedback (GRF) constructs the query signal from the LLM-generated passage focused on the intent, delivering up to 19% improvements in MAP and 24% in NDCG@10 over RM3 on several benchmarks [102]. The query2doc method further improves this by asking the LLM to generate pseudo-documents that emulate relevant passages, showcasing 3-15% to gains in BM25 performance [103]. These methods showcase how the synthesis of textual expansions can align the query with the semantic space of the document. Furthermore, relevance-aware building blocks such as Generative Relevance Modeling with RASE aim to refine the expansion signal using a reranker’s score to filter out hallucinated or irrelevant expansions, achieving +6-9% MAP and +2-4% R@1k [104]. This step implicitly connects to the importance of rerankers—which will be examined in the following subsection—yet its primary contribution lies in ensuring query ‘expansion terms are of high fidelity and directly usable by sparse and dense retrievers.

While expansion enriches the query’s context, query rewriting aims for a more holistic transformation, making the query a better start point for the interaction with the retrieval engine. Relying on prompting is not sufficient for adaptation, and this line of work focuses on tailoring the language model itself. The Retriever’s Preference Optimization (RPO) approach collects 410K query rewrites across 12K conversations and fine-tunes a smaller LM to align with retriever preferences [105]. In parallel, the Rewrite-Retrieve-Read pipeline proposes a hybrid strategy: an initial zero-shot prompt with an LLM for reformulation, combined with a trainable rewriter. This rewriter is trained via a Reinforcement Learning loop that obtains feedback weight from the reader to optimize the entire RAG loop, delivering consistent improvements on open-domain QA benchmarks [106]. The value of rewriting is most prominent in conversational and multi-turn sources of complexity where the query consolidates the conversational information into a single searchable statement.

For complex, multi-hop knowledge needs, query understanding goes beyond single-turn generation, evolving into an explicit **query planning and decomposition** approach. The Search-in-the-Chain framework demonstrates this by guiding the LLM to generate a Chain-of-Query, segmenting the original user query into intermediate, atomic sub-queries. These sub-queries are then sent to an IR system to obtain verification and evidence at each node, enabling the LLM to iteratively correct the reasoning trajectory based on the retrieved signals, rather than adhering to a fixed plan [107]. Similarly, ProTIP adopts a progressive retrieval strategy for tool utilization: instead of explicitly decomposing a task, constitutes rate to use **hierarchical** scenarios. By using contrastive learning without explicit hierarchical labels to maintain the subtask-tool atomicity, ProTIP achieves a 24%

improvement in Recall@K=10 for tool retrieval and a 41% enhancement in tool accuracy [108]. These techniques illustrate how the enhancement phase gradients into planning, generating a sequence, tree, or graph of actionable queries that are tuned to first retrieve the knowledge to generate robust chain-of-thought reasoning.

The last necessity to address the diversity of languages and modalities, where the raw query may not be in a target document’s representation often requires **cross-lingual adaptation**. This work often involves translation with intelligent underpinnings. For instance, intermediate-language translation is used in retrieval systems that also incorporate embedding-based expansion and user-profile tokens to ensure the semantic fidelity through the multilingual process [109]. Simultaneously, cross-lingual query generation has emerged, which improves the Augmented Passage Representation: rather than matching in the query’s original language, the dense retrieval operates when key query translation is also densified to the target language [110]. In this way, query understanding becomes an integral part of manipulating the feature spaces in the global document collection.

The convergence of these techniques marks the transition from query refinement to a broader stage of *query intent formulation*. In summary, the field has evolved not only from purely lexical to be semantic and generative, but also into a *predictive* element that can anticipate the retriever’s challenges and the LLM’s needs. Whether through the robust selection of PRF signals, the generation new text, adaptation of variational rewrites, decomposed queries, or cross-lingual bridging, these enhancements constitute a **quality-determining pillar** for the entire pipeline, because a well-formulated query allows subsequent stages—whether they are building an index, re-ranking candidate documents, or synthesizing the final response—to operate at peak efficiency. In this context, query understanding is no longer a preliminary step but a strategic tool that aligns the users need with the resource capacities of RAG architectures.

3.3 Indexing Structures and Knowledge Source Diversity

The construction and management of large-scale indexes form the backbone of any Retrieval-Augmented Generation (RAG) system, determining both the efficiency of knowledge access and the ceiling of retrieval quality. While query understanding and enhancement ensure that the user’s information need is articulated as effectively as possible, the index determines whether the knowledge base can be navigated at scale and precision to satisfy that need. This process involves a series of critical decisions regarding the representation of knowledge, the structural organization of these representations, and the strategies for maintaining the index in the face of dynamic information. The choice of these elements profoundly impacts downstream generation, influencing everything from the semantic recall of relevant facts to the computational cost of serving queries.

A central design principle in indexing is the **granularity of the retrieval unit**. Traditional methods often index long documents or passages, which, while coherent, may contain a multitude of facts, diluting the relevance signal for a given query. To address this, an alternative more atomic unit, the **proposition**, has been proposed. A proposition is defined as an atomic expression encapsulating a distinct factoid, presented in a concise and self-contained format. This finer granularity in retrieval significantly outperforms passage- or sentence-based methods, yielding more relevant and condensed results for downstream tasks while reducing the inclusion of extraneous information [111].

While proposition-level retrieval provides focused context, it can also increase the volume of index entries. In response, systems often employ hierarchical structures to support **efficient indexing and retrieval**. For instance, in large-scale recommender systems, a category-guided hierarchical clustering scheme can be used to generate a tree of item IDs, allowing the model to first locate an efficient cluster before finding the specific document item [112]. Similarly, the use of a hierarchical and adaptive framework can jointly learn both the document embeddings and the approximate

nearest neighbor search tree, such as an inverted file index, to produce optimized retrieval within a given search distance. This end-to-end learnable index structure ensures that the tree structure is aligned with the learned dual-encoder representation rather than being an afterthought, leading to substantial performance improvements on benchmarks like MS MARCO and TREC DL [113]. Beyond tree-based structures, **learned indexes** architecture models the underlying data distribution to map keys to their positions, offering significant memory savings and reduced search latency. The **LIDER** framework demonstrates the application of a high-dimensional learned index for dense passage retrieval; by combining a recursive model index with a dimension-reduction component such as SortingKeys-LSH, it is able to efficiently navigate the high-dimensional embeddings from a deep neural model, achieving higher search speed than traditional approximate nearest neighbor (ANN) indexes for many workloads while maintaining high recall rates [114]. This showcases how learned indexes capitalize on the data’s distribution.

The diversity of knowledge sources adds another layer of complexity to the indexing process. Beyond unstructured corpora, structured sources such as knowledge graphs carry explicit connections between entities. Query-specific knowledge graphs constructed for the finance domain can be built with state-of-the-art ranking systems, utilizing entity relevance signals that positively correlate with document relevance to improve retrieval. This approach generates graph context to feed downstream processes; however, indexing these graphs faces significant challenges including extracting all relevant entities and relationships, and handling the sparsity and size of complex-topic graphs [115]. Moreover, for **interdisciplinary science**, where scientific principles are intrinsically interconnected, a structure-aware approach builds a heterogeneous document graph that encodes relationships between papers (e.g., citations, co-authorship, disciplinary boundaries) to enhance the pertinence and faithfulness of responses. By treating this structural embedding as an additional input along with raw text embeddings, retrieval-augmented language models can condition on a and more cohesive source of context [116].

In the shift toward **generative retrieval architectures**, the paradigm for indexing breaks from the traditional exact-match vector index. Here, a language model is used to encode the corpus in its parameters and generate a document identifier (DocID) for a given query. Within these frameworks, the choice and design of the identifiers become crucial. For instance, **CorpusLM** introduces a unified sequence-to-sequence model that learns an integrated relationship between the corpus, user queries, and answer generation to support knowledge-intensive tasks [117]. Such **DocID generation** often employs a specially-designed ranking-oriented list generation strategy; however, a major challenge for generative retrieval is scaling to millions—the learning and maintenance of the index structure. Scaling studies on the full MS MARCO collection reveal significant computational bottlenecks, but also show the positive impact of adding synthetic queries to represent documents during the indexing stage, noting that standard architectural modifications become impractical when accounting for cost [118]. To address constraints in the DocID space, alternative **unified** DocID schemes have emerged, where the document ID generation process is reformulated as dot product between query and document vectors, conceptually unifying text generation and dense retrieval paradigms [119].

The indexing challenge extends within mutable corpora. Given the high volume of index updates in real-world scenarios, the ability to **efficiently update the index** is critical. In the context of generative retrieval (e.g., Differentiable Search Indices), new documents must be integrated into the model’s memory incrementally to prevent a catastrophic drop in performance on previously indexed documents—the continual learning problem. One solution introduces a set of modifications, including optimizing for flat loss basins and generating pseudo-queries, to substantially reduce forgetting and achieve stable updates with a **+21.1%** Hits@10 improvement compared to simple fine-tuning on sequential benchmark tasks [120]. Similarly, the **CLEVER** framework integrates

incremental product quantization for generating new docIDs with a memory-augmented learning mechanism to connect old and new documents, demonstrating that efficient, continual adaptation is feasible when combining proactive compression strategies with network learning [121].

Finally, a significant modern departure is to enrich the index with additional context. In this area, LLM-based strategies for query generation can ensure that fewer relevant items are missed. **Pre-training with LLM-based Document Expansion** uses an LLM to produce a set of queries for a document and then pre-trains the retriever to map those expansions into the query representation, leading to strong zero-shot and out-of-domain transfer capabilities [122]. On the other hand, **DSI-QG** addresses distribution mismatch by indexing documents with pseudo-queries generated from an external LLM, and then re-ranking and filtering those queries to connect each document identifier to relevant query. This bridging method substantially improves the retrieval performance of the DSI architecture, both across languages [123].

As the field evolves, it moves toward **targeted, decomposed, and iterative** index creation. A comprehensive vision even proposes the index as an additional module within a **large language model**, with generation and retrieval formulated as a unified, autoregressive text generation problem [124]. Ultimately, the convergence of these approaches—choosing the optimal retrieval unit and eliminating noise; augmenting the index with generated knowledge; and building structures that adapt to both the model and the semantic constraints—is essential for the next stage of RAG systems. The choice is inherently tied to the type and scale of the knowledge source, the specific retrieval architecture, and the dynamic frequency of updates to the underlying data. As these findings show, this remains a vibrant and open problem that significantly impacts the entire RAG pipeline, from indexing to generation.

4 Generation Techniques and Context Integration in RAG

4.1 In-Context Learning and Demonstration Selection in RAG

In-Context Learning (ICL) has emerged as a powerful paradigm that enables large language models (LLMs) to adapt to new tasks without parameter updates, simply by conditioning on a few task-specific demonstrations prepended to the input. This capability is particularly crucial for Retrieval-Augmented Generation (RAG), where retrieved documents and demonstrations are jointly structured into prompts to condition the generation process. However, a fundamental challenge lies in the fact that LLMs are highly sensitive to the choice, order, and composition of these demonstrations, with performance varying widely across different configurations [125, 126]. Therefore, the selection and structuring of demonstrations represent a critical bottleneck in the success of RAG systems.

Given this sensitivity, the majority of research in demonstration selection has moved beyond random sampling towards more principled, query-specific methods. The most intuitive strategy is **semantic similarity**, where a text retriever (e.g., a dense retriever like DPR or a lexical one like BM25) selects examples from a candidate pool that are most similar to the test input. Retrieving demonstrations for each test query individually improves performance over using a fixed set across a wide variety of tasks [127]. Notably, the effectiveness of such retrieval-based selection is robust to the retriever choice; simple word-overlap measures like BM25 can outperform random selection, demonstrating that even basic lexical alignment boosts ICL [128].

However, further research suggests that semantic similarity alone remains sub-optimal, as it neglects the target task’s intrinsic structure and the LLM’s own internal knowledge. Moving beyond the selection of individual examples, a recent paradigm emphasizes the interdependence within demonstration sets by introducing the concept of “Comparable Demonstrations” (CDs), which are

constructed by minimally editing the texts of the examples to flip their output labels, thus forcing the LLM to focus on the essential input-label mappings rather than spurious inter-example correlations [126]. This method excels in out-of-distribution scenarios where such shortcuts usually cause failure. Similarly, another line of work looks into the non-additive nature of demonstrations, focusing on the *collective* value of a set rather than which individual examples to pick. Understandably, a select-then-organize approach that treats instances in isolation often ignores the inter-example relationships [129]. Addressing the unknown nature of what makes a good example, **Coverage-based Example Selection** [125] designs a BERTScore-based score that selects examples capturing the most salient reasoning patterns present in the test input, often proving superior to standard retrieval metrics on many tasks.

Initially motivated by the principle of sample efficiency (i.e., improving the performance without overwhelming the context), the demonstration selection process has also looked to pick out “informative” examples for reasoning tasks in RAG: observing that LLMs are more receptive to certain formats, **Reasoning Skill Discovery (RSD)** improves prompting by learning a latent skill space from rationale examples to guide the selection, making the process grounded in theoretically-informed skill classification [130]. Similarly, the active learning philosophy proposes to choose the most *uncertain* queries for annotation, which often “distills” more informative knowledge for new tasks [131]. This synergy turns example selection itself into a structured, adaptive process, rather than a static retrieval step.

Beyond demonstration selection, the well-established sensitivities of ICL—including order and label distribution—also directly impact RAG performance. The label bias, or demonstration shortcut, is a significant hurdle: LLMs might rely on output patterns from the examples instead of the *input-label mapping*. Consequently, **In-Context Calibration** proposes a response rectification method, which explicitly aims to eliminate the pre-trained semantic priors, allowing the model to generalize correctly [132]. Research on few-shot counts yields additional insights: on standard datasets, a single “correct” demonstration often outperforms providing all instances, indicating that including redundant or low-quality examples can introduce biases, and negatively influence the model’s ability to generalize [133].

To fully utilize the LLM’s intrinsic knowledge, novel approaches for generating demonstrations on the fly have emerged, departing from querying external repositories of human-crafted examples. The **Self-ICL** framework prompts the LLM to generate pseudo-inputs and predict pseudo-labels, thus bootstrapping in-context learning without any human-annotated demonstrations [134]. Similar in purpose, Self-Demos generates query-correlated pseudo-demonstrations to bridge the gap between few-shot and complex or out-of-distribution queries [135]. These approaches effectively turn the LLM itself into a source of context, serving as a promising and practical alternative to external data pools.

The next level of sophistication in RAG marks a departure from the static “retrieve-then-read” pipeline, wherein the LM and retriever are orchestrated into dynamic, multi-step reasoning chains. **Demonstrate-Search-Predict (DSP)** is a foundational framework that expresses high-level programs, using natural language texts in sophisticated pipelines between an LM and a retrieval model to systematically decompose problems into smaller transformations, achieving new state-of-the-art results in open-domain, multi-hop, and conversational settings [136]. Similarly, frameworks like Self-Ask take a focused planning approach with follow-up questions to guide retrieval. As a demonstration of the strong interplay between reasoning and retrieval, agentic planning frameworks such as **Search-in-the-Chain** [107] go further by generating not just answers but also intermediate reasoning steps and then validating them for correctness via the retriever, thereby improving both accuracy and traceability. Collectively, these retrieval-augmented prompting strategies present a strong departure from simple prompting, enabling LLMs to execute multi-step reasoning, plan their

next retrieval, and correct their own mistakes, forming the core of reliable and adaptable RAG systems.

4.2 Instruction Tuning and Grounded Response Generation

Instruction tuning has emerged as a pivotal technique for enhancing the capabilities and controllability of large language models (LLMs), bridging the gap between the models’ next-word prediction objective and the users’ objective of having LLMs adhere to human instructions [137]. In the context of Retrieval-Augmented Generation (RAG), where the preceding discussion has established the critical role of demonstration selection and dynamic reasoning pipelines, instruction tuning plays a complementary role in enabling models to effectively utilize retrieved evidence and produce grounded responses. While the previous section demonstrated how demonstrations and retrieval structures guide the model’s attention toward relevant information, this subsection shifts focus to how explicit instructions, task formulations, and parameter updates can further align the model’s generation behavior with the retrieved evidence, thereby ensuring that RAG systems not only retrieve and process information effectively but also generate outputs that are faithful, controllable, and aligned with user intent. This subsection examines the instruction tuning methods designed to enhance grounded response generation, covering the interplay between in-context learning (ICL) and instruction tuning, the use of natural language instructions to steer outputs, techniques for improving factual grounding and citation generation, and the trade-offs between parametric and context-based knowledge.

The synergy between in-context learning and instruction tuning is the most fundamental layer connecting this subsection to the preceding one. ICL and instruction tuning are the two primary paradigms for adapting LLMs to the downstream applications discussed previously, operating in fundamentally different ways: ICL provides demonstrations at inference time without parameter updates, while instruction tuning uses demonstrations to update parameters during training [138]. Research has sought to uncover the relationship between these paradigms, examining how hidden states of LLMs change under each. Notably, findings suggest that ICL can be viewed as a form of implicit instruction tuning; ICL changes an LLM’s hidden states as if the demonstrations were used to instructionally tune the model [138]. This convergence is of direct relevance to RAG, as the demonstration selection strategies detailed earlier—such as coverage-based selection or comparable demonstrations—can be understood as implicitly shaping the model’s grounded generation behavior in ways that complement explicit parameter updates. Furthermore, the integration of natural language demonstrations with learned prompt embeddings—a method known as instruction prompt tuning (IPT)—has been proposed to combine the benefits of both approaches [139]. However, empirical studies reveal that IPT does not consistently outperform basic prompt tuning and requires semantically similar demonstrations to yield improvements, highlighting the nuanced interplay between these techniques [139]. Additionally, the sensitivity of causal language models (CausalLMs) to the order of in-context examples—which directly relates to the order sensitivity issues raised regarding demonstration selection—presents a challenge for stable performance in ICL-based RAG, with variations in attention masks contributing to this issue [140].

A key advantage of instruction tuning is its ability to improve sample efficiency and generalization across tasks, complementing the adaptive selection of context from the previous section. In the SuperNI benchmark, instruction-tuned models using less than a quarter of downstream training data can surpass state-of-the-art fully supervised models, demonstrating that instruction tuning significantly aids in both sample efficiency and transfer learning for unseen tasks [141]. This is highly relevant for RAG, where the ability to quickly adapt to new domains or knowledge-intensive tasks—often with limited task-specific annotated data—is crucial. However, existing instruction-tuning

methods often learn the correlation between instruction-formatted samples and target labels, rather than true causal relationships. A meta Structural Causal Model has been proposed to integrate NLP tasks under a single causal structure, aiming to learn the task-required causal representations that can predict the target labels for a given task [142]. Moreover, instruction-tuning can inadvertently introduce cognitive biases, such as the decoy effect, the certainty effect, and the belief bias, which is especially critical for RAG outputs where the model must reason over retrieved evidence and reach sound conclusions [143]. These biases highlight the need for careful evaluation and calibration in instruction-tuned RAG systems, complementing the verification mechanisms introduced in the prior section.

Beyond general methodologies, active instruction tuning has been shown to enhance cross-task generalization in a more efficient way via a novel framework based on prompt uncertainty [144]. This approach identifies informative tasks by quantifying the disagreement of model outputs over perturbed prompts, offering a data-driven method to select tasks for instruction tuning. Similarly, instruction diversity is essential for the set of tasks; experiments with symbolic tasks indicate that diversity in the training set—analogue to the diversity of demonstrations in ICL—is essential for generalization to unseen tasks [145]. This suggests that for building robust RAG systems capable of handling various query types, a diverse set of instructive tasks directly complements the heterogeneous range of retrieved contexts.

The construction of high-quality instruction data in RAG is central to grounded generation, leading to various synthetic data generation methods. CodecLM, a general framework for adaptively generating instruction data, uses LLMs as codecs to guide the data generation process, encoding seed instructions into both metadata and decoding content [146]. Additionally, the ensemble-instruct method explores the application of in-context learning to a heterogeneous mixture of language models to generate instruction data, proposing a categorization and simplification of templates and ensembling multiple LM outputs to help select high-quality examples [147]. Since closed-source models can carry risks or usage restrictions, exploring alternative approaches to generate high-quality instruction data is also important [148].

Within RAG, instruction tuning can be specifically applied to align the model’s generation with retrieved evidence, bridging the retrieval-focused methods the previous section with the generation efficiency techniques in the following one. For example, a context-based instruction tuning framework uses the previous context to generate instructions that self-guide the model’s responses in multi-turn dialogue [149], demonstrating its natural ability to adapt to the input and improve performance. In another approach, instruction tuning is used to enhance entity disambiguation (ED) in LLMs by using a stepwise hard-prompting and instruction-tuning, resulting in a model that improves grounding and also shows consistent accuracy on entity disambiguation tasks and solving a variety of QA tasks [150]. This kind of adaptation is important for grounding in large knowledge bases, which often present the context overload challenges described in the following subsection. However, careful examination of instruction tuning has been raised, as some approaches may not gain performance through genuine instruction comprehension but rather by picking up superficial patterns [151]. It is critical to separate when models are truly learning to follow instructions for grounded generation versus learning task surface patterns—a distinction that directly impacts the trustworthiness of RAG outputs.

The use of task descriptions to steer LLM behavior is integral to RAG. For example, the Guideline-Learning (GL) framework enables LLMs to reflect and follow structured guidelines. It begins with automatically synthesizing a set of guidelines based on a few error cases, and during inference, it retrieves helpful guidelines for in-context information extraction [152]. This framework directly builds on the retrieval and selection strategies discussed in the previous section. Similarly, known works demonstrate that especially for generating grounded content, instructions can be

optimized in such a way that the generalization ability to unseen tasks improves [153]. Furthermore, instruction tuning across a more diverse set of tasks improves generalization to unseen tasks and across multiple benchmarks, as demonstrated in the OPT-IML setup of instruction-following and positive transfer [154]. From a cognitive and behavioral standpoint, instruction tuning has been shown to align with human brain activity, exhibiting closer patterns in world knowledge and reasoning, which could be important to get more human-aligned grounded answers [155]. Instruction tuning can also serve as a self-improving mechanism for knowledge-grounded dialogues, where a model generates intermediate steps not relying on the ground truth [156].

Additionally, instruction tuning has been used for automatic evaluation of natural language generation with learned metrics that emulate human judgments for a particular task and evaluation criterion [157]. Since the central the evaluation of factual consistency of RAG outputs is of paramount importance, this type of instruction-based evaluation is noteworthy. For instance, the TRUE meta-approach has been standardized across 11 grounding datasets, demonstrating that large-scale NLI and question generation-and-answering approaches achieve strong and complementary results for factual consistency detection [158]. These instruction-based evaluation metrics for factuality are an emergent path for assessing the grounding of generated replies, connecting directly to the not just downstream faithfulness but also the context of the next section.

Techniques for improving factual grounding have also been studied, e.g., controllable grounded response generation (CGRG), where lexical options and elicitation strategies are controlled is either supplied by a user or automatically extracted by a predictor from dialogue context and grounding knowledge [159]. This allows the model to avoid generation hallucination. Likewise, taking a simpler approach, large pre-trained language models can leverage store knowledge to ground open-domain dialogues, and fine-tuning with knowledge-grounded dialogues can yields the state-of-the-art outputs [160]. However, this may not require external knowledge. When knowledge is somewhere internal, instruction tuning might change how the model allocates grounding. A study with fake wiki-like data shows that when a model is trained to use target facts, it can use the fact, but the interaction between internal parameters and prompts is unclear [161]. In real RAG systems, the balance between parametric and contextual knowledge is dynamic, and instruction tuning may shift this balance in its own way.

Finally, strategies such as self-refinement and iterative tuning are important to improve not only the factuality actually but also the utilization of context. Systems exist that steer LLMs to verify the correctness of given examples and use the verification results to elicit better answers, demonstrating improved performance on in-domain and out-of-domain tasks [152].

In summary, instruction tuning is a critical enhancement technique for building RAG systems with grounded responses. It works synergistically with the context selection and retrieval-augmented prompting strategies detailed in the previous section to enhance adaptation, to expresses explicit and explicit natural language guidelines to steer generation, and it supports improved factual grounding and evaluation. The careful construction of training data and instruction-based adaptations is key to overcoming the biases and pitfalls highlighted, ensuring robust, grounded, and trustworthy outputs. As RAG systems move toward the efficiency-driven architectures and compression techniques described in the next section, instruction tuning, thus also ensures that the models’ skills and behaviors are resilient and efficient when the model capacity is restricted. Future research must address how to best combine these instructional signals, parametric memory, and contextual evidence—along with attention and positional encoding—to enable more the capable, efficient, and trustworthy RAGs.

4.3 Context Compression, Fusion Strategies, and Attention Efficiency

The integration of retrieved knowledge into the generation phase of Retrieval-Augmented Generation (RAG) systems presents a fundamental tension: while providing more context can enhance factual accuracy and reasoning capabilities, it also introduces significant computational overhead and can paradoxically degrade model performance. This challenge is particularly acute when models are required to process long or multi-document contexts, leading to issues such as the “lost-in-the-middle” problem and the proliferation of memory-hungry key-value (KV) caches. Unlike the trustworthiness-focused techniques discussed in the previous section—which address conflicts between retrieved evidence and the model’s internal prior—the methods examined here tackle the complementary problem of making context integration both efficient and effective. This subsection explores the principal techniques designed to achieve this balance, focusing on context compression, advanced attention mechanisms, and architectural strategies that enable scalable yet faithful grounding; these methods ultimately strengthen the reliability of RAG systems by ensuring that the most relevant evidence is both accessible and retained during generation.

A primary line of research addresses the computational cost and cognitive overload of long contexts through **context compression**. The goal is to reduce the token count of retrieved passages while preserving the information crucial for the downstream task. This can be achieved through extractive or abstractive methods. Extractive compressors aim to select the most informative sentences from retrieved documents. For instance, introducing Selective Context, which identifies and prunes redundancy in the input context, reduces memory cost and inference time while maintaining comparable performance [162]. Similarly, the “Deleter” paper proposes a fully unsupervised model that discovers an “optimal deletion path” for a sentence, using a pretrained BERT to score candidate deletions based on perplexity, achieving competitive results with supervised models [163]. More directly relevant to RAG, RECOMP introduces compressors that generate textual summaries to be prepended to the LM’s input, with an extractive variant selecting useful sentences and an abstractive variant synthesizing information, all trained to improve end-task performance [164]. A key feature is selective augmentation, where the compressor can output an empty string to filter out noise entirely [164].

Another form of compression is the use of **soft prompts or memory slots**. The In-context Autoencoder (ICAE) proposes a lightweight method to compress a long context into short compact memory slots that can be directly conditioned on by the LLM for various purposes [165]. By training an autoencoder with language modeling objectives, it can generate memory slots that represent the original context, achieving 4× compression based on Llama [165]. Similarly, **LLoCO** combines context compression with parameter-efficient finetuning (LoRA) to extend the effective context window of a 4k token LLaMA2-7B model to handle up to 128k tokens, enabling an LLM to create a concise representation of the original context and efficiently retrieve relevant information [166]. At a meta-level, **LongLLMLingua** studies prompt compression in long-context scenarios, showing that the performance of LLMs is sensitive to the density and position of key information in the input, and proposes methods to improve LLM perception of the key information, addressing computational cost, latency, and performance degradation [167].

Beyond token-level compression, the **underlying architecture and positional encoding of the Transformer itself** plays a crucial role in how context is processed, and is central to mitigating the “lost-in-the-middle” challenge, where models fail to identify relevant information in the middle of a context window. One hypothesis is that this issue stems from positional biases, such as those introduced by Rotary Positional Embeddings (RoPE). To counter this, **Multi-scale Positional Encoding (Ms-PoE)** has been proposed as a plug-and-play method that uses different scaling ratios on position indices to relieve the long-term decay effect, forming a multi-scale fusion

from short to long distances [168]. Others have hypothesized that the issue is due to a lack of explicit supervision during long-context training. Information-intensive (IN2) training is a data-centric solution that uses a synthesized long-context QA dataset to teach the model to be aware of fine-grained information in short segments placed anywhere within a long context [169]. Beyond decoding time, a specialized supervised fine-tuning stage that incorporates paraphrasing information into training samples can improve the model’s retrieval capabilities under long-context scenarios [170].

To manage the **KV cache**, which grows linearly with sequence length and becomes a major system-level bottleneck in RAG generation, recent methods propose compression techniques. The GEAR framework combines three synergistic techniques: quantization applied to the majority of entries, a low-rank matrix to approximate the quantization error, and a sparse matrix to handle outlier entries, achieving near-lossless high-ratio compression [171]. A similar philosophy is presented in MiKV as a reliable cache compression method that preserves details by retaining the evicted KV pairs at low precision and keeping important pairs at higher precision to safeguard coherence [172]. More dynamic methods, like **SnapKV**, exploit the fact that each attention head consistently focuses on specific prompt attention features during generation. By identifying these important positions, the model can automatically compress KV caches, achieving a $3.6\times$ increase in generation speed and an $8.2\times$ increase in memory efficiency [173].

Moreover, the structure of attention modules can be optimized. A newer approach, **SqueezeAttention**, moves beyond token-level sparsification and first identifies the most important attention layers. By recognizing that layer importance matters, the authors allocate different budgets to different layers on the fly, achieving substantial memory reductions and throughput improvements [174]. Another approach, **LoMA**, introduces **Lossless Compressed Memory Attention**, which aims for a lossless compression of the KV cache resulting from a specialized training or fine-tuning process, thereby reducing memory and computation pressure during autoregressive decoding [175]. A distinct approach, implemented in **StreamingDialogue**, compresses long dialogue history into special “conversation attention sink” (EoU tokens) with minimal degradation [176], reducing computational complexity quadratically with the number of utterances and allowing effective processing of long dialogues [176].

System-level innovations also contribute to attention efficiency. For example, **Prompt Cache** proposes to reuse attention states across different LLM prompts, given that they have overlapping text segments (e.g., system prompts). Through a schema that defines reusable text segments, they implement a cache that streams segments, significantly reducing latency without modifications to the model parameters [177].

Finally, **use of memory mechanisms in the Transformer** can offer a more flexible approach. Architectures like **GMAT** augment sparse Transformer blocks with a dense attention-based “global memory” of constant length (M), providing an aggregate global view of the entire input sequence, and it can also be used for sequence compression [178]. An alternative is to maintain a FIFO memory of past chunks, where eviction policies (e.g., LRA, LFA) are applied to reduce memory usage, as explored in the Attendre layer, which enables wait-to-attend in long-context processing [179].

Collectively, these methods—from token-level compression and optimized positional encoding to KV cache quantization and memory-based augmentation—form a necessary toolkit for the practical realization of RAG. By strategically reducing the data and memory load, they not only push the boundaries of context length but also, critically, allow the model to maintain the “situational awareness” needed to reason effectively and focus its limited capacity on the most relevant portions of the evidence. This efficiency, in turn, supports the credibility and faithfulness goals discussed in the following subsection: systems that can focus on crucial parts of a long context are better

suited to attend to contradictory, noise, and truly adhere to the grounding sources, rather than being distracted by irrelevant retrievals. The choice of the optimal fix is highly dependent on the task, constraint, and architecture. The field is moving toward hybrid strategies, where selective compression, layer-wise optimization, advanced attention, and system-level reuse are combined to achieve scalable and efficient RAG systems, ultimately contributing to the overall trustworthiness and utility of the generation process.

4.4 Faithfulness, Reasoning under Conflict, and Output Control

The integration of retrieved knowledge into the generation process, while powerful, presents a critical challenge: ensuring that the final output is not only accurate but also faithful to the provided evidence. While the previous subsection focused on making context integration efficient and scalable, this subsection addresses the complementary problem of trustworthiness—how to ensure that models, once given access to relevant information, actually and reliably use it to produce grounded outputs. We explore strategies for navigating conflicting evidence, mitigating inherent model biases, controlling factuality, and aligning outputs with evidence through advanced decoding, calibration, and self-refinement techniques.

A primary concern in RAG systems is the presence of knowledge conflicts, where external evidence contradicts the model’s internal parametric knowledge. Understanding the dynamics of this “tug-of-war” is foundational to building robust systems. Research has systematically analyzed this tension, finding that while providing correct retrieved information can fix many model mistakes, the model’s reliance on its internal prior—its pre-trained biases—significantly influences whether it accepts new, conflicting evidence [37]. This is rejected particularly when the model’s prior is strong, making it resistant to updating even when correct information is presented. Conversely, with a weak prior, the model can easily be swayed by corrupted retrieval passages. This underscores that retrieved context does not always deterministically guide the generation, especially in realistic scenarios where incorrect parametric facts are updated with conflicting documents [28]. Specifically, the model’s pre-existing knowledge biases can hinder retrieval-augmented adaptation when a contradictory parametric fact is present in context [28]. This reality is further illustrated by real-world challenges, such as when the model’s parametric knowledge biases its understanding of context, preventing the knowledge update [28].

A primary goal of RAG is to mitigate the hallucination problem, which is a central motivation for their development. When state-of-the-art LLMs are combined with RAG, they can still struggle with factual robustness if the external prompt conflicts with their internal memory without clear superiority in the conflict [180]. The **Credibility-aware Generation (CAG)** framework is a direct attempt to teach models to discern and process information based on its credibility, thereby enhancing their robustness against noise and misinformation [35]. This approach explicitly trains models to assess and use the credibility of retrieved documents: if the context has relevant yet noisy information, the model should put more trust in high-quality, credible sources [35]. This idea of discerning information quality is directly complementary to the context compression techniques discussed earlier—while compression techniques aim to prune noisy sentences, credibility-aware generation aims to down-weight them, providing a finer-grained, more robust alternative when pruning is not feasible [35].

In the context of the generation decoding process, the probability of the output should be calibrated. A method called **Decoding by Contrasting Layers (DoLa)** proposes a decoding strategy to reduce fact-conflicting hallucinations: the next-token distribution is obtained by contrasting the differences in logits from later and earlier layers [181]. It leads the model to select the tokens that are factually more truthful, reducing its amount of hallucination [181]. In

knowledge-grounded dialogue generation, the incorporation of sequence-level certainty demonstrates a significant correlation with the level of hallucination [182]. Given that, a method called **Certainty-based Response Ranking (CRR)** uses either probabilistic or semantic certainty scores to encode the response, and ranks different candidate outputs to reduce hallucination in the final answer [182]. As a further technique that provides a self-verification loop, the **Chain-of-Verification (CoVe)** method proposes to (i) draft an initial response; (ii) plan verification questions; (iii) answer those questions independently; and (iv) then generate the final verified response, which reduces hallucinations in a variety of tasks [183]. This method can be used to detect whether the answer generated is trustworthy by integrating retrieved-augmented knowledge.

A distinct and statistically grounded approach to ensure trustworthiness in RAG is through conformal prediction. The **CONFLARE** framework applies conformal prediction to the retrieval process, thus guaranteeing that the correct answer is captured within the retrieved context with a $(1-\alpha)$ -confidence level [184], accommodating user-adjusted error rates in areas such as historical books [184]. Similarly, the **TRAQ** framework provides a statistical guarantee on the final answer correctness by utilising conformal prediction for the *complete* RAG pipeline [185]. The conformal risk framework was later formalized in the form of the **C-RAG** methods, which provides an upper confidence bound (i.e., conformal generation risk) for the “good” and “bad behavior” of RAG models [186]. This includes providing the theoretical guarantee that if the quality of the retrieval model and the transformer is non-trivial, RAG will achieve a lower conformal generation risk than a single LLM [186]. When confidence is low, these risk-based frameworks can trigger a fallback mechanism, such as abstention or a request for more information, thereby contributing to a trustworthy system under strict requirements. This connection to querying the user for more evidence also foreshadows the adaptive retrieval and update strategies discussed in the following subsection [187].

Beyond verifying the final answer, ensuring the model’s reasoning process is sound and not self-contradictory is also vital. The **RCoT** method tackles factual inconsistency in the reasoning path by generating verification questions, and then using the differences between the original and reconstructed answers to guide a rectification step [188]. Used in combination with the autoregressive generation of the final answer, this method can detect and rectify factual inconsistencies (i.e., misconceptions, and question misinterpretations) for the output [188]. Another critical challenge is the presence of **self-contradictory reasoning**. Evaluating models for such clues helps to expose logic failures or, in the context of chain-of-thought, a clear gap in the world where higher accuracy does not correspond to a lower self-contradiction rate [189]. This failure cannot be detected via traditional performance metrics like accuracy alone [189]; even state-of-the-art models such as GPT-4 struggle to recognize misaligned logic [189]. This emphasizes the need for methods that make the reasoning process more explicit. In the field of fact-checking, the **Multi-Agent Debate Refinement (MADR)** framework leverages multiple LLMs as agents, each with a specific role in an iterative refinement loop, to generate faithful explanations that are more aligned with the provided *evidence* [190]. This process significantly improves the faithfulness of the generated explanation. Adding to this, another approach to generate more grounded responses is the method of **Self-Endorsement**, which leverages fact-level, fine-grained self-consistency. Fact, or rather a knowledge *atom*, the model weights the facts that multiple sampled versions agree on, rather than taking a whole-sentence majority vote [191]. The Self-endorsement method makes clear the promising approach of self-consistency aggregation for improving factual accuracy [191].

One way to improve the trustworthiness of RAG is to design procedures that reduce the **over-reliance** on external sources, recognizing their potential to be distracting. In contrast to having strong evidence or a correctional document, we found that if the retrieved document is of poor quality (e.g., **misleading or noisy**), the model would be more tempted to copy or hallucinate the distracting result when its internal prior is weak [37]. This finding is compatible with the

behavior described in work on “reliance on the prompt is conditional on the model’s uncertainty; if the prior is weak, it will conflict with the provided task instructions” [15]. Self-consistency and uncertainty-aware prompting are crucial here. Rather than simply training the model to generate a fixed-length response, **uncertainty-aware in-context learning** allows the system to provide the ability to adjust its output into abstention or to self-correct when uncertain [192]. When the relevant pieces of evidence are missing from the passages, the model might be reluctant to generate or decide how to resolve the missing information, and providing few-shot examples of such challenging scenarios, through a **Use-More-Examples** approach, can improve robustness [193]. A robust framework needs accurate handling of uncertainty to decide when to abstain, ask for more recent data, or amplify search. Here, adaptive decoding methods can be helpful, such as those that involve retrieving from a counterfactual set or probing the model’s hidden states for hallucination signals [194]. Hallucination can be a result of overloading faith in unreliable chunks; a two-stream feedback approach can detect and rectify such problematic behaviors in many settings RCoT [195]. **To resolve when the model is uncertain of evidence, a new paradigm, as proposed in reconstructing a decomposition framework, can decompose** the question into smaller steps, retrieve for each, and solve accordingly, iteratively strengthening the chain [196].**

It is worth noting the risk of lower generalizability for fixed prompt strategies. Many RAG systems rely on a fallback strategy: if the context is poor, they will ask for the same information in a different form, or request more guidance from the user. However, a potentially more robust approach is to incorporate iterative retrieval and self-feedback. For example, the **Ra-ISF** framework learns to decompose a task and strengthen its answer chain through iterative self-feedback, allowing the system to refine its own retrieval and reasoning steps [42]. By integrating these deliberation and verification techniques—calibrating evidence trust, verifying reasoning, and enabling self-correction—the field moves towards RAG systems that are not only grounded in the most relevant documents but also demonstrably and faithfully trustworthy, a crucial pre-requisite for deploying them in high-stakes environments where evidence integrity is paramount.

5 Training, Fine-tuning, and Optimization Strategies

5.1 Foundations of Fine-Tuning in RAG: Full, Unsupervised, and Few-Shot Paradigms

The optimization of Retrieval-Augmented Generation (RAG) systems fundamentally hinges on the training paradigms applied to its core components: the retriever and the generator. While the architectural design dictates the flow of information, it is the fine-tuning strategy that determines how effectively a system adapts to specific domains, tasks, and knowledge bases. This subsection establishes the core training objectives for RAG, contrasting full-parameter fine-tuning with unsupervised and few-shot paradigms. We delve into how fine-tuning coordinates the retriever and generator, the role of synthetic data and self-correcting examples, and the critical trade-offs between in-context learning, lightweight tuning, and full updates in terms of accuracy and computational cost. This discussion of training strategies at a holistic level naturally sets the stage for the next subsection, which examines how such objectives can be achieved with far fewer trainable parameters through parameter-efficient tuning methods.

The most straightforward approach to adapting a language model for a downstream task is full-parameter fine-tuning (FFT), which updates all the weights of the pre-trained model. In the context of RAG, this allows the generator to be jointly optimized with the retriever, fostering a coordinated learning process where the generator learns to rely on the retrieved context provided by a simultaneously-updated retriever. However, FFT is resource-intensive and can be prone

to instability, as shown by research proposing various stabilization methods, including periodic re-initialization of the classifier layer, which can introduce significant performance gains [197]. Furthermore, while FFT is effective, it often requires substantial task-specific data, and studies have shown that simple fine-tuning of a pre-trained model for few-shot classification tasks can be highly competitive, challenging the need for more complex meta-learning approaches [198, 199].

A key challenge in the RAG-specific fine-tuning lies in distinguishing between the injection of new knowledge into the parametric memory of the language model and the refinement of how that existing knowledge is utilized. Research indicates that fine-tuning is often ineffective for instilling entirely new factual information compared to retrieval. An empirical study found that unsupervised fine-tuning offers some improvement but consistently lags behind RAG, both for existing and new knowledge [30]. This difficulty in learning new facts is particularly pronounced when dealing with less-popular or low-frequency knowledge, although fine-tuning can significantly boost performance for these long-tail entities when sufficient quality synthetic data is available [25]. This highlights a crucial paradigm at the heart of the RAG-generation dilemma: fine-tuning can modulate how the model uses its existing knowledge, but it is less reliable for absorbing entirely new, isolated facts, whereas RAG is the preferred method for dynamic and explicit knowledge injection.

Given the high cost of full fine-tuning, particularly for large models, the paradigm of parameter-efficient solutions (PEFT) has become a dominant field. While various approaches like scaled prompt-tuning have been proposed for natural language generation [200], research demonstrates that the selection of which parameters to tune, or relying on basic strategies, can significantly impact performance and control overall training costs. This concept of carefully selecting a small subset of parameters to update is the central theme of the subsequent Subsection 5.2, which extends these ideas further by considering low-rank, sparse, and hybrid adaptations for even greater efficiency. The following discussion on unsupervised and few-shot learning provides a foundation for the practical considerations that drive the need for such parameter-efficient deployments.

Unsupervised learning strategies are pivotal for RAG because of the transient nature of the retrieved data. A novel perspective treats the LLM’s role in RAG as that of an “Information Refiner”. The proposed InFO-RAG method optimizes LLMs for RAG in an unsupervised manner, teaching them to consistently generate texts that are more concise and accurate than the retrieved texts, regardless of their quality, thereby improving performance as well as robustness across various tasks [201]. This is crucial for the efficient use of retrieved content, as it can operationalize the generator’s objective without any additional supervision labels, achieving strong and scalable improvements. In unsupervised fine-tuning, the use of small target data is important, and replaying a sparse portion of source data, or data mixing, is a critical detail for ensuring effective transfer [202].

Fine-tuning under limited annotation presents unique and practical challenges. In the low-resource setup, an instruction-tuned model can demonstrate efficient few-shot learning, effectively tracing from its initial training procedure and becoming a strong few-shot learner [141]. The selection of the data for this fine-tuning also matters substantially. The loss-based SFT data selection method (LoBaSS) uses the inherent “learnability” of data—measured by loss—as a guiding principle, allowing a model to beat full-data fine-tuning with only a fraction of the composition of the data [203]. Existing studies show that, for many task and cost-constrained contexts, parameter-efficient fine-tuning (PEFT) can outperform few-shot in-context learning (ICL) both in terms of accuracy and cost, positioning it as a practical alternative [204]. This shift is particularly important in RAG, where the context passages provided to answer a query take up precious KV-cache and can slow down the entire training and inference process.

In the context of RAG, the choice to adopt ICL or a fine-tuning approaches often boils down to the trade-off between flexibility and efficiency. While in-context learning is extremely flexible and requires no weight updates, it is computationally expensive in batched scenarios and often

shows degradation in performance as context grows. ICL can also suffer from overconfidence and miscalibration, and in limited data there are trade-offs between performance and calibration, which given can be addressed using self-ensembling techniques [205]. PEFT offers a middle ground per model, achieving better accuracy at a lower cost than ICL while offering a reduction from full fine-tuning. The final choice and the design of the fine-tuning strategy therefore depends on multiple factors, namely the availability of robust data, the computational budget, and the need for model autonomy. Ultimately, the choice between these paradigms hinges on balancing predictive performance against computation, cost, and desired autonomy. The survey shows that the design choice of which parameters to target heavily influences performance and efficiency, with the scale of the model also playing a critical role. To address these considerations systematically, the following subsections will explore methods for achieving this balance in greater detail.

5.2 Parameter-Efficient Tuning for RAG: From LoRA to Sparse and Hybrid Methods

Building upon the holistic training strategies discussed in the previous subsection—where full-parameter fine-tuning, unsupervised learning, and few-shot paradigms establish the foundational trade-offs between accuracy, computation, and adaptability—this subsection narrows the focus to a family of techniques that have become indispensable for making RAG both practically deployable and scalable: Parameter-Efficient Fine-Tuning (PEFT). While the preceding discussion highlighted the resource-intensive nature of full parameter updates and framed the need for more economical alternatives, PEFT directly answers the call for smaller trainable footprints without sacrificing task-specific performance. In the RAG context, where the generator is a massive LLM and the retriever may also be large, PEFT methods enable the coordinated adaptation of both components within strict compute and memory budgets, thereby bridging the gap between theoretical capability and real-world feasibility.

Low-Rank Adaptation (LoRA) has emerged as the cornerstone of PEFT, demonstrating that adaptations to pre-trained weights can be effectively approximated by low-rank incremental matrices. In RAG, LoRA proves instrumental for domain-adapting both the retriever and the generator. For instance, adapting a generator to a specific corpus with LoRA can significantly boost downstream generation quality in domain-specific settings, including tasks such as clinical dialogue summarization and multilingual news classification, where it matches or even outperforms full fine-tuning [206, 207]. A further key benefit in RAG is the ability to serve multiple tasks or domains concurrently; LoRA-based adaptation supports this by constructing task-specific modules from a shared base model, enabling shared infrastructure with multiple expert adapters [208]. In this setting, “LoRA composability” becomes an attractive property: task adapters can be combined dynamically to handle unseen tasks without additional training [209, 210]. This aligns naturally with the earlier discussion of few-shot learning and the practical need for rapid, low-cost adaptation to dynamic knowledge bases, without the overhead of re-tuning the entire backbone.

Despite its efficiency, standard LoRA is constrained by a fixed intrinsic rank, which may not match the actual capacity each module requires. Recognizing this limitation, substantial work has focused on rank-adaptive methods that dynamically adjust the number of trainable parameters per module, moving away from a one-size-fits-all solution. For example, ALoRA learns to prune low-importance ranks and reallocates the “pruned budget” to Transformer modules that require higher-rank adaptation [211]. Similarly, SoRA employs proximal gradient methods to gate and dynamically control rank cardinality during training [212]. These adaptive approaches are beneficial in RAG, where retrievers and generators may have heterogeneous adaptation needs, and they build on the principle introduced earlier of selectively choosing which parameters to tune, now refining

that selection dynamically. Moreover, understanding the underlying representation of LoRA as a single linear layer that transforms the pre-trained weights into task-adapted counterparts enables more efficient parameter reduction [213].

A significant extension of LoRA integrates it with Mixture-of-Experts (MoE) architectures to enhance expressiveness while preserving parameter efficiency. In this setup, the usual single LoRA update is replaced with a set of parallel LoRA experts, with inputs dynamically routed among them [214]. For example, MoELoRA uses contrastive learning to encourage expert specialization [215], and other studies suggest that allocating more LoRA experts to higher layers yields greater gains [216]. This technique—especially valuable in multi-task and multimodal RAG pipelines—helps mitigate data conflicts during instruction tuning of multimodal LLMs [217]. The MoE-based LoRA framework is particularly well-suited for RAG systems with broad task coverage, allowing each expert to capture distinct features without interfering with others [215].

Beyond low-rank adaptations, a wider family of sparse and decomposed methods extends the PEFT toolkit, often inspired by observed sparsity in gradient landscapes. Sparse update methods modify only a subset of parameters, but they require careful selection of which parameters to update. Yet, it has been shown that task-agnostic sparse masking is possible, with gradients often dominated by a small fraction of components [218]. To preserve pre-trained knowledge effectively, one promising direction merges low-rank with sparsity. For instance, RoSA jointly trains low-rank and highly-sparse components to approximate full fine-tuning [219], offering a more principle-driven adaptation that balances accuracy and resource use. Even more flexible decompositions are TENable via LoTR, which uses tensor decomposition to share components across layers, yielding better parameter efficiency [220]. These approaches strengthen the goals of the prior subsection by enabling fine-tuning hierarchies from the fundamental rank selection to more complex structural sharing, preserving knowledge while adapting to specialized needs.

Memory and computational constraints remain persistent bottlenecks in large-scale RAG systems. To circumvent the storage of full activation gradients—which is required even when updates are low-rank—methods like LoRA-FA freeze the projection-down weight A , thereby eliminating the need for full-rank input activation storage [221]. Others introduce skip connections to reduce accumulated error and improve convergence stability during LoRA training, such as ResLoRA [222]. Freezing has also been made adaptive; for example, AFLoRA dynamically freezes and unfreezes low-rank adapters to balance trainability and efficiency [223]. At the optimizer level, Gradient Low-Rank Projection (GaLore) applies low-rank approximation to the optimizer states of the full-gradient matrices, enabling full-parameter updates at a fraction of the memory cost [224]. Additionally, quantized updates such as QLoRA further reduce memory and compute, making strong performance possible within memory constraints of consumer hardware, even when used within RAG pipeline [225, 226]. These optimizations are crucial when deploying large models in RAG architectures, where inference latency and memory storage serve as primary constraints.

A final integral aspect of PEFT within RAG is the service-time deployment. Efficient serving of multiple LoRA adapters is enabled by specialized systems like S-LoRA and vLLM with paged memory management [208]. For RAG built on a shared base LLM, such serving systems reduce costs and enable simultaneous, on-the-fly adaptation across distinct tasks—critical for dynamic, multi-domain interaction.

In summary, the evolution from vanilla LoRA to adaptive-rank, sparse, decomposed, and hybrid PEFT methods is instrumental to the practical viability of high-performance RAG. It directly builds on the earlier calls for efficiency in the previous subsection, by refining *how* parameter selection is performed to address computational, memory, and storage constraints—thereby enabling large-scale deployment at lower cost. This predictable progression toward scalability and resource-awareness naturally dovetails with the next subsection’s focus on advanced optimization paradigms that

further tune RAG’s overall behavior, demonstrating that parameter-efficient preparation is only the groundwork for more sophisticated, learning-driven adaptation of the entire retrieval generation pipeline.

5.3 Training Augmentation, Self-Feedback, and Iterative Refinement in RAG

The effectiveness of Retrieval-Augmented Generation (RAG) systems hinges not only on the quality of their architectural components but also on the sophisticated training and optimization strategies employed to integrate them. While the parameter-efficient and resource-aware methods discussed earlier focus on *how* to update model weights efficiently, an equally critical dimension is *what* optimization signals and learning paradigms drive those updates. Foundational fine-tuning adapts retrievers and generators to a coordinated objective, yet advanced optimization paradigms are required to address the dynamic challenges of retrieval quality, noise, and complex reasoning. This subsection investigates three such advanced strategies: reinforcement learning for pipeline optimization, iterative self-feedback and self-optimization mechanisms, and knowledge distillation, concluding with a discussion of hybrid approaches that enable continuous improvement.

Reinforcement Learning for End-to-End Pipeline Optimization

A significant shift in optimizing RAG systems involves moving beyond supervised fine-tuning to directly optimize the end-to-end behavior of the pipeline using reinforcement learning (RL). The integration of RL is a natural fit, as the ultimate goal of a RAG system—providing a correct, faithful, and helpful answer—is often a non-differentiable objective, particularly when interacting with the environment through retrieval actions and text generation. The challenge of optimizing non-differentiable objectives via RL is a well-established concept [227], and it is directly applicable to aligning the RAG pipeline with complex, multi-faceted objectives that go beyond simple token-prediction accuracy.

This paradigm becomes especially critical when addressing the “preference gap” between what a retriever deems relevant and what an LLM finds most useful [55]. Traditional retrieval models are often trained to find documents that are lexically or semantically similar to a query, which does not guarantee an ranking of evidence that enables an LLM to produce the ground-truth answer. To bridge this gap, a **bridging** model can be introduced and trained using a combination of supervised and reinforcement learning. In this framework, the bridge model is first warmed up with supervised learning to identify a candidate set of relevant documents, then an RL algorithm (often with Proximal Policy Optimization) is employed to directly optimize the model’s behavior by rewarding it for ultimately enabling the LLM to answer the question correctly. This RL-driven method directly optimizes the entire pipeline for the final task objective, leading to a more effective coupling between retrieval and generation and enabling the system to produce higher-quality outputs in question-answering and personalized tasks [55].

The use of RL extends beyond retrieval ranking to enhancing the LLM’s reasoning capability through **Reinforced Fine-Tuning (ReFT)**. Where SFT relies on static, potentially suboptimal Chain-of-Thought (CoT) annotations, ReFT improves generalization by first warming up the model with SFT, and then employing online RL to fine-tune it further [228]. During this phase, the model receives a question and retrieved context, explores a variety of reasoning paths, and is rewarded only for those paths that ultimately lead to gold-standard answers. This exploration and reinforcement loop teaches the model to discover its own effective reasoning strategies, thereby synthesizing more robust and accurate outputs tailored to the retrieval-augmented input [229].

In the broader system context, RL also provides tools to optimize under complex human preferences and constraints. For instance, integrating a generative adversarial network with RL to construct a **generative adversarial reward model** enables fine-tuning of the RAG

pipeline according to learned reward signals, effectively aligning outputs with complex end-user preferences related to helpfulness and faithfulness without a human-in-the-loop at every iteration [230]. Given the fallibility of learned reward functions, however, it is critical to guard against **policy overoptimization**, where the model exploits flaws in the proxy rather than achieving its true objective. **Reward Model Ensembles** with conservative optimization objectives, such as worst-case and uncertainty-weighted optimization, have been shown to effectively mitigate this risk. These ensemble-based approaches are relevant and robust for improving RL-based RAG pipelines, particularly when combined with large-scale **gold** reward models to simulate expensive human feedback [231].

Self-Feedback, Iterative Self-Refinement, and Automatic Optimization

Beyond external optimization signals, RAG models can significantly benefit from **iterative self-feedback** mechanisms, where the model itself acts as its own critic and data generator. These strategies can substantially reduce hallucination and enhance factual reasoning by iteratively decomposing complex tasks and refining outputs based on a self-generated critic. A prominent framework is the **Retrieval-Augmented Iterative Self-Feedback (RA-ISF)**, which decomposes a complex problem into manageable sub-questions and processes them through self-feedback modules. By enabling the LLM to iteratively generate a task plan, retrieve relevant information for each step, and receive self-generated feedback on its sub-tasks, the model can mitigate reasoning errors and substantially bolster factual reasoning [42].

Similarly, with roots in **game-theoretic training**, another approach uses the concept of self-play to convert the model’s own predictions into a training signal. **Self-Play fine-tuning (SPIN)** refines RAG models by replacing target labels with **self-generated data** produced during previous iterations. This creates a mini-max game where a **main player** improves its policy by learning to discern its current self-generated responses from those sampled more trustworthy target data distribution, effectively strengthening the coherence between the model’s internal knowledge and its retrieved context [232]. Such self-play techniques enforce robustness without additional human annotation.

Moreover, this iterative nature extends to **self-referencing** in sequence modeling, where historical information from past policy updates or past retrievals is used to stabilize the current learning step and prevent the unlearning of useful knowledge. By explicitly referencing previous interactions, strategies such as **self-reference** can correct biases and keep past knowledge efficiently, while **boosting learning efficiency** and preventing catastrophic forgetting [233]. This suite of mechanisms points to a pattern where the system does not just “retrieve and generate” once, but rather processes its own past outputs as new context. RAG components are thus transformed from a static function to an adaptive computational pipeline that actively learns beyond its original training distribution.

Knowledge Distillation and Parameter-Efficient Learning (PEFT)

A key bottleneck in RAG is the computational expense of training large generator models. Knowledge distillation offers a pathway to transfer specialized capabilities from a large, capable teacher model to a smaller, more efficient student, thereby optimizing deployment. This distillation is particularly relevant for RAG systems, where a large model can teach a more compact student to approximate its output distribution. Distillation is valuable to increase the performance of a **low-capacity student agent**; a compelling empirical demonstration shows that a **low-capacity PPO agent** can become significantly more effective by **imitating the action probabilities of a high-capacity teacher**, and after a “finetuning” stage in the environment, the student can approach the teacher’s utility [234]. Distillation thus efficiently trains a compact, deployable policy, leading to faster inference and lower memory footprint while maintaining good performance.

Such approaches are often paired with **parameter-efficient fine-tuning (PEFT)** to make

training and adapting the RAG components on downstream tasks more accessible. Residual learning from **Chain of LoRA (COLA)** allows the system to generate a sequence of conjoined, low-ranked modules and iteratively gap the difference between typical dimension and full-rank updates without additional inference costs. COLA’s iterative optimization framework, inspired by Frank-Wolfe, can also be used to merge multiple LoRA modules adaptively into the existing model as it is being fine-tuned, gaining residual learning within a constrained memory budget. This systematically increases model capacity without introducing significant additional computation, drastically improving RAG performance without gradient explosion or excessive memory overhead [235].

Hybrid Methods and Continuous Adjustment

The ultimate line of RAG optimization is not a single discrete method but a **hybrid architecture** that orchestrates various objective functions, gradients, and optimization updates together. Underpinned by **Dynamic Corrective Self-Distillation (DCS)**, a type of self-adaptive boosting, techniques can refine the systems in an iterative loop: at each iteration, a student model is heavily re-weighted by features from a teacher model and corrects itself dynamically as it adapts to new data, leading to improved robustness on long-tail label distributions—a pertinent approach for RAG systems that aim to optimize over multiple tasks [236]. This self-distillation mechanism offers a route to improve performance when the RAG fine-tuning is applied across diverse scenarios.

Another powerful direction is the direct extension of the concept of residual learning to a whole pipeline, where the feedback and optimization loops are tuned jointly in a system. For instance, an **adaptive self-tuning actor-critic** method can be introduced to automatically adjust the hyperparameters of the entire architecture, thereby removing the need for costly manual hyperparameter tuning [237]—which is a critical advantage in the resource- and time-constrained settings considered previously in this survey. Similarly, **AutoRL** methods can be employed to automate the reward design inside RAG-related tasks, treating reward functions as hyperparameters and searching for more effective reward signals via evolutionary algorithms [227]. The combination of adaptive learning with self-correcting trajectories provides a means to sequentially preserve capable policies while concurrently optimizing accumulated errors—transcending traditional fixed-optimization pipelines.

In summary, the recent training augmentation breakthroughs for RAG are defined by to a shift from a supervised, offline paradigm to a richer, adaptively trained system that integrates *efficient*, online RL, *robust*, internal reward design, and *continuous* self-improvement. The ecosystem makes the training and updating nature of the entire components mutually coupled, while transferring knowledge from high- to low-capacity models seamlessly, achieving the practical balance between model accuracy and computational viability that remains at the heart of deploying modern RAG systems.

5.4 Optimization, Robustness, and Resource-Aware Fine-Tuning for Practical RAG Deployment

The deployment of Retrieval-Augmented Generation (RAG) systems in real-world, resource-constrained environments necessitates a careful balance between model accuracy, computational efficiency, memory footprint, and inference latency. While the advanced optimization paradigms discussed previously—reinforcement learning, self-feedback, and knowledge distillation—focus on *what* learning signals drive model updates, an equally critical dimension is *how* those updates are executed under practical hardware and memory limits. Fine-tuning, while essential for adapting general-purpose LLMs to specific domains and tasks, is notoriously resource-intensive. This subsection explores the optimization, robustness, and resource-aware strategies that are critical for making RAG fine-tuning not only feasible but also stable and practical for production use,

complementing the behavioral adaptations with architectural and computational efficiency.

A central challenge in fine-tuning large language models is the prohibitive memory cost, particularly for optimizer states and intermediate activations. Full-parameter fine-tuning (FFT) is often infeasible for large models, requiring powerful server-grade GPUs. To bridge the performance gap between FFT and parameter-efficient fine-tuning (PEFT) methods, a novel framework termed QFT was introduced, which integrates the efficient Lion optimizer with quantized model states to enable memory-efficient full-parameter tuning, demonstrating that full-parameter performance can be approached with significantly reduced resources [238]. Similarly, parameter-efficient and quantization-aware adaptation (PEQA) has been proposed to directly fine-tune compressed models. By only updating quantization scales, PEQA proves that it is possible to restore sub-4-bit quantized LLMs to their full-precision performance while reducing the overall model size, making them more practical for edge deployment [226]. Other memory-efficient strategies have explored reducing the memory footprint during training. For instance, a side-network approach called Quantized Side Tuning (QST) cleverly avoids backpropagation through the frozen, quantized LLM by using a separate side network, which reduces both activation memory and computational costs, delivering up to a 7x total memory reduction compared to full fine-tuning [239].

Among the most prominent solutions for efficient fine-tuning, QLoRA combined a 4-bit quantized frozen backbone with trainable Low-Rank Adapters (LoRA). This approach redefined the state-of-the-art for single-GPU fine-tuning, allowing technologies like the retrieval-augmented generation pipeline to be adapted to models up to 65B parameters on a single 48GB GPU while preserving full 16-bit task performance [240]. The impact of QLoRA extends beyond its direct application; it catalyzed an ecosystem of techniques designed to improve accuracy fidelity and further efficiency gains. Low-rank plus quantized matrix decomposition (LQ-LoRA), for example, generalizes the approach by decomposing pretrained matrices into a high-precision low-rank component and a memory-efficient quantized one. An integer programming formulation dynamically configures quantization parameters to meet a target memory budget, enabling even sub-3-bit quantization to perform strongly [241].

Within this ecosystem, aggressive low-bit quantization often necessitates advanced error correction mechanisms to maintain quality, and low-rank components are leveraged not only for adaptation but also for rectifying quantization errors themselves. Approaches like INT2.1 leverage a Low-Rank Error Correction (LREC) method to rectify the gap between the quantized model and its float-point counterpart, resulting in a fine-tunable INT2/INT3 model capable of generating coherent text. This highlights the role of LoRA as a powerful tool for error correction and for alleviating catastrophic forgetting caused by extreme quantization [242]. To avoid catastrophic forgetting during quantization entirely, the ApiQ framework restores lost information by concurrently initializing LoRA components while quantizing the model weights to correct error propagation, showing strong performance on language tasks [243]. These quantization-aware methods synergize with the knowledge distillation strategies from the previous subsection, similarly transferring high-fidelity knowledge from a full-precision teacher to a efficiently deployable student, but now at the representational level.

Beyond parameter efficiency, the choice of quantization and fine-tuning methods is intertwined with system-level and inference optimizations. Traditional post-training quantization (PTQ) faces the critical challenge of activation outliers in particular channels, which represents a major distributional shift that hurts model quality. State-of-the-art methods like QUIK are designed to address this, introducing for the first-time end-to-end hybrid quantization that keeps only minimal outlier weights and activations in higher precision. This leads to practical speedups for batch inference and prompt processing, translating to up to 3.4x throughput improvements relative to FP16 execution [244]. Likewise, the design of quantization must be model-centric: sensitivity-based non-uniform

quantization, which searches for the optimal transform for the model while storing outliers in a Dense-and-Sparse format, acknowledges that the memory bandwidth bottleneck is key for inference. When applied to the LLaMA models, this method achieves strong 3-bit compression with minimal loss in accuracy, and up to 2.3x speedup during inference [245].

While quantization reduces the memory footprint of the model and accelerates inference, the interaction with training stability and robustness cannot be overlooked. The recent exploration of emergent abilities in the case of low-bit quantized models highlights how 2-bit quantization can severely degrade important capabilities like in-context learning, chain-of-thought reasoning, and instruction-following [246]. This suggests that a one-size-fits-all quantization approach is not enough and that 4-bit models capable of low-bit generation require further optimization. To address this, workflows with sharpness-aware minimization and robust perturbation techniques improve generalization from reentrant data, learning to identify flat minima rather than sharp ones, which proves especially useful in low-resource settings [247]. These optimization-aware training methods connect directly to the robustness themes in the previous subsection, as they aim to improve parameters against noisy retrieval inputs without additional data collection.

To further improve the quality of the RAG pipeline, there is a compelling direction that optimizes the retrieval component itself. A significant computational bottleneck is often the negative impact of extremely long retrieval-augmented prompt sequences. Experiments using a JAX just-in-time compiler and tensor sharding demonstrate over 12x improvement in runtime over Hugging Face/DeepSpeed implementations with four GPUs while consuming less than half the VRAM conveyance. This library, titled JORA, delivers an ideal GPU efficiency for fine-tuning extensive retrieval contexts [70]. Also, one can fine-tune embedding models within a RAG system through a method like Mafin, which augments a black-box embedding model by training a separate trainable embedding model, avoiding them having to update the frozen black-box model directly [67].

To implement these efficient and robust PEFT models, hyperparameter optimization (HPO) has been instrumental in achieving the performance limit for low-cost tuning. While gradient-free hyperparameter tuning for LoRA-based fine-tuning is possible, operations can be, in practice, computationally expensive; hyperparameter optimization methods can build on risk modeling to identify promising configurations, as demonstrated by HyperJump, which accelerates the search for optimal parameters [248]. When applied alongside blackbox optimization techniques, it yields significant boosts in performance and alignment of the tuned model on corpora [249]. This optimization runs parallel to the automatic adjustment of training pipelines lies within the previous subsection on hybrid methods, ensuring that the RAG system can adapt its own hyperparameters.

Collectively, these resource-aware and optimization techniques create a viable path for integrating robust RAG systems in environments with limited hardware, memory, and latency constraints. They ensure that the enhanced accuracy derived from the advanced training strategies discussed earlier is not traded against unreachable scalability. The choice of quantization and fine-tuning methods thus introduces distinct trade-offs—in the deployment of RAG systems, ensuring that enhanced accuracy does not come at the cost of unreachable scalability. These optimization paths are integral for making RAG systems robust to sparse and heterogeneous hardware when deployed broadly, allowing the balance between accuracy, memory, and inference speed to be struck adaptively.

6 Robustness, Security, and Knowledge Conflict Management

6.1 Knowledge Conflicts Between Parametric Prior and External Evidence

The integration of external knowledge into large language models (LLMs) through retrieval-augmented generation (RAG) presents a fundamental tension: the model’s parametric memory,

encoded during pre-training, may conflict with the non-parametric evidence provided at inference time. These knowledge conflicts are not merely academic curiosities; they arise naturally in real-world applications where documents may be outdated, factually fallible, or contextually narrow. Understanding when and why LLMs prioritize one source over another is central to building trustworthy RAG systems. The original motivation for mitigating these conflicts stems from the broader recognition that LLMs suffer from hallucinations and outdated knowledge, limitations that retrieval augmentation was designed to address [24]. However, as this subsection details, the mere presence of external evidence does not guarantee that the model will correctly update its beliefs; a complex tug-of-war ensues.

Systematic studies have identified three primary categories of conflict: context-memory (between the prompt and the model’s internal memory), inter-context (between different passages in the prompt), and intra-memory (within the model’s parametric knowledge) [250]. To properly assess the factors that govern model behavior, research has moved beyond simple binary scenarios of “correct” versus “incorrect” answers. The KNOT framework introduces a critical perspective by evaluating conflicts across three reasoning levels: direct extraction, explicit reasoning, and implicit reasoning [251]. By structuring the problem this way, KNOT reveals that an LLM’s ability to resolve a conflict is not monolithic but degrades significantly as reasoning complexity increases, moving from simpler retrieval to complex inference.

The bulk of the research aims to determine whether a model will either (a) faithfully read and update its knowledge based on new context or (b) rely on its pre-existing, often outdated parametric memory. Some findings have revealed a striking asymmetry in this behavior. Early work suggested LLMs were overly reliant on internal biases when faced with counter-memories [252]; however, more recent realistic experimental designs show that LLMs can be quite receptive to external evidence if presented coherently [28]. In a seeming contradiction, the same studies note a prominent tendency akin to “confirmation bias”: if the external context offers even a glimmer of agreement with the parametric memory, models are much more likely to cling to their original answers, aptly captured by the “Adaptive Chameleon or Stubborn Sloth” dichotomy [63]. This demonstrates that the internal knowledge not only serves as an alternative hypothesis but also actively influences how the model processes new, potentially more accurate, information.

A novel factor affecting this is the model’s prior confidence. The “tug-of-law” between the LLM’s internal prior and correctly or incorrectly provided reference documents shows distinct biases: LLMs are notably more resistant to edits that substantially deviate from their internal knowledge, showing a clear preference for information that is closer to what they already “know” [37]. Similarly, a related observation is a parametric bias, where failure to update knowledge is more common when the incorrect parametric answer is mirrored in the context [28]. This suggests that a model prioritizes its existing memory over new information.

There are often conflicting outcomes in these experiments, partly due to the nature of the evidence; synthetic designs often exaggerate these phenomena, and thus sometimes conflicting results emerge. It is particularly important to account for broader reasoning skills. Proponents of the realistic knowledge conflict framework argue that if the model has a weak prior, providing a “correct” but conflicting document will typically fix the model’s mistakes [28]. However, if the model is faced with a strong known fact and a subtly perturbed one, it may fail to override it. When the LLM’s knowledge base is shaky, the external evidence can trump; but it may also be the case that the model turns to a “majority rule” voting or its ability to differentiate memory. In fact, disentangling behaviors shows that models exhibit tendencies like the “Dunning-Kruger” effect: stronger models will commit to a wrong answer if the internal signal is faulty, even when the correct answer is present in the same input [253].

A key practical factor lies in how we prompt the LLM. The “context-faithful” community has

shown that specific “opinion-based” or “counterfactual” demonstrations can be used to signal to the model that it *should* abandon prior knowledge in favor of the context [254]. Also, we can actively anticipate failure patterns. Instead of expecting an LLM to inherently handle contradiction, we can train or prompt it for more nuanced “abstainment.” With careful modifications of prompting, models can be made to be near-perfect “editors” that correctly give preference to contextual knowledge when it conflicts, regardless of what their parametric memory says [255]. This works by explicitly presenting the model with a “note” and getting the LLM to take on a “role,” e.g., an professor, which forces a game of mental model-switching that decouples the control of the generative model from the contextual evidence. More sophisticated frameworks like “Dialectical Alignment” take this one step further and add that the safety community must not conflate stubbornness with trust; a model highly adaptive to context might be a risk if the context is poisoned. Through special preference alignment, LLMs can be trained to be more cautious and question the context when the mixture seems off [180]. In tests, this boosts the efficacy of getting a model to reject poisoned attacks that attempt to flip its answer.

However, we need to calibrate our expectations. Building a robust system requires more than one single decoding strategy, prompting call, or one-step retrieval. In a certain sense, it is not that models “don’t want to read.” The evidence suggests they are cautious about how and when to apply evidence. The preceding analysis suggests that the path to resolving these conflicts does not necessarily need massive fine-tuning for each new setting. Only by carefully designing the prompt, reshaping the context, and dynamically implementing techniques like synthetic counterfactual logic can we hope to steer the model between being adaptive (when adjustments to solid evidence are correct) and defiant (when the contextual “noise” is polluting the signal). This remains an open challenge, but one we can now frame, forecast, and direct—understanding which boundaries matter, where the cautious state fails, and how to calibrate the value *and* direction of an LLM’s belief change.

This foundational understanding of knowledge conflicts and the behavioral asymmetries they induce naturally raises the question of how RAG pipelines behave when the retrieval process itself is unreliable—an issue that extends beyond epistemic tension to explicit adversarial manipulation, noisy context, and the need for systems to abstain when no valid support exists.

6.2 Adversarial Robustness, Noise, and Negative Rejection

The knowledge conflicts discussed in the previous subsection reveal a fundamental fragility: LLMs struggle to adjudicate between parametric memory and non-parametric evidence. This fragility is not merely a cognitive limitation but a security vulnerability. When retrieval itself is unreliable—whether due to natural noise, adversarial manipulation, or outright poisoning—the very mechanisms designed to ground LLMs in external knowledge become vectors for attack. RAG systems, while powerful for grounding LLM outputs, inherit vulnerabilities from both the parametric generation process and the external retrieval pipeline. The integration of retrieved context, often sourced from massive, uncontrolled corpora, introduces a broad attack surface, as evidenced by the limitations of LLM application frameworks developed without sufficient consideration for the risk of external content [256]. This subsection examines the behavior of RAG pipelines under noisy, irrelevant, and adversarial conditions, detailing prominent attack vectors, defense mechanisms, and evaluation benchmarks and metrics for assessing overall robustness.

Attack Vectors: From Noise to Poisoning and Manipulation

The primary threat to RAG and its foundational components—retrieval and generation—arises from manipulated context. The retrieval database, often containing millions of texts, is a prime target for knowledge poisoning attacks [257, 256]. Knowledge poisoning is characterized as an

optimization problem that seeks to identify a set of crafted poisoned texts that, when injected into the knowledge base, cause the LLM to generate an attacker-chosen answer for a target question. Through both black-box and white-box approaches, adversarial texts can achieve a 90% attack success rate with only a small set of poisoned texts [257]. This direct manipulation of the retrieval database—analogue to the context-memory conflicts described earlier—demonstrates how an attacker can exploit the model’s tendency to prioritize contextual evidence over parametric knowledge, inverting the corrective intent of RAG.

The pervasiveness of this threat is heightened by the fact that even low-level perturbations, such as typographical errors common in real-world databases, can severely disrupt a RAG pipeline [258]. The Genetic Attack on RAG (GARAG) method was designed to simulate these naturally occurring imperfections, revealing vulnerabilities within each RAG component and showcasing that documents with minor text inaccuracies are enough to devastate the synergy between retrieval and generation [258]. Attacks have also been conducted on the querying side; the insertion of a short, nearly benign prefix into the prompt can manipulate a RAG system into generating factually incorrect answers. One notable technique, Gradient Guided Prompt Perturbation (GGPP), successfully steers the outputs of RAG-based LLMs to targeted wrong answers [259]. This prompt-level manipulation is a direct extension of the context-faithfulness challenges discussed in the previous subsection: the model’s receptivity to external evidence, which we observed can be enhanced through careful prompting, is the very property that adversarial prompts exploit.

The adversarial attack surface extends from the retriever to the generator. Traditional attacks from the information retrieval literature, including attacks on neural ranking models (NRMs), can directly compromise retrieval backbone. Black-box, decision-based attacks can promote a target document in the rankings using small text perturbations [260]. These attacks often exploit the interaction-focused architecture of NRMs, which prioritizes the relevance matching, while newer approaches, such as adversarial retrieval attacks (AREA), formalize attacks on dense retrieval (DR) models as a contrastive learning problem to account for their distinct representation-focused architecture [261]. Additionally, retrieval can fail generically, returning content that does not answer the query—a problem reminiscent of the “majority rule” or “Dunning-Krueger” behaviors observed under knowledge conflicts, where the model must decide whether to trust a noisy or irrelevant context. Even when retrieval succeeds with the relevant passage, noisy or irrelevant context can degrade model performance, challenging the integrative capabilities of the LLM [34]. This degradation is a practical manifestation of the confirmation-bias phenomenon introduced earlier; the model’s parametric priors are recruited to process a “glimmer of agreement” in the noise, reinforcing its original beliefs rather than updating them.

Defense Mechanisms: Robustness and Abstention

A multi-pronged approach is required to address adversarial and noisy inputs. Robustness is not only about filtering attacks but also about resilience to naturally perturbed inputs. As an overarching strategy, “trusting” components need to be flexible without being naively compliant; for example, while alignment methods make LLMs highly receptive to external evidence, this can backfire—models become “adaptive chameleons” that follow the corrupted context over their parametric memory, amplifying the risk of attacks in RAG systems, a concern directly connected to the “tug-of-war” from a previous subsection [180]. The work on Dialectical Alignment (DA) mitigates this by generating data and strategies that equip models with a style of reasoning that does not allow for conflict to be exploited, improving defense against poisoned contexts by 20 points while preserving the effectiveness of in-context knowledge editing [180]. This aligns with the observation from earlier analysis that simply expecting a model to “read” better is not enough; the model’s default deference to context must be modulated with adaptive caution.

At the dedicated retrieval level, defense can be conceptualized similarly to adversarial robustness

constraints in other domains. As adversarial attacks in the image and NLP domains have shown, incorporating adversarial training and robustness constraints robustly bolsters the retriever’s ability to ignore imperceptible perturbations [262]. Techniques like Perturbation-Invariant Adversarial Training (PIAT) have demonstrated theoretical guarantees on the effectiveness-robustness trade-off for NRMs, enforcing a smooth output that resists adversarial documents [263]. Similarly, certified robustness methods use randomized smoothing to guarantee that a document’s rank cannot be manipulated by word substitution, providing formal certified top-K robustness for ranking systems [264]. Beyond model tuning, another crucial defense is the model’s own ability to recognize when retrieved context is pure noise.

RAG systems should detect when retrieval fails, either returning irrelevant content or missing the answer entirely. This notion of “knowing when you don’t know” is central: both the non-relevant correctly empty subset of a test corpus (when no answer exists) and the relevant subset (where a relevant passage exists) need to be handled differently [265]. Predicting abstention, or selective prediction when external evidence cannot support a confident generation, becomes a massive behavioral defense against adversarial noise. This concept of the model actively rejecting the prompt—the opposite of it blindly following noisy context—is the encapsulation of the “stubborn” side of the spectrum discussed in the previous subsection. When the model’s own prior confidence is low, and the external evidence is ambiguous, abstaining is the safest course, a tactic far more robust than forced generation. This ties directly to the broader theme of trustworthy generation: the ability to say “I don’t know” is thus both a factual safeguard and a security hardening strategy.

Evaluation Benchmarks and Metrics for Robustness

Properly evaluating robustness, adversarial attack, and negative rejection is a challenge in itself. Comprehensive benchmarks have been designed to isolate and diagnose the capabilities of RAG, particularly its noise robustness, negative rejection, information integration, and counterfactual robustness. The Bombench Retrieval-Augmented Generation Benchmark (RGB) is a pioneering work evaluating these core abilities in both English and Chinese, and it reveals that models still struggle with the latter abilities [34]. Similarly, the NoMIRACL dataset across 18 typologically diverse languages evaluates the LLM’s robustness and focuses on hallucination rates when answers are irrelevant and error rates when they are relevant [265]. More broadly, framework specific to summarization, such as the targeted “LogicSumm” for summarization, demonstrate that scenario-specific evaluations are needed for RAG robustness in complex tasks [266]. These benchmark results provide a rigorous foundation for understanding not just whether the next token is correct, but whether the system trusted the source it should have, and rejected the one it should not.

In conclusion, the robustness of RAG is an evolving story that interconnects directly with the prior discussion of internal knowledge conflict. When the external source becomes an adversary, the model’s sticky “stubbornness” to internal priors is no longer a cognitive bug but a security feature—it resists attacks by refusing to blindly endorse conflicting, poisoned evidence. The attack surface comprises three critical entry points: the retrieval store (poisoning), the prompt (manipulation), and the retriever itself (ranking attack) [258], [257], [261]. The defense, in turn, must be equally multi-faceted: from careful strategic alignment to certified retriever robustness, and finally, the effective act of abstention—a deterministic “no” when the evidence is bad. This tri-partite defense mechanism not only secures the system but also provides a clear path for the next step: framing RAG behavior within formalized control and calibration frameworks, ensuring that its responses are not just efficient but can be trusted under any coercion.

6.3 Security Threats and Privacy Implications

Retrieval-Augmented Generation (RAG) systems, while powerful, introduce a complex and expanded attack surface that spans both the parametric language model and the non-parametric retrieval infrastructure. This subsection investigates the multifaceted security threats and privacy implications that arise from this architecture, particularly highlighting the vulnerabilities in retrieval databases, the risks of prompt manipulation, and the dual-edged nature of privacy in such systems. Understanding these threats is crucial for building robust and trustworthy RAG deployments.

6.3.1 Security Threats: The Expanded Attack Surface

The integration of a retrieval component into LLMs creates new vectors for adversarial manipulation. The most prominent among these is retrieval poisoning, where an attacker injects malicious or subtly crafted documents into the knowledge database to influence the model’s output. The [256] paper demonstrates that an attacker can craft documents that are visually indistinguishable from benign ones but, when used as context for RAG, mislead the LLM into generating incorrect responses. This attack is particularly dangerous because it weaponizes the RAG system’s core functionality, undermining its reliability. Furthermore, [267] introduces the concept of RAG poisoning as an indirect jailbreak vector, where malicious content is used to circumvent an LLM’s safeguards entirely. The [257] model formalizes these attacks as an optimization problem, attempting to inject a small amount of poisoned text to trigger a pre-determined (target) response from the LLM, achieving high success rates even in massive databases.

The retrieval and generation pipeline can be particularly vulnerable to nuanced perturbations. For instance, even low-level textual errors can disproportionately impact shared RAG performance. The [258] paper introduces GARAG, a genetic algorithm-based attack targeting noisy documents in the pipeline. It demonstrates that minor, seemingly inconsequential typos can discover vulnerabilities not just in the retriever but also in the final generative answer. Another key threat vector is prompt manipulation, especially in multi-turn interactions. When a model’s behavior is conditioned on the system, an attacker using the LLM’s instruction-following capabilities is a serious risk. [268] shows that an adversary can directly prompt the system in a way that leads to verbatim extraction of the text from the datastore, even showing that the vulnerability escalates as model size increases. That type of unintended response is extended to the RAG’s risk of leaking user-provided data. The [269] paper explicitly investigates this, highlighting a “prompt leakage” effect in which a system’s private instructions and knowledge can be extracted through multi-turn interactions.

These attacks on the RAG pipeline have broad implications. [270] highlights methods of adversarial attacks, but in the context of RAG system, such attacks are insufficient. [271], [272], and [256] collectively demonstrate that existing defenses are insufficient.

6.3.2 Privacy Leakage and the Dual-Edged Role of RAG

The intersection of RAG and privacy is multifaceted. On one hand, RAG promises to reduce the leakage of training data by adjusting external knowledge. On the other hand, the retrieval database itself becomes a new source of privacy risk. The study [273] considers this dual role, revealing a “Good and the Bad”. It empirically shows RAG can, in fact, mitigate leakage from LLMs’ implicit training data in some cases, but, conversely, the RAG system becomes more vulnerable to leaking the private data stored in the retrieval store. This is further evidenced by [274], which reveals the datastore as a new vector for extraction. The design of content-based retrieval makes it difficult to prevent such attacks.

Recent studies have contributed to a deeper understanding of this dual nature. For instance, [275] shows the addition of a retrieval store is more susceptible to leaking than the parametric baseline, highlighting a detrimental effect. Additionally, [276] investigates membership inference attacks on models, which could be relevant for the privacy risk of the RAG’s associated LM. The direct line of attack is the **retriever’s** database, which can be targeted by a membership inference attack [277]. This allows attackers to infer whether a specific data point is in the retrieval database or not.

6.3.3 Defense Strategy: Hardening and Privacy-Preserving Mechanisms

Defending against these diverse threats is challenging, and there is a fundamental trade-off between security and the utility of the RAG system. This section outlines the various defensive approaches.

Black-box prompt hardening and query rewriting is an effective first line of defense against prompt-based attacks. The [269] paper introduces a suite of black-box defense strategies. [278] and a multi-tier combination of defenses can achieve a lower attack success rate, though still with limitations, revealing a mandate for future work. This approach is limited by the LLM’s core instruction-following ability.

Differential Privacy (DP) and Membership Inference considerations provide a more principled defense, but the cost to model quality remains a major limitation. [279], [280], and [281] The Application of Piecewise estimate the trade-off. However, our knowledge of these cases also shows there are trade-offs in protecting core language models, which are different. A critical insight comes from [275], which shows how the retrieval store can expose more (compared to parametric model) and how the use of sanitized data can eliminate risks. Other work on [282] [283] [283] also provides solutions to close the trust gap.

Limitations of current DP as a defense are also exposed. [284] and [285] provide a sense of the boundaries and inevitable conflicts.

Secure Retrieval and Secure aggregation protocols are another type. [286] highlights that secure aggregation still leaks information across multiple rounds. This has direct implications for RAG, suggesting that secure aggregation of embeddings does not fully prevent privacy leakage. [287] underscores that guaranteeing privacy in distributed settings is difficult even with DP.

Hardening against adversarial perturbations is also a direction, but the foundation of the RAG architecture makes it difficult. [288] suggests that anti-adversarial defenses can be applied to the components of the RAG pipeline, but current models still are often vulnerable to attacks from the knowledge base.

In summary, RAG inherits and amplifies the security and privacy challenges of LLMs while introducing new vulnerabilities unique to the retrieval infrastructure. The attack vectors range from subtle prompt manipulations to large-scale knowledge base poisoning; the privacy risks shift from parametric memorization to the exposure of external datastores; and defenses remain an open arena of research. By understanding these threats and their interplay, we lay the groundwork for the robust and trustworthy deployment of RAG systems, which is the focus of the subsequent subsections on formal guarantees and calibration.

6.4 Formal Guarantees and Calibration for Trustworthy RAG

The preceding subsection highlighted the vulnerabilities that threaten RAG systems; this one addresses the complementary imperative: providing formal, verifiable assurances of trustworthiness—the mathematical counterpart to hardening. While Retrieval-Augmented Generation (RAG) significantly enhances the factual grounding of large language models, its deployment in high-

stakes domains—such as healthcare, finance, and law—demands more than empirical accuracy; it requires formal, verifiable assurances of trustworthiness. The probabilistic nature of both the retriever and the generator introduces inherent uncertainties, and the dynamic interaction between parametric knowledge and external evidence can lead to distribution shifts that undermine reliability. This subsection examines the formal and probabilistic methods designed to quantify, control, and certify the performance of RAG systems under such uncertainty. The central tool in this pursuit is conformal prediction, a distribution-free framework that provides finite-sample statistical guarantees, alongside adaptive calibration and dynamic selection strategies that ensure operational safety even under non-stationary conditions—directly addressing the vulnerabilities exposed in the preceding security analysis.

Conformal Prediction (CP) provides a powerful, model-agnostic framework for uncertainty quantification, transforming point predictions into set-valued outputs with defined statistical guarantees. Traditional CP methods guarantee *marginal coverage*—that is, the constructed prediction set is guaranteed to contain the true or valid answer with a user-specified probability (e.g., 90%) under the assumption of data exchangeability. This foundational property offers a principled approach for RAG beyond simple heuristic confidence scoring. However, a primary challenge is the direct application of CP to RAG pipelines, as the retrieval step introduces a significant source of hidden uncertainty. If the retriever fails to identify the necessary evidence, the generated response cannot be grounded, and the conformal guarantee on the generation step becomes meaningless—a failure mode analogous to the retrieval-database vulnerabilities described in the previous section. To address this, the **CONFLARE** framework introduces a four-step procedural application of conformal prediction explicitly designed to quantify retrieval uncertainty [184]. By constructing a calibration set of questions known to be answerable from the knowledge base, CONFLARE analyzes the similarity scores of retrieved chunks to establish a threshold. Similar to the application of conformal methods in related work [289], this threshold is used at inference time to retrieve all chunks with a similarity score above it, providing a statistical guarantee $(1-\alpha)$ that the set of retrieved contexts will collectively contain the information necessary to generate the correct answer—thereby closing the loophole of incomplete or partial evidence that attackers might exploit.

While CONFLARE guarantees that the correct context is *included* (coverage-of-retrieval), the C-RAG framework tackles a further and critical challenge: certifying the *generation risk* itself, even when the context may be corrupted or noisy. C-RAG [186] extends conformal analysis to provide an upper confidence bound on the risk of the *final generated answer* being incorrect. Under a general bounded risk function, C-RAG offers a theoretical guarantee bounding the risk of RAG models. This can be conceptually linked to more general formulations of distribution-free, risk-controlling prediction sets [290], though applied specifically to the RAG setting. Crucially, C-RAG demonstrates that when retrieval and the transformer are of non-trivial quality, a RAG model achieves a lower certified risk than a standalone LLM, offering a quantified end-to-end guarantee on the system’s prediction safety [186]. In practice, such certified guarantees are the intended countermeasure to the attack vectors described earlier: the certification boundary creates a formal barrier against adversarial noise, ensuring the reliability of the final answer even if the retrieval infrastructure has been partially compromised.

A purely marginal guarantee is often insufficient; for practical decision-making, we need guarantees that are robust to distribution shifts—another silent vulnerability that parallels the attack surface of prompt manipulation but originates from the natural dynamics of user queries and data drift. Formal conformal methods have demonstrated that standard validity breaks when data exchangeability is violated, such as in sequential data, covariate shifts, or out-of-domain topics. For RAG, this is particularly salient when the question distribution at inference time differs from the calibration data. To handle this, extensions of the conformal framework, such as Non-Exchangeable

Conformal Risk Control, extend guarantees to control monotone loss functions even when data is not exchangeable, allowing for weighting of data that handles distribution drift [291]. This is complemented by Adaptive Conformal Inference (ACI) methods that can provide robust coverage in an online setting, making them suitable for RAG systems that must handle *online or streaming* retrieval and generation [292]. Specifically, for language generation, tools like *Non-Exchangeable Conformal Nucleus Sampling* [293] use nearest neighbors to achieve post-hoc token-level calibrated prediction sets without retraining, a vital step for handling variable-length outputs under drift.

While conformal guarantees can be criticized for being marginal (wide) rather than conditional, specific works have addressed *conditional* coverage in data with missing values, which is relevant to RAG’s conditional uncertainty. For instance, it has been shown that conformal prediction validity can be affected by the outcomes of missingness, and novel frameworks like generalized conformalized quantile regression have been developed to produce prediction intervals that are valid conditional on those patterns [294]. In the RAG setting, where context can be considered a latent variable that is “retrieved” or potentially missing—a direct bear of the retrieval-poisoning attacks from the previous section—such methods could be adapted to provide coverage guarantees that account for this additional source of uncertainty, ensuring that even if the evidence is partially unavailable, the final set remains valid and actionable.

The final layer of trustworthy RAG model deployment concerns **inference-time calibration and dynamic fallback** mechanisms, which operationalize the formal guarantees. While the aforementioned *statistical guarantees* [184] provide the safety harness, we must include the ability for the system to **abstain** from answering when the uncertainty exceeds a safe threshold. A straightforward application of conformal prediction can be used for selective classification, such as filtering out low-quality predictions via *conformal prediction for LLMs in multiple-choice QA*, which demonstrates that conformal prediction set size is tightly correlated with prediction accuracy [295]. Moreover, if we lack access to model internal logits, methods like “*API is Enough*” [296] show that constructing conformal predictors using *semantic similarity* yields valid and informative (“small”) sets, allowing for deployment in realistic, system-level settings.

These statistical guarantees and formulations must be synthesized with the design of the generation strategy itself. Training models with an awareness of credibility, as done in the *CAG* approach, can be synergistically combined with a conformational loop to mitigate the impact of unreliable information [35]. In high-stakes application, more recent work explores the construction of certifiably robust conformal frameworks against adversarial perturbations, where the generated set contains the true label with a guaranteed confidence level even in presence of an attacker [297]; this is a vital direction to ensure that crafted missing or misleading contexts cannot circumvent the system’s formal guarantees, thereby closing the loop with the security concerns from Section 6.3.

Thus, establishing a layered and rigorous suite of mathematical guarantees is an essential step for building a trustworthy RAG. The systematic deployment relies on a multi-pronged approach: (1) *Certify before generating* with guarantees on the retriever (CONFLARE) and on the generation (end-to-end via C-RAG); (2) *Calibrate in real-time* under dynamic and non-exchangeable distribution shifts (Adaptive Conformal Inference for Online Settings); and (3) *Terminal rejection or fallback* for cases that fall outside the confidence set. This robust overlay—spanning retrieval, generation, handling of shifts, and post-hoc calibration—is the mathematical counterpart to the threat analysis, ensuring that the system does not merely **claim** trustworthiness, but *proves* it using formally guarantee the security, privacy mitigation, and reliability necessary for real-world, high-stakes deployment.

7 Efficiency, Scalability, and Practical Deployment

7.1 Model Compression and KV-Cache Reduction

The deployment of Large Language Models (LLMs) in real-world applications presents significant computational and memory challenges, and these challenges are magnified in Retrieval-Augmented Generation (RAG) systems where the model must process both retrieved documents and user queries under strict latency and throughput constraints. The memory footprint of an LLM during inference is dominated by two primary components: the static model weights and the dynamic key-value (KV) cache, which stores intermediate attention states to avoid redundant computation. As context lengths and batch sizes increase—both of which are typical in RAG scenarios where retrieved passages extend the input context—the KV cache can easily surpass the model size itself, becoming the critical bottleneck in speed and memory usage [298, 299]. Consequently, optimizing model inference through both model compression and KV cache reduction is a crucial area of research, with a focus on balancing memory savings with minimal impact on output quality. This subsection delves into the techniques developed to address these challenges, categorizing them into model compression, KV cache reduction, and system-level memory management strategies.

Model Compression: Quantization and Pruning

Model compression, particularly quantization, aims to reduce the memory footprint of the model’s parameters. Post-training quantization (PTQ) is a popular method due to its efficiency and lower computational cost compared to full fine-tuning [300]. The primary challenge is that aggressive quantization to low bit-widths can lead to significant performance degradation. To mitigate this, advanced methods have been developed. For instance, SqueezeLLM proposes a Dense-and-Sparse decomposition, which allows for non-uniform quantization by separating outliers, which are sensitive to low precision, into a sparse, high-precision format. This approach achieves lossless compression to up to 3-bit precision for models like LLaMA, storing sensitive weights separately to maintain accuracy [245]. Similarly, SpQR (Sparse-Quantized Representation) identifies and isolates outlier weights that cause disproportionate error when quantized, storing them in higher precision while compressing other weights to 3-4 bits, leading to near-lossless compression for highly accurate models [301]. The concept of “pivot tokens” is another critical insight, as these tokens allocate most of the attention scores and are fundamental to the performance of a quantized model. IntactKV introduces the idea of preserving the KV cache of these pivot tokens losslessly, significantly improving the performance of quantized LLMs across various downstream tasks [302].

Beyond quantization, pruning is another compression technique that reduces the model’s complexity by eliminating less-foundational layers or parameter structures. Depth pruning, which removes entire layers of a model, has been shown to be a surprisingly effective strategy, competing with more complex width pruning methods, and is particularly beneficial in memory-constrained environments where it can lead to significant inference speed gains [303]. These model-level compression techniques are complementary to the more dynamic approaches for reducing the burden of the KV cache.

KV-Cache Compression and Eviction

While weights are static, the KV cache grows dynamically during the auto-regressive generation process. The footprint of this cache is a primary bottleneck, especially for serving requests with large batch sizes—a common situation when multiple indexing queries are processed concurrently. The storing of large KV cache also limits the inference speed as the GPU must load the entire cache for each token, which can cause computational idle cycles [298]. Therefore, several key strategies are proposed to compress the KV cache at runtime.

- **Quantization of KV Cache:** Quantizing the KV cache is a direct approach to reduce its

memory footprint. The KIVI method presents a tuning-free 2-bit quantization algorithm. Key to its success is its asymmetric, per-channel quantization for the key cache and per-token quantization for the value cache, based on an in-depth study of the element distribution. This method enables models to use up to 2.6x less memory, resulting in up to 4x larger batch sizes and significant throughput gains [298]. More nuanced approaches, such as MiKV, suggest a mixed-precision KV cache. They posit that evicting KV pairs entirely can lead to information loss, causing a severe degradation in the quality of generation. MiKV uses low-precision for “evicted” pairs and high precision for important pairs, offering a better trade-off between compression ratio and quality [172]. Other works, like GEAR, combine quantization with low-rank approximation and sparse matrices. By quantizing most entries to ultra-low precision and separately handling structural errors and outliers, GEAR achieves near-lossless 4-bit compression [171]. WKVQuant is another framework which quantizes weights and activations simultaneously, incorporating additional strategies such as a two-dimensional quantization scheme for the KV cache to better handle its unique distribution [304].

- Eviction and Selection Policies:** Beyond quantization, many methods have been proposed to identify and retain only the most “important” tokens in the KV cache. The “persistence of importance” hypothesis, which asserts that only pivotal tokens with substantial influence at one step will significantly influence future generations, is the premise of the Scissorhands system. It maintains the KV cache at a fixed budget by keeping these pivotal tokens with higher probability, reducing the cache memory by up to 5X [305]. Similarly, the H2O (Heavy-Hitter Oracle) method observes that a small portion of tokens, the “Heavy Hitters,” contribute the success of the attention scores. The algorithm formulates the KV cache eviction as a dynamic submodular problem to retain a balance of recent and heavy-hitter tokens, which leads to significant memory compression [306]. Keyformer identifies “key” tokens by using a novel score function, retaining only these tokens in the KV cache, thus ensuring efficient memory bandwidth utilization and latency [307]. Algorithms such as SnapKV discover that each attention head consistently focuses on specific prompt features during generation, allowing it to compress KV caches based on selected important positions in an “observation window,” leading to a significant increase in generation speed and memory efficiency [173]. CORM addresses the problem by finding that the similarity of query vectors between adjacent tokens is high, and future queries can rely on attention information from a small portion of preceding queries. This forms the basis for a policy that dynamically retains key-value pairs, reducing inference memory usage by up to 70% [308].
- Learnable Compression: Dynamic and Lossless:** Instead of relying on post-hoc heuristics or policies, some methods, such as Dynamic Memory Compression (DMC), retrofit pre-trained LLMs to learn how to compress the KV cache at inference time. The model learns to apply different compression rates per head/layer and through continued pre-training, it achieves up to a 3.7x increase in throughput while preserving performance [309]. Alternatively, Lossless Compressed Memory Attention (LoMA) introduces a specialized training procedure to learn context compression during the generation process itself. By compressing the KV cache after a set number of tokens and compressing it to a target length, LoMA achieves lossless KV cache compression [175]. In a similar, the model can be retrofitted with a compressor that performs incremental compression on-the-fly for better integration [310].
- Memory Management with Paging-inspired Systems:** Beyond compressing, efficiently managing the allocated memory can have a massive impact on the other bottlenecks. The core idea is encapsulated in PagedAttention, which is inspired by memory management strategies

from the operating system. The KV cache is divided into fixed-size blocks that can be managed via a table, ensuring near-zero waste. On top of PagedAttention, systems like vLLM improve the serving throughput by 2-4x with the same latency, as the memory savings allow for much larger batch sizes [299]. Advanced algorithms such as SqueezeAttention can further optimize this by identifying the importance of attention layers and applying a dynamic budget across the sequence of layers [174]. These memory management techniques directly reinforce the system-level optimization strategies discussed in the following subsection, where the efficiency of both model weights and KV cache access is crucial for deployment.

7.2 System-Level Optimization for Latency, Throughput, and Scaling

System-level optimizations are paramount for the practical deployment of Retrieval-Augmented Generation (RAG), where the computational demands of large language models (LLMs) intersect with the latency and bandwidth constraints of real-time knowledge retrieval. While the previous subsection focused on reducing the memory footprint of individual components—through model compression and KV cache reduction—this subsection addresses the equally critical challenge of orchestrating computation and data movement across hardware resources. The performance of a RAG system is not solely determined by model architecture or algorithmic innovations; it is profoundly influenced by the engineering decisions governing how computation and data communication are synchronized across heterogeneous hardware. Optimizing for both latency and throughput requires a holistic approach that spans distributed parallelism, hardware-software co-design, and strategic management of memory bandwidth.

To address the inherent bottleneck of memory bandwidth and achieve efficient scaling, distributing the model across multiple processing units is often necessary, but it introduces communication overheads that can undermine efficiency. **Distributed parallelism**, specifically tensor parallelism, splits model layers across devices, requiring careful orchestration of communication. The communication required for tensor parallelism can serialize execution and reduce scaling efficiency, as the producer layer must finish computing before data can be sent to the next device [311]. To mitigate this, fine-grained hardware-software co-design can be employed to overlap communication with computation, effectively hiding data movement latency behind useful work [311]. Furthermore, traditional transmission mechanisms for tensor parallelism often suffer from straggler issues and bandwidth/resource contention due to static hardware allocation and dynamic resource contention from concurrent workloads [312]. Hence, dynamic workload balancing and flexible execution strategies are critical for achieving robust performance under varying system loads.

Given the resource-constrained nature of end devices, a **hardware and software co-design** approach is essential to enable the deployment of RAG-capable LLMs. Large models with hundreds of billions of parameters cannot be accommodated on a single device, and in resource-constrained environments, the I/O bottleneck from moving weights and KV-caches onto and off of accelerators can cause severe stalls. A promising approach is to decompose the transformer model into two conceptually distinct parts: compute-bound operations (e.g., matrix multiplications) and memory-bound operations (e.g., attention). The memory-bound portion can be efficiently processed by CPUs, which offer high memory capacity and bandwidth, while the compute-bound portion is processed on the GPU, effectively bypassing the CPU-GPU bandwidth bottleneck [313, 314, 315]. This strategy, deployed in systems like *HeteGen*, demonstrates that low-latency inference can be achieved by overlapping memory transfer with compute on heterogeneous CPU/GPU systems [314]. Similarly, an NPU-PIM architecture that dynamically orchestrates workloads based on whether kernels are compute-bound or memory-bound can significantly improve performance for batched LLM inference [315]. Beyond these execution strategies, the hardware architecture itself can be tailored to the

workload. For instance, the memory hierarchy of a GPU, such as the cache size and DRAM bandwidth, can be specialized for deep learning workloads rather than generic high-performance computing, thus maximizing utilization of the memory system [316]. Such specialization allows systems to strike a higher-performance yet more power-efficient balance.

A complementary high-level design philosophy is to **disaggregate compute phases** to better manage distinct service-level objectives. LLM serving has two phases with strikingly different compute and memory characteristics: prefill, which is compute-bound and batch-friendly, and decoding, which is memory-bound and progresses one token per GPU. When batched together, these phases can interfere with each other. Disaggregating them—allocating prefill and decode computation to distinct GPUs—isolates their resource demands and allows independent parallelization and hardware configuration [317]. This not only eliminates interference but also allows the system to independently shard and scale each phase to meet time-to-first-token (TTFT) and time-per-output-token (TPOT) service-level objectives. As a result, disaggregation substantially improves the sustained request rate under strict latency constraints [317].

Beyond distributing the model, the throughput of LLM serving systems is often limited by the repeated read and caching of key-value storage. The batch size and throughput are frequently constrained by GPU memory capacity, which directly limits the number of concurrent sequences that can be processed. Analyzing and optimizing how KV caches are accessed can yield significant speedups. However, simply moving the KV cache to host memory can bottleneck performance by exceeding the CPU-GPU bandwidth. This can be bypassed by distributing the model across the heterogeneous system, where memory-bound tasks are processed on the CPU (with its large memory capacity) and just the compute-bound tasks are sent to the GPU, thus containing expensive data transmission [313]. Through such optimization, including the system in *FastDecode* demonstrates significant performance improvements.

Optimizations can also occur at the **scheduling level**. In a real heterogeneous system, jobs contend for shared memory and cache, so efficient **interference-aware scheduling** is integral. In the context of RAG, scheduling is also critical for jointly executing the retrieval and generation phases. For example, retrieval can be launched concurrently with the model compute of the previous turn. Moreover, the way the model is scheduled across devices (the memory access flowing along the distributed memory hierarchy) is critical. Simply supporting high bandwidth or large capacity of the memory system is not sufficient; carefully managing where data is placed and how it moves between devices is foundational to effective efficiency [318, 319]. Simple concurrency without careful scheduling can lead to lock-step patterns and idle hardware, posing performance penalties [320]. Scheduling frameworks that reason about computational intensity and allocate resources accordingly—like the *Mage* system—are necessary to maximize throughput [321].

Oftentimes, *the compute must be coupled with the memory*. The optimal scheduling of work spans **CPUs, GPUs, and FPGAs**. This is enabled by **offloading** specialized kernels to accelerators, particularly memory-bound kernels such as KV-cache operators, which reduces the data transmission overhead on the smaller GPUs and hides communication latency within computation [315, 313]. This allows more specialized hardware to be exploited alongside a generic, portable compute substrate.

The **co-design** also extends across the entire software stack, from scheduling to compilation. The final system must incorporate **tracking** mechanisms to monitor execution phases, as these phases inform the decision of where and when to place or move data. Even the performance of specialized network interconnects must be tailored (e.g., for the training and inference communication patterns) [322]. This entire web of hardware and software optimizations needs to be driven by an overarching goal across the entire data platform. In the final break down of the problem, it becomes clear.

From this holistic view, we have identified a set of key components to the system engineering problem. This subsection has delved into how to **orchestrate** the memory and compute of different

components of a serving system, how to layer **capable and cooperative hardware** to give parallel efficiency, and how to **co-optimize** shared KV cache via smart scheduling of compute and I/O under a single performance rule. Importantly, these system-level strategies directly interact with the memory reduction methods described earlier. Quantized weights and compressed KV caches not only save memory but also reduce the bandwidth required for weight and activation transfers, making them particularly complementary to the heterogeneous execution and offsetting strategies described above. Furthermore, they complement the dynamic execution strategies presented in the next subsection, where algorithmic advances work at the token or layer granularity to adaptively reduce computations, while the system orchestration discussed here provides the broader machinery for scheduling and resource management. The latest generation of hardware provides the basic tools, yet the nuanced coordination between “storage” and “compute” is ultimately what most impacts the latency, throughput, and scaling ceiling of RAG systems as they rise.

7.3 Advanced Decoding and Dynamic Execution Strategies

The autoregressive nature of large language models (LLMs) creates a significant bottleneck for inference, as tokens are generated sequentially, leading to low hardware utilization and high latency. To overcome this, advanced decoding and dynamic execution strategies have emerged, moving beyond standard token-by-token generation to parallelize, skip, and intelligently manage computational resources. These techniques are crucial for scaling LLMs in practical settings, especially for long-context and high-throughput applications, and directly complement the system-level optimizations discussed earlier by providing algorithmic avenues to reduce the computational burden and memory footprint.

Speculative Decoding: Draft-and-Verify Paradigms

A prominent family of techniques for accelerating inference is speculative decoding, which draws inspiration from the speculative execution used in computer architecture [323]. This approach typically employs a smaller, faster “draft” model to generate a sequence of tokens, which are subsequently verified in parallel by the larger “target” model [324]. The target model can accept or reject the draft tokens, ensuring the final output distribution is preserved. This method can yield substantial speedups for very large models like Chinchilla [324]. A core challenge is ensuring that the draft model closely aligns with the target model’s output, as well as the need for the draft model to be small enough for speed but capable enough for accuracy. To improve alignment, knowledge distillation can be used to fine-tune the draft model based on on-policy data from the target model, a technique known as [325]. The scalability and diversity of the drafting process can also be improved by organizing draft candidates in a tree of tokens rather than a single sequence, with dynamic programming algorithms finding near-optimal tree structures to verify multiple candidate sequences simultaneously [326]. Similarly, [327] uses a tree-based approach with a sampling-without-replacement strategy to maximize diversity among candidate tokens from the draft model. In systems with long contexts, hierarchical speculative decoding can also be applied, where an intermediate draft based on dynamic sparse KV retrieval accelerates the drafting stage as well [328].

Early-Exit Architectures and Dynamic Layer Skipping

Another efficiency avenue is dynamic computation, which reduces the number of layers a model must process for each token. This approach is conceptually orthogonal to the system-level heterogeneous execution discussed earlier, as it directly reduces the total compute demand rather than optimizing data movement. Early-exit strategies allow tokens to finish processing at intermediate layers if a confidence threshold is met, but they have limitations as they have to wait until the last token in a batch exits before they can stop computing [329]. If token-level early exits

are applied in batches, the system may have to wait for the slowest token to exit, which is not suitable for batch optimization, and can therefore conflict with the throughput goals of system-level scheduling. To overcome these limitations, frameworks such as [330] set a single exit point for all tokens in a batch at a given sequence position. It guarantees a monotonic decrease in exit points, thus allowing the heaviest computation to be spent on upper layers. This method is fully compatible with batching and Key-Value (KV) caching [329]. An alternative method to layer-skipping is to use a “Unified Layer Skipping” strategy that selects the number of layers to skip based on a target speedup ratio, independent of the input sample, so it is compatible with batch decoding and KV caching [331]. Another early-exit approach, [332], dynamically allocates varying amounts of compute per token and uses a *sequence-level confidence measure* to decide on early exits, with considerations for sequence-level constraints and to ensure hidden representations (are properly managed) [332]. However, effectively implementing early exits in models with both an encoder and a decoder, such as unified vision-language models, presents challenges. To address this, [333] proposes a method for *feedback across modalities* and enables multiple exit points in both the encoder and decoder.

KV-Cache Compression and Dynamic Eviction

The KV cache for the entire context can eventually become a bottleneck, consuming a large amount of memory and limiting parallelism. This is especially daunting for longer sequences and subsequent large batches, undermining the efficiency objectives of system-level scheduling. This has motivated a large number of strategies for compressing the cache, typically by evicting the “least important” tokens. A widely-used approach is to make decisions based on a token’s importance, such as the “Heavy Hitters” (H2O) method, which retains tokens that have a large impact on the attention score [306]. Alternatively, [305] builds on the idea of “importance persistence”, where only pivotal tokens that have a prevalent influence are kept, reducing the memory footprint by up to 5X in practice. Instead of fully evicting tokens, some recent work uses a mix of precision quantization rather than a complete eviction [172]. This work shows that keeping even a small amount of information from evicted tokens in low-precision formats can substantially recover any incurred degradation. More refined compression methods like [171] use a combination of quantization, low-rank approximations, and a sparse matrix to correct errors, producing near-lossless outputs. Importantly, these compression techniques can be composed with the system-level memory management strategies discussed in the previous subsection, allowing the KV cache to fit within constrained CPU or heterogeneous memory tiers.

System-Level Scheduling and Parallelism

Software systems also create significant gains beyond algorithmic strategies. Using system design where prefill and decode phases are run in parallel with the disaggregation of the prefill and decode phases avoids the stall created by sequential phases, improving TTFT and JCT [317]. Many eager scheduling algorithms heavily depend on the batch size in the schedule or latency constraints for the model. To handle workloads that mix different characteristics, as on a processor where a CD is the system, or where the model’s decode phase is often separated, systems like Sarathi-Serve use “stall-free” scheduling to improve the latencies of new requests by performing chunked-prefilling. The model runs a chunked prefill while keeping the decode and batch sizes as large as possible, avoiding the stalls that arise from waiting for a prefill to finish [334]. Additionally, given that multi-turn interactions often repeat the same history, [335] monitors the memory hierarchy to reuse the KV cache from previous turns to avoid redundant computation, decreasing the time-to-first-token (TTFT) by up to 87%.

Asynchronous and Parallel Decoding

Another avenue for reducing latency is to generate tokens in parallel rather than sequentially. The prior approaches to layer skipping, such as early exits, often proceed at different rates for different tokens. [336] proposes a framework that *synchronizes the decoding* of the current token with

previously generated tokens that have exited early. In this way, the model can observe predictions from both an current token and the early-exited tokens, improving the overall predictive accuracy. This synchronized approach can lead to a more robust and efficient decoding process, which is compatible with batched inference.

Hardware-Aware and Model-Generic Optimization

In the quest to run large models efficiently, the hardware-software co-design is paramount. For example, accelerating speculative decoding with hardware-awareness is a direction summarized in [337]. These systems might use high-bandwidth memory caches in the “KV” space. As models grow and long-context become more standard, the specializations needed for dynamic execution will each have a measurable effect, and when combined with the co-design and system-level orchestration discussed previously, these techniques must be carefully selected and integrated on a per-deployment basis to achieve the optimal latency, throughput, and cost trade-offs.

7.4 Cost–Accuracy Trade-offs and Lightweight Deployment in Resource-Constrained Environments

Deploying Retrieval-Augmented Generation (RAG) systems in production environments is a complex balancing act between maximizing output quality (accuracy, faithfulness, reasoning capability) and minimizing operational costs, which encompass latency, monetary expenditure, and energy consumption. This challenge is amplified in resource-constrained settings—such as edge devices, serverless platforms, and mobile environments—where computational power, memory, and bandwidth are limited. While the previous subsection introduced algorithmic and system-level techniques to accelerate and streamline LLM inference—such as speculative decoding, dynamic layer skipping, and KV-cache compression—these methods ultimately assume a fixed deployment model. In practice, however, the infrastructure hosting a RAG system must grapple with variable workloads and heterogeneous hardware budgets. This subsection extends that discussion to the orchestration layer, presenting strategies for navigating the cost–accuracy trade-off, and emphasizing a departure from static, monolithic deployments toward dynamic, hierarchical, and workload-aware architectures that prioritize efficiency without sacrificing the quality benefits enabled by the decoding and execution techniques discussed earlier.

A fundamental principle in cost-efficient RAG deployment is recognizing that resource demands are rarely uniform across time. Production workloads often exhibit significant temporal variability, including bursty traffic patterns and unpredictable spikes, which make static resource provisioning—allocating a fixed set of resources to handle peak load—inefficient, leading to severe underutilization during off-peak hours and high latency during surges. To address this, modern serving systems are increasingly designed with elastic scheduling capabilities. Elastic scheduling allows the system to dynamically scale computational resource allocation in response to real-time workload changes, thereby aligning resource consumption with actual demand. For instance, research on dynamic resource provisioning highlights the importance of scaling models to handle unpredictable and bursty request arrival rates while carefully balancing latency and accuracy requirements [338]. Furthermore, architectures that incorporate capacity loaning to automatically scale infrastructure to satisfy fluctuating resource availability can provide high performance without permanent over-provisioning [339]. This dynamic scaling ability is not merely a convenience; it is a core mechanism for maintaining cost and performance efficiency, complementing the algorithmic speedups from the previous subsection by ensuring those speedups are used only when and where computational resources are actually available. Some systems demonstrate significant improvements in resource utilization and latency during bursty conditions [340].

Within this dynamic resource allocation, the well-known trade-off between latency and utilization

often leads to suboptimal scheduling decisions. This problem is particularly acute in systems with unpredictable workloads and strict service level objectives (SLOs), where scheduling weights must dynamically adapt to the traffic characteristics [341]. Similarly, for managing heterogeneous latency-critical applications, methods to decide how to allocate compute resources and manage them dynamically can be strategically deployed to balance application performance and system utilization [342]. The consideration of heterogeneous and dynamic workloads benefits from borrowing ideas from networking, where differentiated service primitives define a bounded degradation in service for less-critical traffic [343]. By applying this “bounded degradation” to RAG, some queries—e.g., low-priority background analysis—can undergo a slightly lower quality-of-service in exchange for the system’s ability to guarantee high quality for more critical, real-time tasks. This philosophy is particularly useful in light of the early-exit and layer-skipping techniques described earlier; a system can map a low-priority query to a pool of tokens processed with early exits, while high-priority tasks use full computation. It makes the system will become more resource-efficient, gracefully degrading performance on less critical tasks to make resources available for more demanding ones, while still retaining the architectural mechanisms to ensure the final output distribution is faithful when needed.

A practical and increasingly prevalent strategy to address both resource constraints and cost is the use of hierarchical resource-tiering. In the cloud context, this takes the form of multi-tiered architectures, where a workload can be intelligently routed to different types of hardware. A system can utilize a hierarchy of computational resources, from cloud-based inference servers to cheaper, less powerful edge or endpoint devices, dispatching queries to the most appropriate tier based on their complexity and accuracy requirements. For instance, a simple query like fact-checking a date might be routed to a lightweight model on a local server, while a complex, multi-hop reasoning chain might be sent to a large, more expensive model in the cloud and, similar to the dynamic execution strategies from earlier, could use speculative decoding with a draft model for speed on resource-rich nodes. This is analogous to load balancing where user requests are assigned to distributed servers and can be handled in a decentralized manner [344]. The operational principle is to adapt to requests that are interleaved across different models, not just the individual complexity of the query, but also the context of the most profitable action based on the overall load. The use of a range of models, from large to small, is a system leveraging this heterogeneous hierarchy: the essential goal is to have processors cooperate and maximize cluster usage by balancing load, cost, energy, and performance while orchestrating resources for workloads [345]. In environments lacking circuit bandwidth, groups of systems can provide real-time benefits but also yield an opportunity to provide a mechanism for principle exchange between configurations and costs [339]. We can further adapt these principles to benefit from mobility and extreme resource-constrained scenarios, using the ability to move computation to the point where the data resides, ensuring that processing occurs within a deadline or budget [346].

The shift toward serverless computing presents a new layer of challenges and opportunities for RAG deployment. Serverless designs are historically accountable for managing and allocating resources for a powerful fleet of Kubernetes-oriented tasks. However, the dynamic scheduling of function instances results in the problem of “cold start” which can exacerbate the cost–accuracy trade-off [347]. Serverless scheduling is a distinct class of problem that combines load balancing, memory allocation, and the management of application instances [348]. Efficient scheduling of requests to functions in a serverless model is key to reducing these costs [349]. With a bit of management overhead, we can guarantee SLOs while over-subscribing resources. For example, adaptively bin packing jobs to fewer containers using function-aware container scaling and intelligent request batching can help in this regard [340]. The key is to ensure that serverless functions are “warm” to avoid cold starts [347], and implementing a pool-based approach to mitigating cold-starts

in a resource-conserving deployment is essential [347].

Similarly, significant consideration must be given to what it means to be cost-effective on the hardware itself. Leveraging heterogeneous and mobile-grade hardware, with strategies for co-scheduling tasks to co-locate small, latency-sensitive queries with more compute-intensive workloads, requires strategies in which these disparate goals should be carefully coordinated [350]. When resources are limited, such as on a device with low memory and no powerful GPU, the serving machine must be designed to optimize for the task [351], and all related processes should be managed with knowledge of the available resources to handle the strict timing of inference [352]. For example, a system may use a *passive* approach that compensates for the lack of variability by adopting a strategy that a priori defines the computational expenditure for each request. This is strategically akin to the batching of requests: multiple requests can share the same model forward pass and process them returning similar results, minimizing the “dry bits”. This also ties back to the KV compression methods: rather than holding a full KV cache for every user, a cost-efficient system can compress the cache for low-priority requests, acknowledging that minor losses in quality are acceptable.

Ultimately, the solution to the cost-accuracy trade-off is not a single technology but a systems approach that combines the elastic computing power of the cloud with the near-real-time response of edge infrastructure to advance computing as an ecosystem. The goal is to move beyond independent components and toward a tight “full-stack” approach where the network, distributed compute systems, and the allocation policies are co-designed with an awareness of both the accuracy needs and the cost of handling the computation. For bursty traffic, the backend must not only perform but also prioritize the needs of leading users and multi-tenant service demands. The system must be able to identify the bottleneck and decide whether to spend a low-latency instruction on a single tier or to use a unified architecture and dispatch the query to an appropriate, lighter model that, although lower in quality, suffices for the request [353].

In conclusion, the cost-accuracy trade-off in RAG deployment in resource-constrained environments is dominated, not by a single technology, but by an ecosystem design principle that just-in-time quality and adaptivity to load. This builds directly on the algorithmic resource reductions examined previously: speculative decoding reduces total compute, early-exit reduces per-token compute, and KV-compression reduces memory; combined with system-level components—elastic scaling, hierarchical routing, and serverless bin packing—to ensure that compute is only paid for when it is absolutely necessary. The pivot to “dynamic” is the key: elastic scheduling to adapt to the load, a model (and sometimes an inference) tier for task-specific resource usage, and the ability to place computational power where and when it is needed, whether this is at the edge, in the center, or both. Each of these strategies is important, but their blend is what allows a high-quality LLM application to be delivered, even on a bounded, fluctuating budget. Therefore, lightweight deployments should be designed as adaptive, resource-aware systems, where the complexity of the queries—and not just the model logic—becomes the core guiding principle for system design, aligning the algorithmic efficiency of the previous subsection with the deployment pragmatics of this next generation of serving infrastructure.

8 RAG for Multimodal, Multilingual, and Domain-Specific Applications

8.1 Multimodal RAG: Architectures and Applications Across Modalities

The extension of Retrieval-Augmented Generation (RAG) to multimodal data represents a significant paradigm shift, moving beyond the limitations of text-only knowledge sources to leverage the rich, heterogeneous information inherent in images, videos, and structured clinical data. While

foundational RAG systems primarily index and retrieve textual corpora [23], the real-world information landscape is inherently multimodal, necessitating architectures that can ingest, align, and reason over multiple modalities simultaneously. This subsection surveys the key architectural innovations, application domains, and challenges that define the emerging field of multimodal RAG, complementing the cross-lingual challenges discussed previously by addressing a parallel dimension of heterogeneity: the diversity of data types rather than languages.

A critical challenge in deploying RAG for large-scale multimodal data is the computational cost and latency associated with transcribing and indexing the entire content upfront. Traditional RAG systems often perform a one-time, exhaustive conversion of all media into text descriptions, an approach that is highly inefficient for large video repositories and can lead to irreversible information loss, as not all semantic information in a video or image is readily captured in text [354]. To address this, systems like iRAG introduce an incremental workflow, which initially creates a quick, lightweight index of the multimodal data and then, upon receiving a user query, opportunistically extracts more detailed context from specific, relevant segments on-demand. This avoids the high processing cost of upfront conversion while ensuring responses are grounded in specific fine-grained aspects of the original media, directly mitigating the issue of information loss during data ingestion [354]. This pragmatic ingestion strategy mirrors the retrieval-side adaptations seen in multilingual systems, where efficient bridging of heterogeneous representations is paramount.

Beyond lifecycle management, integrating structured knowledge sources such as knowledge graphs (KGs) with multimodal data has proven highly effective. A key example is the REALM framework, which addresses the limitations of multimodal electronic health records (EHR) analysis [355]. In this case, a large language model (LLM) is used to encode long-context clinical notes while a recurrent model (GRU) processes time-series vital signs. The architecture goes a step further by prompting an LLM to extract disease-related entities from the text and align them with an external, professionally curated medical knowledge graph (PrimeKG). This multi-stage, knowledge-of-the-loop approach ensures that the retrieval is aligned with clinical standards and does not introduce hallucinations into the retrieved context, effectively blending the LLM’s generation capacity with the structural validity of a KG [355]. This emphasis on grounding retrieved evidence in structured, verifiable knowledge is a recurrent theme that strengthens the trustworthiness of multimodal RAG systems.

A more intricate architecture for multimodal retrieval and generation is the *Multimodal Retrieval-Augmented Generator (MuRAG)*. Unlike earlier systems that retrieve only text latent or vectors [356], MuRAG accesses an external non-parametric memory containing both images and text, allowing the system to integrate visual knowledge with textual knowledge. By pre-training on a mixture of image-text and text-only corpora through a joint contrastive and generative loss, MuRAG implicitly learns to align and retrieve across modalities, leading to substantial improvements in open-domain question-answering tasks that require reasoning over both images and text [357]. This can involve complex “multi-view” retrieval where the query itself can be rewritten into many semantic forms, as seen in MVRAG for legal and medical case retrieval, which treats different domain perspectives to enhance recall and precision [358], paralleling the cross-lingual re-querying strategies used to bridge language gaps.

However, the goal of retrieval is often to generate or “fuse” a holistic representation for downstream tasks. Success in this area hinges on the robustness of fusion architectures, which can be generally grouped into early, late, and intermediate or hybrid strategies [359]. One approach to ensure robustness is to precisely model the correlation between modalities and estimate the interaction through representations that preserve fidelity. Variational fusion approaches have been proposed as a way to minimize information loss between unimodal and multimodal representations [360]. While truly intelligent systems must adapt to decide “how” to combine data, leading to

dynamic, context-aware fusion models rather than deterministic concatenation [361].

In contrast to fusing full outputs, recent advances focus on more sophisticated intermediate fusion. The *Progressive Fusion* paradigm is a method that uses backward connections to make later-stage fused representations available to early layers of a network, retaining the advantage of late fusion while improving the fusion quality by the richer, high-level guidance at the early stage, which enhances prediction quality [362]. This iterative refinement technique is distinct from standard fixed-layer-based strategies, introducing a dynamic recursion process that is also captured by deep equilibrium (DEQ) models, which seek a fixed point of the fusion process to capture complex interactions [363]. A robust *Multimodal DenseNet* architecture similarly extends beyond simple concatenation with dense connectivity, providing flexibility in fusing multiple levels of features [364]. These architectures highlight a shift toward adaptive, hierarchical fusion over static concatenation, a theme that parallels the need for adaptive retrieval in cross-lingual settings.

Critically, in practice the predominant architectures will not always have access to all modalities at test time. Models often face missing or corrupted modalities, which can severely degrade performance if the model lacks the supporting information to maintain the integrity of meaningful correlations. The *MultiModN* model specifically addresses this challenge by fusing latent representations in a sequence of an arbitrary number of modalities and adding a level of modularity at the late stage, making it robust to various patterns of missing data while enhancing interpretability [365]. Another family of solutions introduces parameter-efficient adaptation, which enables flexibility to fully implement various missing-modality scenarios at test time, compensating for transmission dropouts [366]. When data is limited, “Borrowing Treasures from Neighbors” addresses the acute scenario of both missing modalities and very limited data samples by retrieving examples with full modalities as “neighbors” for in-context learning at test-time [367]. In these cases, the retrieval component becomes not just a source of knowledge but a mechanism for data imputation, directly bridging the gap between sparse observations and complete representations—an interactive process that logically extends the retrieval-driven language gap shown in multilingual RAG.

Yet, the main unresolved challenge of multimodal RAG is the adequacy of alignment between modalities. When representing images and text in a common vector space, the “modality” stems from the need to have a appropriately learned representation for the embedding space, which can only be achieved by proper pre-training, e.g., CLIP-style contrastive learning [368]. Failing to align retrieved representations forces the model to rely on less informative shallow correlations. For example, in medical imaging, a separate multi-modal retrieval index was built from chest X-rays and their reports to augment the image representation with relevant past cases; yet fine-tuning the extraction of disease-related content (over general visual presentation) was essential to ensure that the retrieved reports benefitted the correct classification [369].

In conclusion, the field of multimodal RAG is demonstrating sophisticated and pragmatic approaches, from the incremental ingestion of videos and knowledge graphs to the interactive and multi-level fusion of health and imaging. The recent progression highlights a shift from static, one-shot pipelines to dynamic, adaptive systems that emphasize efficient ingestion (iRAG), external structured knowledge integration (REALM), cross-modal retrieval (MuRAG), and robust intermediate fusion (Progressive Fusion). In summary, the development of robust multimodal RAG systems hinges on the design of architectures that prioritize both efficient retrieval of relevant information and fidelity of the fusion, coupled with robustness to inevitable data imperfections—both data missing and data heterogeneity.

The only way to achieve true enhancements is to adopt architectures that assign relevance and representation of the modality-specific signals throughout the early and intermediate fusions. The hedging principle, especially the dynamic recursion between the modality-specific signals, is an advanced trend [366]. For clinical and medical AI to expand and be capable, these multimodal

systems must be consciously managed, maintaining robustness and interpretability in the loop. This promises a future of true multimodal RAG for diagnostics and education, while also echoing the cross-lingual challenges of grounding in low-resource settings—both require adaptive, retrieval-driven strategies to overcome heterogeneity and scarcity.

8.2 Multilingual and Cross-Lingual RAG

Multilingual and cross-lingual retrieval-augmented generation (RAG) represents a critical frontier in extending the benefits of grounded generation to the vast majority of the world’s languages, which remain underserved by mainstream natural language processing systems. Building directly upon the multimodal heterogeneity discussed in the previous subsection, this domain confronts a parallel yet distinct form of diversity—the diversity of human languages—and similarly necessitates adaptive retrieval and generation strategies to bridge these gaps. The core challenge in this domain is twofold: ensuring high-quality retrieval of relevant information across language boundaries, and enabling the generator to faithfully synthesize answers in the target language, all while contending with data scarcity and typological diversity. This subsection provides a comprehensive overview of the techniques, benchmarks, and inherent challenges that define the current landscape of multilingual RAG, and sets the stage for the domain-specific adaptations explored in the following subsection.

The foundation of any cross-lingual RAG system lies in its retrieval component, which must bridge the language gap between a user’s query (often in a high-resource language like English) and a multilingual knowledge corpus. This challenge is analogous to the retrieval-side adaptations seen in multimodal systems, where bridging heterogeneous representations is crucial. One prominent class of methods focuses on creating linguistically agnostic embedding spaces. Dense retrieval models trained with multilingual objectives, such as the multilingual DPR (mDPR) adaptation for the Mr. TyDi benchmark [370], aim to embed semantically related text from different languages in proximity. However, early studies from [370] and [371] revealed that these representations often only achieve “weak” cross-lingual alignment, where relevant cross-lingual pairs are closer than unrelated same-language pairs, but a “strong” alignment requiring cross-lingual proximity to outrank unrelated same-language pairs is harder to achieve. To improve this, methods have been developed to induce language-agnostic representations, for instance by removing language identity signals through vector space re-alignment and text normalization techniques [372]. A more practical and robust alternative to circumventing the weaknesses of zero-shot alignment is to train a cross-lingual retriever that can directly score relevant documents cross-lingually for a given query, as demonstrated by the CORA model, which further emphasizes the need for generation to be grounded in the target language [373].

Another viable strategy to enhance retrieval fidelity is to pair a first-stage retrieval system with a strong retriever, often followed by a reranker. These rerankers are often fine-tuned on a small amount of cross-lingual relevance labels and can use deep bilingual representations [374]. Large language models (LLMs) have also shown promise as zero-shot rerankers for cross-lingual information retrieval, a recent and effective avenue to boost retrieval quality for low-resource languages [375], though early results have shown that, while promising for bridging languages, the effectiveness of this approach depends on LLMs’ multilingual capabilities for the target languages [375]. The domain-specific generality of rerankers is also a concern; supplementing this, training rerankers that are compact and generalize well across domains, as shown in the context of Polish, is crucial for improving retrieval quality in languages beyond English [53]. Another avenue that has proven effective is data augmentation—generating synthetic queries in the target language to enrich passage representations [376, 110].

Even with a solid retrieval system, the entire RAG pipeline must be tailored to handle the

heterogeneity inherent in multilingual and cross-lingual settings. The benchmark dataset **Mr. TyDi** provided a crucial testbed, and other benchmarks have followed, each purpose to assess robustness in multilingual RAG. NoMIRACL comprehensively evaluates LLM robustness to irrelevant passages in both the relevant and irrelevant subset across 18 different languages, revealing that most models fail to distinguish relevant from non-relevant evidence and often hallucinate [265] [265]. Research on cross-lingual open-retrieval QA for 16 languages, such as the MIA Shared Task on Cross-lingual Open-Retrieval Question Answering for 16 diverse languages, emphasizes the challenge of retrieving across language boundaries, and in this task, retrieval often faces impediments for low-resource languages such as Tagalog and Tamil [377]. To bridge the language gap often encountered in multilingual retrieval, translation and pivoting via a high-resource language like English is often used [378]—a strategy that reflects the cross-lingual re-querying techniques used in multimodal RAG to handle missing modalities.

Beyond the technical mechanics of retrieval and generation, the success of multilingual RAG hinges on a foundational question: how to handle low-resource languages. The research enterprise must therefore take into account not just linguistic transfer but also the scarcity of data in target languages. For generating in a low-resource language, methods have been developed to create synthetic supervision—for instance, performing task-adaptive pretraining with large-scale synthetic answers and leveraging a pivot language for transfer—all to be applied in a target language setting [373] [379]. For instance, CORA generates answers in the target language and directly uses cross-lingual data without the need for translation [373]. Others have leveraged synthetic supervision to pretrain a model across languages, allowing for zero-shot transfer to low-resource languages [380], echoing the incremental ingestion and retrieval strategies used to overcome sparse multimodal data described earlier.

Finally, extending RAG to multilingual domains also implies mitigating against the potential for hallucination when retrieval results are unhelpful or unhelpful. To balance the model’s internal knowledge with parametric memory, we see advancements like Self-RAG, which trains model to reflect on and assess where to rely on retrieval versus its own knowledge, enhancing the model’s ability to utilize cross-lingual evidence and explicitly judge whether retrieved passages contain useful information [41]. To choose valuable evidence, methods such as fine-grained reranking provide insights into optimal retrieval choices [375] for multilingual scenarios, and the ARAGOG study compares retrieval methods to guide configuration choices [381]. Ultimately, real systems must understand when the retrieval corpus cannot satisfy the user’s query, so they neither hallucinate nor overwrite reliable data—a core principle of robust multilingual RAG that directly parallels the flexibility and robustness required for multimodal systems, and which will become even more critical in the high-stakes domains where RAG is increasingly applied.

8.3 Domain-Specific RAG in High-Stakes Fields: Healthcare, Legal, and Finance

The deployment of Retrieval-Augmented Generation (RAG) in high-stakes domains such as healthcare, legal, and finance represents one of the most consequential applications of the paradigm. Unlike general-purpose question answering, these fields are governed by strict requirements for accuracy, verifiability, explainability, and regulatory compliance, where a single hallucinated or misattributed output can have severe consequences. Consequently, domain-specific RAG systems have evolved beyond the generic retrieve-then-generate pipeline to incorporate specialized components—domain-adapted retrievers, curated corpora, and self-reflective mechanisms—that must be orchestrated to handle the unique constraints of each vertical. This subsection, building on the multilingual and cross-lingual challenges discussed above, turns to how RAG systems are adapted for these knowledge-dense and safety-critical environments, and in doing so, anticipates the

cross-domain generalization and human-centric interaction concerns that follow.

8.3.1 Healthcare: Precision, Clinical Reasoning, and Domain Adaptation

Healthcare is perhaps the most active frontier for specialized RAG research, largely due to the need to mitigate hallucinations and enable grounded reasoning in clinical decision-making. The integration of RAG into medical practice has demonstrated a clear and consistent benefit over both generic LLMs and traditional chain-of-thought prompting. Large-scale benchmarks like the Medical Information Retrieval-Augmented Generation Evaluation (**MIRAGE**) are foundational in this effort, serving as a systematic resource for evaluating RAG systems across five medical QA datasets with over 7,663 questions. Through extensive experimentation with over 1.8 trillion prompt tokens on 41 combinations of corpora, retrievers, and LLMs, MIRAGE demonstrated that RAG can improve the accuracy of various models by up to 18% over standard prompting, even elevating the performance of GPT-3.5 and Mixtral to GPT-4 levels [27]. This comprehensive scale also uncovers the “lost-in-the-middle” effects and log-linear scaling properties in the medical context, emphasizing the critical importance of corpus selection and retriever construction over model size.

Beyond benchmarking, specialized systems are trained to handle domain-specific reasoning challenges. The **Self-BioRAG** framework, for instance, achieves domain-specialization through being trained on 84k filtered biomedical instruction sets to generate explanations, retrieve domain-specific documents, and self-reflect on its responses. It uses customized reflective tokens to assess its own generations, leading to a significant 7.2% absolute improvement over the state-of-the-art open-source models under 7B parameters on three major medical QA benchmarks, proving the necessity of domain-specific components (retriever, corpus, instructions) for adhering to clinical instructions [65]. Extending this, systems that attempt to unify specialized medical models, such as the MedAGI paradigm, leverage expert selection algorithms to automatically choose among diverse domain-specific models by analyzing user questions, emphasizing that a single-purpose RAG may not be feasible, but a unified orchestration of increasingly available domain-specific components can achieve versatility with lower cost [382].

Fundamentally, the architecture of medical RAG systems must adapt to the complex multimodal and domain-specific knowledge inherent in biomedicine. **REALM** presents a framework that uses a RAG-driven approach for enhancing multimodal electronic health records, tackling the common inability of existing models to incorporate the specialized medical context from a knowledge graph (KG). This framework uses an LLM to encode clinical notes, matches entities with the PrimeKG KG, and integrates structured multimodal data with KG knowledge through an adaptive fusion network, substantially outperforming traditional methods on mortality and readmission tasks [355]. Similarly, a suite of LLM-based medical agents named MedReAct, MedEval, MedReFlex, and MedFinalParser has developed an agent-based system of observation, evaluation, and reflection to detect and correct errors in clinical notes. The RAG pipeline built upon a custom ClinicalCorp corpus—the ninth rank in the MEDIQA-CORR 2024 leaderboard—was central to this reasoning pipeline [383].

The evaluation of these systems presents unique challenges beyond simple accuracy. Metrics must be domain-adapted; the generic RAGAS framework for evaluating RAG pipelines assesses dimensions of context relevance, answer faithfulness, and answer relevance, but its reference-free nature means it operates without ground truth human annotations [26]. As a result, evaluations must often employ specialist human judgment. For instance, work on LLM-generated medical citations demonstrates that while GPT-4 is highly accurate in validating source relevance (agreeing 88% of the time with a panel of practitioners), 50% to 90% of LLM responses are not fully supported by the sources they provide [384]. For domain-specific fine-tuning, studies have shown that combining a fine-tuned embedding model with a fine-tuned LLM and iterative reasoning can significantly

outperform generic models, with an even bigger jump when fine-tuned embeddings and reasoning iterations are used [71].

Finally, a critical aspect of healthcare RAG is that it does not only apply to answering questions but also to supporting decision-making and ensuring safety. In a set of 61 prescription error scenarios across 23 complex clinical vignettes and 12 specialties, RAG-LLM, especially in a “co-pilot” mode with a human pharmacist, leads to boosted accuracy in detecting moderate to severe drug-related problems [385]. This also emphasizes that precision might be traded off for recall, informing practical approaches. Similarly, adding accurate, up-to-date information from RAG to LLM inputs for generating clinical trial protocols significantly reduces “hallucination” and improves the usefulness of writing quality [386]. For diseases like Osteoarthritis, specialized systems such as DocOA have been developed specifically to manage complex disease by integrating RAG and clinical data to generate more tailored suggestions than general LLMs [387].

8.3.2 Legal and Finance: Knowledge-Critical Reasoning and Structured Knowledge

In the legal domain, the case-based nature of the law and the importance of evidence require a specialized approach beyond generic semantic search. The **CBR-RAG** framework uses Case-Based Reasoning to structure retrieval for legal question answering. It enhances the original LLM query with contextually relevant cases by utilizing the CBR cycle’s retrieval stage and similarity knowledge containers. By using different representations (general vs. domain-specific embeddings) and methods of comparison (inter, intra, hybrid similarity), the work demonstrates that context provided by case reuse improves answer quality, highlighting that legal RAG must align query requirements with the representational semantics of the knowledge base [388]. Similarly, the need to synthesize multiple sources from different domain viewpoints is pivotal in knowledge-dense legal and medical contexts; the MVRAG framework applies intention-aware query rewriting from multiple domain perspectives to improve retrieval precision [358]. For pharmaceutical regulatory compliance, the **QA-RAG** model improves accuracy over generic RAG by searching and providing answers grounded on relevant guidelines, emphasizing the importance of a question-answer structure [389].

Finance presents a similar landscape, with a need to answer complex queries grounded in specific financial filings. In this context, RAG systems benefit from fine-tuning and reasoning-based customization. Experiments on the **FinanceBench** SEC financial filings dataset have shown that for RAG, combining fine-tuned embedding with fine-tuned LLMs achieves better accuracy than generic models, and the use of iterative reasoning on top of RAG enables the system to get closer to human expert performance [71]. This data-driven finding underscores the importance of domain-specific optimization. In addition, retrieval from structured sources such as electronic health records for answering epidemiological questions has been performed through a combination of text-to-SQL and RAG; a method that augments the text-to-SQL process with domain-specific retrieval shows performance improvement over simple prompting [390]. This demonstrates that in testbeds like finance and healthcare, where structured data is central, the integration of RAG with executable, structured SQL queries can be a powerful tool for grounded reasoning.

8.3.3 Balancing System Implementation, Human Oversight, and System Evaluation

The large-scale deployment of RAG in high-stakes fields is characterized by a constant tension between domain-specification and generalization, requiring strategies for evidence-based reasoning and trustworthy generation. Medical work underscores the importance of architecture choices, as highlighted by fine-grained benchmarking frameworks like **MIRAGE** which explore how different corpora and retrieval methods not only lead to different performance outcomes but also reveal

optimal configurations [27]. Systems like the **LLM-AMT** system, which integrate medical textbooks directly into the LLM framework via plug-and-play modules, such as a Hybrid Textbook Retriever, show that by tailoring the data, domain-specific content can become more effective than a broad, general-purpose web corpus like Wikipedia [391].

What becomes clear is that, irrespective of the domain, RAG cannot be treated as a single monolithic algorithmic approach; instead, the implementation must adapt to the verifiability, context constraints, and human-centric evaluation requirements of the field. First, for the medical domain, the “co-pilot” mode of interaction mentioned earlier demonstrates how a RAG system can be designed to work *with* a human expert, with the system providing an output of confidence or risk scores to provide the user a sense of control and trust. The enterprise **GutGPT** system, for example, has demonstrated the potential of deploying LLMs—carefully integrated inside an interface orchestrating clinician feedback—to operate in controlled, but real-world, decision environments [392]. This system is a representative example of domain and environmental awareness, leading to practical and clinically relevant deployment. Moreover, the issue of hallucination persists: current models still fail to provide fully supported references, as related work such as ‘SourceCheckup’ highlights that nearly half of responses are not fully supported by cited sources [393]. This necessitates careful evaluation frameworks and robust tools for introspection, leading to the use of integrated and holistic modalities for open-sourcing evaluation, such as InspectorRAGet, which can analyze aggregate and instance-level performance of RAG systems by combining human feedback and algorithmic metrics [394]. In sum, it is in this manner—by embedding RAG into human-in-the-loop, domain-aware, and evaluation-driven designs—that RAG is maturing across critical verticals, bridging the gap between individual components (retrievers, generators) and integrated, reliable AI systems.

8.4 Cross-Domain Generalization, Personalization, and Human-Centered Interaction

The evolution of Retrieval-Augmented Generation (RAG) from a purely technical solution for knowledge grounding to a versatile framework for interactive and adaptive AI necessitates a critical examination of its cross-domain generalization, personalization capabilities, and the design of human-centered interactions. Building directly on the domain-specific adaptations for healthcare, legal, and finance discussed in the previous subsection, this subsection explores how RAG systems are being extended beyond these structured, high-stakes verticals into more dynamic and personalized fields such as education and long-term conversational assistance. This shift emphasizes the necessity to handle “shifts” between multiple reasoning sources and to adapt to users with varying levels of expertise and literacy, while incorporating human-centric controls to ensure safety and efficacy in less formally constrained environments.

The ability of a RAG system to generalize across domains is not merely about transferring a model from one set of data to another; it is a complex process involving the dynamic orchestration of retrieval and generation to meet new, often unseen, challenges. Whereas the previous subsection focused on domain-specific architectures tailored for a single vertical, here the focus turns to the *flexibility* of a system to move between multiple domains. This generalization challenge becomes particularly acute in knowledge-intensive tasks that require multi-step reasoning that spans multiple domains. Research has shown that existing RAG systems often struggle with complex queries, such as multi-hop questions that necessitate synthesizing evidence from multiple, distinct sources [49]. The paradigm of knowledge source selection requires the model to “shift” from relying on a single static repository to dynamically choosing and integrating information from a variety of sources across domains. This process is exemplified by frameworks like **UniMS-RAG**, which decomposes personalized dialogue generation into knowledge source selection, retrieval, and generation tasks, allowing a language model to adapt its behavior to diverse task requirements [56]. The architecture

of a RAG system must therefore be robust enough to handle the ambiguity and complexity of real-world, cross-domain queries, which often require the seamless integration of internal parametric knowledge and external evidence, a capability that extends the domain-locked architectures of the previous subsection.

At the heart of this cross-domain generalization is the paradigm of personalization, particularly crucial in the context of long-term human-agent interactions. A static RAG system is insufficient for maintaining coherent and contextually relevant dialogues over multiple chat sessions, which demand memory and user-specific adaptation. Given that long-context LLMs and RAG techniques have been applied to address this, their efficacy in extremely long-term dialogues needs further investigation [395]. Approaches like **RAISE** (Reasoning and Acting from Scratchpad and Examples), which integrates a memory-enhanced architecture through a dual-component memory system, mirror human short-term and long-term memory to maintain context and continuity, thereby enhancing control and adaptability in complex multi-turn dialogues [396]. Similarly, memory-enhanced RAG systems such as **RAM** attempt to learn from user feedback to continuously update their ever-improving memory, handling complex queries through recursively reasoning-based retrieval and experience reflections, which is essential for a lifelong-learning RAG system [397]. This move toward user-specific adaptation fundamentally extends the role of RAG from a simple knowledge retriever to an active long-term personalized assistant.

Beyond basic memory management, interactions in dynamic environments require user-adaptive and user-centric design. This involves personalization of content as well as the adaptation to varying user literacy levels and linguistic styles. The challenges of information accessibility in multicultural and multilingual environments are critical: various user groups may interact in different languages or have different levels of literacy, requiring systems that can understand the language and match its responses to the user’s level of comprehension [398]. The design of such systems must be proactive while maintaining the domain-specific standards of verifiability and explainability established earlier. Meanwhile, evaluation of system performance must go beyond traditional single-dimensional question-answering to cover a broader spectrum of scenarios, dealing with user requests that may be Create, Read, Update, and Delete semantics, calling for a multi-faceted, comprehensive benchmark [399].

Perhaps the strongest advancement in cross-domain and personalized interaction is in the so-called “user-centric” system design that prioritizes user trust and safety. In medical education, for example, RAG systems need to integrate external domain-specific knowledge bases as seen in the prior section, but now with a focus on producing educational, accessible content rather than clinical decision support [400]. Beyond accuracy, a truly user-centric tool must be robust against errors and malicious attacks. For instance, poisoning attacks to RAG systems could lead to the generation of attacker-chosen answers, underscoring the need for secure defenses [257].

This necessity for security brings us to the fundamental issue of aligning the agent’s behavior with human expectations. As these more personalized agents become integrated into daily tasks, user interaction patterns change. Rather than being used as a tool to answer queries, users begin to view them as personal assistants or even collaboration partners, influencing the system’s ability to understand and respond to complex requests and dynamic tasks [401]. This includes the need for the system to be “adaptive” to communication patterns and to provide more suggestive and supportive responses rather than just following instructions.

From this broader perspective, this also means a shift RAG from a single interaction interface to a human-AI co-adaptive system. The “shifts” in reasoning are about switching between retrievers or knowledge sources, and about shifting the responsibility of action between the human and the system. A successful RAG must use different “modes” of interaction that match the user’s context and skill level; the system must adapt to variations in user certainty and sources’ reliability.

This leads to the final pressing requirement: the resilience and robustness of the system when deployed in dynamic, real-world settings. Seven failure points across case studies have shown that RAG systems are only truly validated during operation and can be weakened by failed retrieval, and that robustness evolves rather than being designed in at the start [402]. To ensure success in such environments, systems must align with user expectations and maintain core values, such as being helpful, honest, and harmless, even when faced with adversarial or noisy data [180]. Robustness must also include a continuous updating, since knowledge changes over time, and involve careful consideration of feedback loops, for the retrieval process may not always guarantee useful results [58]. This is the main challenge for human-centered design of RAG systems: to build a system that is effective and secure, offering the user a sense of control, given that even the most advanced techniques may not yet fully optimize the complex interaction between users, knowledge sources, and LLMs.

9 Evaluation, Benchmarks, and Interplay with Other LLM Techniques

9.1 Evaluation Metrics and Automated Assessment Frameworks

The evaluation of Retrieval-Augmented Generation (RAG) systems is a multifaceted challenge that extends beyond the traditional metrics used for standalone language models. A comprehensive evaluation must assess not only the quality of the final generated text but also the efficacy of the retrieval component and the faithfulness of the generation to the retrieved context. This subsection provides an overview of the evolving landscape of RAG evaluation, tracing the progression from conventional automatic metrics to more sophisticated, component-aware frameworks that systematically measure the key performance axes of correctness, faithfulness, and robustness.

Traditional automatic metrics, both reference-based and reference-free, form the initial layer of RAG system assessment. Reference-based metrics, such as BLEU and BERTScore, which are widely used in tasks like question generation and machine translation, compare generated outputs against human-written references. However, their reliability is often called into question, particularly when only a single reference is used, which fails to capture the diversity of valid outputs [403]. The limitations of this approach are not solely confined to RAG; research has shown that reference-based metrics can exhibit a statistical advantage in estimating system-level quality compared to human judgment in certain settings, due to their low variance, even if they are biased [404]. Furthermore, these metrics can be gamed or can rely on spurious correlations, potentially leading to misleading results about a system’s true quality [405]. Several works have attempted to improve reference-based metrics, for example, by using enhanced distance measures with contextualized embeddings [406] or by augmenting reference sets to improve reliability in complex tasks like open-domain dialogue [407]. However, the core limitation remains: human-written references may be unavailable, expensive to obtain, or may not align perfectly with the multiple valid ways to answer a given RAG query.

This limitation has motivated the development of reference-free evaluation metrics, which assess the quality of generated output without gold-standard references. Quality estimation metrics, which predict a score based solely on the source and system output, are akin to this approach. In machine translation, work reveals that providing additional source context can sometimes substitute for a reference, closing the performance gap between reference-free and reference-based metrics in sentence-level evaluation [408]. Within the RAG context, the fundamental goal is often to provide metrics that do not suffer from the cost and scale limitations of human annotation. However, reference-free metrics must be carefully scrutinized for their own biases, as they can be unintentionally influenced by spurious features such as output length or perplexity [405]. Attention has also been paid to the health of the model itself rather than just the output; high-quality generation must be justified

by faithful reasoning. For instance, in the realm of counterfactual and free-text explanations, new metrics like counterfactual faithfulness that account for actual shifts in the label distribution have been proposed to better reflect whether the reasoning truly reflects the model’s decision logic [409]. In the absence of a single gold metric, a robust evaluation must measure the model’s performance across a range of situations and structures.

Given the complexity and fragility of basic metrics, the field has shifted toward comprehensive, task-specific evaluation frameworks. Central to evaluating faithful systems is an assessment of robustness, defined as whether the model’s prediction remains stable under small, human-meaningful changes to the input [410]. This methodological approach aligns with the principles of reliability and generalizability, suggesting that evaluation should measure how well models perform given shifts in input distributions [411]. In the context of large language models and RAG, this translates to evaluating how sensitive a system is to knowledge conflicts and adversarial inputs.

In this vein, more automated, unified frameworks have been developed specifically for RAG. Two frameworks stand out: RAGAs (Retrieval Augmented Generation Assessment) and ARES (An Automated RAG System). RAGAs positions itself as a reference-free evaluation framework that provides a suite of metrics to assess RAG pipelines along key dimensions without the need for ground-truth human-annotated data [26]. This is crucial because evaluating a RAG system requires not only analyzing the retrieval layer to know the relevance and focus of the context passages it surfaces but also simultaneously assessing the LLM’s capability to utilize these passages coherently and faithfully. Rather than directly scoring based on a single reference answer, RAGAs decomposes generation quality into its component parts. This allows model developers to directly identify bottlenecks—whether systems for the retriever is adding irrelevant context, or generator is prone to hallucinating even when the context is correct (faithfulness). This artifact-free, or reference-free, approach intends to contribute to faster cycles needed for the rapid adoption of LLMs, as it removes the need for creating high-quality human annotations for every new domain or dataset [412]. To achieve scalability, the core components are tuned with synthetic data.

Similarly, ARES (An Automated RAG System) provides a fully automated framework to evaluate RAG systems along the key dimensions: context relevance (quality of context passages), answer faithfulness (how grounded the answer is in the context), and answer relevance (how concisely and relevantly the answer addresses the question) [413]. A key design feature that separates its design from purely synthetic setups is the use of prediction-powered inference (PPI) [414]. This allows the framework to include the ability of an evaluation system to generate correct, high-quality responses, while using a small set of human-annotated datapoints to calibrate and correct for any potential biases inherent in its own LLM-as-a-judge evaluation. This hybrid methodology is significant

; it elevates the accuracy of the system and enables the method to be used without large amounts of human annotation. ARES’s use of PPI directly confronts the pitfalls of over-reliance on purely automatic metrics, which, as prior research has shown, can exhibit spurious correlations or bias [405]. Because ARES is trained on synthetic data, it can cope with domain shifts and consistently evaluate RAG systems on new types of queries and documents without requiring new fine-tuning, a major bottleneck in other evaluators [414].

In many fields, evaluation of a RAG system can be understood as evaluating a multi-component system rather than a singular text-to-text generation. For instance, in the context of a RAG system for question generation, ensuring that the model not only produces fluid text but also extracts the correct information and generates appropriate questions requires specific metrics. For example, using reference-based metrics to assess this can be misleading given the diversity of plausible responses for a given passage in [403]. Frameworks like RAGAs and ARES move beyond the simple output-only view, introducing metrics that measure the intermediate retrieval and generation quality, using LLMs to act as gatekeepers. This dual-modality evaluation is crucial for the diagnostic power of the

evaluation and its ability to identify failing components.

The area of counterfactual robustness is a significant component of a comprehensive RAG evaluation. A system that is robust will not alter its final answer, or will otherwise treat information correctly, when an input receives non-essential variations. This connects directly to the task of explaining predictions of a robust model [410]. In a complex RAG context, these robustness tests have been used to probe model behavior when given erroneous yet relevant documents, measuring the model’s tendency to incorrectly follow the misleading evidence. The proper evaluation of counterfactual robustness has self-referential challenges. “Double Perturbation” frameworks [415] highlight that robustness evaluations themselves can be brittle—where a single subtle perturbation of the test dataset is enough to reveal unseen model weaknesses, and importantly, for a robustness measure to be valid, it must be left engineered to avoid hidden biases and instabilities. The existence of evaluation frameworks that integrate these robustness elements ensures that a RAG system is not simply evaluated for confidence in a narrow sense but for its capability to reason logically under varying inputs.

Eventually, the future of RAG evaluation is likely to be a synthesis of methods. Frameworks in development are moving toward evaluating components of the system with selective use of human-styled judgment, referencing systems to test hallucination, and applying principled conformal prediction to generate valid confidence sets around retrieval and generation, further enhancing the reliability of the overall evaluation [184]. To ensure a system that is both answer-relevant and faithful to external information in the face of unreliable, conflicting, or manipulated contexts, the field of RAG evaluation requires a move away from smeared, monolithic metrics toward a kind of “natural” structure—granular, multi-metric, hybrid, and robust frameworks. The tools from the papers highlighted here provide the basis for building just such an evidence-based evaluation system, which can help practitioners better understand how each component in the RAG system behaves under a structured suite of contexts, thereby contributing to optimizing overall system performance rather than to merely placing model confidence in a narrow evaluation settings.

9.2 Comprehensive Benchmarks and Multi-Task Evaluation

The development of robust evaluation benchmarks is essential for advancing Retrieval-Augmented Generation (RAG) systems. While earlier evaluations often focused on single-hop or simple extractive question answering, the increasing complexity of RAG applications demands comprehensive benchmarks that challenge models across diverse dimensions, including multi-hop reasoning, robustness to noise, long-form generation, and the full spectrum of knowledge manipulation tasks. Moreover, as the preceding discussion on evaluation metrics and frameworks highlights, the field has moved toward granular, multi-component assessments rather than monolithic accuracy scores. Benchmarks, in this light, serve a dual purpose: they not only establish performance rankings but also operationalize the very principles of faithfulness, robustness, and diagnostic power emphasized earlier, providing structured testbeds for scrutinizing each RAG pipeline element. This subsection provides a detailed overview of several influential benchmarks that have been designed to push the boundaries of RAG system evaluation, highlighting their unique tasks, scope, and the insights they offer into the capabilities and limitations of current models.

One of the most critical and challenging frontiers for RAG is multi-hop question answering, which necessitates the retrieval and integration of multiple, often disparate, pieces of evidence. A common observation across multiple studies is the inadequacy of standard RAG systems in this domain, a problem that directly undermines the faithfulness and answer relevance goals central to comprehensive evaluation. For instance, the **MultiHop-RAG** benchmark [49] was specifically developed to address this gap, providing a curated knowledge base of news articles, a large collection

of multi-hop queries, and their ground-truth answers with supporting evidence. Its construction enables fine-grained analysis, allowing for the isolated evaluation of both the retrieval and reasoning components—a diagnostic granularity that mirrors the component-level decomposition promoted by frameworks like RAGAs and ARES. Initial experiments with this benchmark revealed unsatisfying performance from existing retrieval and generation models, including state-of-the-art LLMs like GPT-4, PaLM, and Llama2-70B, particularly when queries require connecting information from multiple sources. This finding underscores a persistent bottleneck: even when retrieval surfaces relevant documents, the generator often fails to faithfully combine them, reinforcing the need for a systematic isolation of failures beyond single-score metrics. Complementing such efforts, foundational work on multi-hop reasoning highlights a related and pervasive risk: data contamination in existing benchmarks. The **MRKE** (Multi-hop Reasoning Evaluation of LLMs by Knowledge Edition) benchmark [416] proposes a corrective solution by creating a new QA benchmark based on edited knowledge from the HotpotQA dataset. Their work demonstrates that LLMs, including GPT-4, show a significant performance gap on these newly edited, unseen facts and achieve a low percentage of complete reasoning chains—revealing both the risk of contamination in older datasets and the persistent difficulty models have in generating faithful intermediate reasoning steps. This benchmark aligns with the broader call for robust, faithful evaluation frameworks by providing a more objective measure that is less clouded by memorize data contamination. Indeed, by setting new knowledge as the premise, MRKE also validates the necessity of RAG for integrating up-to-date, verifiable information, pointing directly to the challenges of continuous knowledge bridging discussed in the following section.

Other benchmarks expand the scope of evaluation beyond the confines of question-answering alone. The **CRUD-RAG** benchmark [399] offers an extensive and comprehensive framework based on knowledge management operations. It categorizes RAG applications into four distinct tasks: Create, Read, Update, and Delete. “Create” refers to generating new, original content; “Read” involves responding to a wide range of questions in knowledge-intensive scenarios; “Update” focuses on revising or correcting pre-existing texts; and “Delete” pertains to the task of summarizing extensive texts into more concise forms. This four-task taxonomy covers a broader spectrum of applications often overlooked by benchmarks that only focus on question answering, and it provides an environment to interrogate how well a system can reconcile its embedded parametric memory with newly provided external contextual evidence. Furthermore, this benchmark distinctly evaluates all components of the RAG system—including the retriever, the context window, the knowledge base, and the LLM itself—rather than treating the system as a monolithic block, echoing the earlier call for multi-metric, component-aware evaluation frameworks. By evaluating each component in isolation for each of the four task types, CRUD-RAG provides actionable diagnosis insights for the optimization of RAG pipelines across a variety of practical scenarios, especially those that require significant weight-based knowledge editing.

A key differentiator in the design of modern benchmarks is the move toward more holistic, scalable, and reproducible assessment methodologies. Several methodologies have been proposed in this direction. For example, **AGIBench** [417] presents a multi-ability benchmarking methodology that focuses on three ability branches, uses a four-tuple approach to label the attributes of each question, and classifies all the questions into five degrees of difficulty according to the average accuracy of human experts. A notable strength of AGIBench is its use of a human-referenced scoring mechanism with the potential for automatic scaling, which helps to guarantee reproducibility and reduce unpredictability in benchmarking—addressing directly the reliability concerns raised in the previous subsection regarding inspection of automated evaluators.

In parallel with these text-centric advances, there is a growing emphasis on evaluating multimodal and low-level understanding within large models, which has direct downstream implications for

multimodal RAG systems. **MMBench** [418] provides a systematic way to continuously evaluate diverse abilities of vision-language models over time, and introduces a novel CircularEval strategy to convert free-form model predictions into pre-defined choices, leading to a more stable and reproducible evaluation of model performance—an essential trait for benchmarking robustness to input variations. The related paper, **A Benchmark for Multi-modal Foundation Models on Low-level Vision** [419], extends the evaluation of multimodal LLMs to low-level visual perception and understanding, introducing tasks that require a nuanced understanding of image details. This is especially important for RAG systems that may need to reason over visual information behind ground with fidelity. Similarly, **MME** [420] presents a total of 14 subtasks to measure both perception and cognition abilities, rejecting to avoid answer leakage by designing instruction-answer pairs that are missing from the model’s training corpus. This careful construction is especially relevant in an era where data contamination is prevalent, directly connecting to the long-running concerns quantified in multi-hop reasoning benchmarks. **MLLM-as-a-Judge** [421] complements these by shifting the focus to the evaluation of the models themselves as judges—a crucial component in the LLM-as-a-judge evaluation pipeline that is widely used for many modern RAG systems. The studies show that while MLLMs have some human-like discernment, they still exhibit significant biases (e.g., towards length and position) and inconsistencies. This sheds a harsh light on a critical facet: the goals in assessing RAG models are often intrinsically bound to the health of the imperfect evaluator, a key uncertainty that was already highlighted with the previous discussion on frameworks like ARES and the conformity prediction.

Detailed comparative analysis of these diverse benchmarks exposes several persistent and systemic challenges. For multi-hop reasoning, as highlighted by **MultiHop-RAG** and **MRKE**, state-of-the-art models still fail simple and straightforward integration, demonstrating a fundamental bottleneck that cannot be solved only by scaling model size or retrieval index capacity; rather, methodologies must be aligned toward enhancing the model’s ability to faithfully fuse external evidence and distinguish it from its parametric priors, a task that requires rethinking how distinct modules in the RAG pipeline are optimized in concert. Meanwhile, the evaluation results from **CRUD-RAG** [399] reveals limitations in handling tasks such as “Update” and “Delete,” suggesting that the parametric nature of LLMs is deeply ingrained and cannot be easily “rewritten” by external context alone, regardless of retrieval quality. The expectation that RAG alone can solve such adaptation tasks must be tempered, underscoring the interactive and layered roles of fine-tuning and external knowledge that were laid out in the preceding analysis. In multimodal settings, the performance gap between low-level visual perception and higher-level cognitive reasoning—so clearly illuminated in **MME** [420]—signals a critical defect for a future multimodal RAG systems that depend on visual retrievers to handle tasks that require fine-grained modality fidelity. Finally, benchmarks like **MLLM-as-a-Judge** [421] further demonstrate that even the evaluator’s themselves are not immune to biases and inconsistencies, questioning the reliability of fully automated evaluation pipelines and reinforcing the need for hybrid human-in-the-loop calibrations that the ARES framework introduced. This set of challenges aligns directly with the previous argument that a robust evaluation cannot rely on a single monolithic metric; instead, a “natural” structure of granular, multi-metric benchmarks that are constantly adjusted to various perturbation is critical. In conclusion, these benchmarks, when used in conjunction with the diagnostic frameworks described earlier, move the field forward not only by establishing models with better predictive metrics but also by providing new, multidimensional perspectives on the deficits and required future work to create more reliable, robust, verifiable, and truly capable RAG systems that can be trusted in knowledge-intensive applications.

9.3 Interplay with Model Editing, Long Context Windows, and Continual Learning

The landscape of knowledge incorporation in large language models (LLMs) is not monolithic; rather, it is defined by a dynamic interplay between several distinct, yet complementary, mechanisms. The three predominant methods—fine-tuning, retrieval-augmented generation (RAG), and prompt modification—operate at fundamentally different levels of the model stack and offer unique trade-offs in terms of adaptability, cost, and reliability. Understanding their synergies and conflicts is crucial for designing robust knowledge-intensive applications, especially in light of the evaluation frameworks and empirical insights discussed in the preceding benchmark analysis and the subsequent focus on verifiability.

Fine-Tuning: Parametric Weight Modification

Fine-tuning modifies the LLM’s parametric memory by updating its internal weights, effectively baking new or reinforced knowledge directly into the model’s parameters. This is a direct and potent method for injecting knowledge, and it remains a cornerstone for specialization. In exploring the balance between fine-tuning and RAG for less frequently encountered knowledge, one study found that fine-tuning “significantly boosts the performance across entities of varying popularity,” particularly for the most and least popular groups [25]. This deep integration means the information becomes a permanent part of the model’s cognitive foundation, influencing all subsequent generations without needing external input. However, this permanence is also its primary liability. As new facts emerge, the model becomes outdated, and the cost to retrain or fine-tune for continuous updates is high. This issue is aggravated by the susceptibility of fine-tuning to catastrophic forgetting, where learning new sequential tasks can overwrite competencies in previously learned domains [422]. Furthermore, the “knowledge” gained through fine-tuning remains largely opaque to users, making it difficult to distinguish between external knowledge and an internal, potentially incorrect, fact. One study directly comparing the two methods on knowledge tasks found that while unsupervised fine-tuning offers some improvement, RAG consistently outperforms it for both previously encountered and entirely new knowledge [30]. This suggests that for many tasks, the ability to explicitly reference external knowledge provides a more robust method than relying on weight updates alone.

Retrieval-Augmented Generation: Non-Parametric Runtime Knowledge

Retrieval-Augmented Generation (RAG) functions as a non-parametric mechanism, acquiring knowledge at inference time by retrieving relevant documents from an external knowledge database and injecting them into the model’s context. This approach offers a distinct advantage of remarkable adaptability: the model can be updated with the latest information simply by updating the external database, greatly mitigating the “knowledge cutoff” issue and diminishing instances of hallucinations in domain-specific Q&A [66]. This makes it a promising solution to mitigate many problems of fully parametric language models [28]. RAG thrives in settings where knowledge is dynamic or vast, and accurate responses demand grounding and context beyond what the parameters alone can provide.

However, RAG introduces a set of unique challenges. It places a premium on retrieval quality; the model’s performance is fundamentally dependent on the retriever’s ability to locate relevant information. The inclusion of noisy or irrelevant text can impair generation output. RAG is also susceptible to new threats such as retrieval poisoning, where a few poisoned texts can influence large models to generate attacker-chosen answers [257]. There is also a constant tension between the model’s parametric prior (its internal memory) and the contextual evidence retrieved at inference time. When these sources of information conflict, the model must decide which to trust, and unresolved conflicts can lead to unreliable behavior [37]. For a static single-turn interaction, even

well-designed RAG can be injected with conflicting evidence or present irrelevant documents, which can sometimes degrade performance if post-retrieval processing and effective context injection are not done well [58].

Prompt Modification: Flexible Contextual Steering

The third mechanism, prompt modification (including in-context learning and instruction tuning), operates at the level of the input interface. The input components can be specifically crafted through design and textual synthesis to guide the model’s behavior without altering its core weights. As observed in the context of some work, fine-tuning after prompt injection is not as effective as the RAG approach, and a soft prompt can significantly amplify the effects of both without fully suppressing them [423]. Superposition-wetting, as in “soft prompts,” allows user knowledge to be processed in parallel and gives relevant paths a way to close context-heavy generation moments [424]. While versatile, and often less computationally expensive than fine-tuning, effective prompt engineering requires deep expertise and may not, by itself, overcome the limitations of the model’s parametric knowledge.

Synergies, Conflicts, and Hybrid Approaches

The interaction between these three mechanisms is not one of isolation; rather, they are often combined in practice to create powerful hybrid systems. An effective RAG system often relies on fine-tuning to mitigate the “tug-of-war” between its internal prior and retrieved text; when a model is fine-tuned on a specific domain, the internal parametric prior becomes stronger, which can either complement or conflict with retrieved evidence. A study indicates that unsupervised fine-tuning may cause the model to rely on common patterns but struggle to learn new facts—knowledge diversity is key [30]. Thus, RAG and fine-tuning are not opposing; they are complementary in practice. When combined with parameter-efficient fine-tuning, for example, LoRA can yield over 12x runtime improvements while maintaining model quality [425].

Benchmarks like CRUD-RAG and MultiHop-RAG are designed to evaluate these combined systems across a comprehensive range of tasks and multi-hop reasoning scenarios, providing structured frameworks for assessing the system’s ability to integrate diverse modules across the entire RAG stack [399, 49]. A deeper synergy is observed when models are jointly tuned with fine-tuning to refine the retriever and generator components, as seen in a joint retrieval-training approach (RAG-OD) that links the retriever and generator for a unified medical QA capability [426]. This synergy framework also aims to make the retriever “understand” what the LLM needs, bridging the “alignment gap” between the two components and addressing the often tricky selection and ranking problems that plague efficient end-to-end RAG systems [39].

Memory for Continual and Transparent Access

The interplay among these mechanisms also has direct implications for memory management in LLMs and for the principles of verifiability and transparency discussed in the next subsection. In a pure fine-tuning setting, knowledge is deeply encoded into model weights, making it opaque and difficult to audit. In contrast, RAG offers a potential route toward more transparent system design: by making the external knowledge store explicit and modifiable, the system can be updated without modifying the underlying model—an attractive property for continual learning and for facilitating user verification. In RAG, new external documents can be added to the index without retraining, which addresses the issue of forgetting specific facts that fine-tuning may inadvertently forget. At the same time, prompt-based techniques can provide more granular manipulation of user-specified reasoning steps, but they are inherently ephemeral and must be constantly re-applied rather than persistently integrated.

The choice between these mechanisms often involves a hierarchy rather than strict delineation: use alignment and continual learning to refine the model’s general intent and set its broad behavioral canvas; use RAG as a tool to access up-to-date, domain-specific facts on demand; and use prompt

modification as a fine-grained control to handle immediate task constraints such as style or reasoning rules. This layered view not only aligns with the design of modern RAG pipelines but also directly sets the stage for the next key challenge: ensuring that the outputs from any of these mechanisms can be attributed—that the information provided by RAG or prompted from the parametric memory can be traced and verified. To that end, the need for robust attribution and verifiability metrics, as the subsequent subsection discusses, emerges naturally: if the boundaries between parametric and external knowledge remain ambiguous, any reliability of the final system is impossible.

9.4 Attribution Evaluation, Automated Citation, and Human-Aligned Assessment

The evolution of Retrieval-Augmented Generation (RAG) systems has brought the issue of verifiability and attribution to the forefront of research. As these systems are increasingly deployed to assist in knowledge-intensive tasks, the ability to audit their outputs—to trace claims back to their sources and assess the trustworthiness of the evidence—becomes a critical evaluation dimension. This imperative follows directly from the mechanisms discussed in the previous subsection: while fine-tuning embeds knowledge opaquely into model weights and prompt modification offers only ephemeral control, RAG’s explicit external knowledge store presents a unique opportunity—and obligation—to make the reasoning of these systems transparent. This subsection delves into the landscape of attribution evaluation, automated citation generation, and the design of assessment regimes that prioritize human alignment and verifiability. We move beyond simple accuracy metrics to examine how RAG systems can be built and evaluated to support transparency, scrutiny, and trust.

A foundational theme is the shift from merely retrieving evidence to actively using it to support claims in a verifiable manner. This marks a move from the traditional fact-verification pipeline, which focuses solely on predicting a veracity label, to systems that must also produce a justification that can be independently checked. The process of verification is not merely about finding evidence but about ensuring the evidence provided is both relevant and of reliable quality. The challenge of assessing the quality of evidence and its connection to a final generation is central to this effort. As demonstrated by work on Wikipedia verifiability [427], the challenge of finding and maintaining high-quality references is monumental. Their work introduced a system called “Side” that learns from the editing behavior of a massive community to recommend better sources than the ones originally cited in articles. In a human evaluation, for the 10% most likely unverifiable claims, users preferred the system’s suggested alternative 70% of the time. This underscores that a key component of attribution is the ability to distinguish good citations from poor or outdated ones.

The limitations of fully manual fact-checking have motivated the development of automated tools that can assist human fact-checkers [428, 429]. These systems are not designed to replace human judgment but to operate within the human and technological infrastructures of a fact-checking organization. A core part of their utility is the ability to generate check-worthiness scores, which identify which claims in a report require verification based on a complex, domain-specific set of criteria [430]. Likewise, tools must be able to retrieve prior art—previously verified claims—and match them against new claims. This task is not simply one of similarity but also involves stance, where the model must determine if a new statement agrees or disagrees with the previously verified one [431]. This combined analysis, where evidence and claims are assessed in concert, is a foundational challenge.

Recent research has moved towards a more integrated approach, where the generation model is itself fine-tuned to produce outputs that are more easily verifiable. One notable method is **Quote-Tuning**, which goes beyond training models to retrieve evidence; it aligns them to quote verbatim statements from trustworthy sources in pre-training data [432]. By maximizing the amount

of language that is a direct quote, the system eases the verification process because the user can easily check the highlighted text against the original document. Experimental results show that models using this technique increased the percentage of *verbatim quoted* text by 55% to 130% relative to untuned models, which in turn improves the truthfulness of the final output. This is among the most direct methods of “attribution by design,” where verifiability is built directly into the objective of the model.

Beyond high-level strategy, a quantitative and nuanced evaluation of a RAG system requires a multi-faceted framework. We need approaches that go beyond just lexical or semantic similarity and capture the nuances of an attribution. A key challenge is that systems must be robust not only against in-distribution evidence but also against carefully crafted, misleading inputs. Adversarial misinformation is often inseparable from the problem, with attacks on fact-verification pipelines representing a significant vulnerability [433]. Robust attribution evaluation must therefore introduce such synthetic “adversarial evidence” cases—where an attacker tries to modify sources to influence the rank order—to see if the final output trusts a misleading citation or fails to flag errors [434]. The reliability of a system under misleading sources is a critical test of its robustness.

Human-aligned assessment also looks at how the system presents the output to the user. The goal is to design a regime of “human-aligned” evaluation that measures whether the physical layout of the final output supports rapid and correct verification. For example, a model might be better if it presents a local explanation next to the cited evidence, making it easier for a user to localize and verify the specific claim. The ability to identify *why* the system is generating a verdict and to which part of the citation it is responding is directly linked to user trust [435].

Another important dimension concerns the broader socio-technical implications. The generated output could need to be filtered through a *scrutability* lens that goes beyond the primary entity being fact-checked. For instance, in datasets like scientific reports, an output can connect to a broader knowledge graph or an entire body of work that is not included in the prompt selection (e.g., the *vesting* the model uses). A fully aligned assessment tool should potentially support carrying this contextual data forward, creating a chain of provenance from the original data all the way to the final generation [436].

Ultimately, devising a human-aligned assessment also demands a careful look at the design of the metrics themselves. Much like older works on reproducibility in general, such as *Reproducibility Issues for BERT-based Evaluation Metrics*, we must be careful that attribution metrics do not create a false sense of confidence [437]. The benchmark itself should be valid for the search engine. For instance, “human-aligned” does not only mean “accurate” in the narrow sense of factuality, but also “adequate” and “complete” in terms of stance, perspective, and sensitivity to the user’s original intent. Systems that produce citations but are not faithful to the reasoning of the model (as discussed in *Can LLMs Produce Faithful Explanations For Fact-checking?*) are insufficient because a user may read a correct citation but the reason the LLM gave is not for the stated conclusion [190]. Human evaluation is vital here to test that the explanation accurately reflects the model’s dependence on the evidence.

The emergence of “Dual-Use” evaluation is another key aspect of this research area. Systems that fail to use a reliable source may present a *confirmation bias* to the user. This entails a “malicious” answer that is confident in its output, but which has used the wrong source to “confirm” a false assumption. In many real-world decision support settings, this misleading evidence is more dangerous than a failure to find evidence; it is a form of “fake” relation. This requires us to design evaluation that takes into account the user’s ability to identify amendment—when the model has used a because it was easy to find but low quality, versus a *pro-choice* counterfeit with high-quality.

In conclusion, the future of attribution evaluation is heading towards a hybrid approach where automated systems generate ratings, but the precise meaning of those ratings is still designed in a

human feedback loop. This requires a system to not only parse language and retrieve, but also to embed a *defensive linguistic* model that understands the harm of a “partial” citation. Through the integration of models that can reason about *evidence in company* with the source, and by evaluating under a set of carefully designed adversarial perturbations, we can produce a system that is valuable not just in its generations but also in its ability to be scrutinized, validated, and ultimately believed.

We should view *automated verification* not as a checker of the “final fact,” but as a bridge between the human and the evidence that supports it, with strong adherence to trust and verifiable epistemology. The ultimate metric is not always performance on a publicly available benchmark, but the performance at the point of decision of a user who must trust the cited verdict.

10 Open Challenges, Ethical Society, and Future Directions

10.1 Persistent Grammar of Hallucination and Knowledge Conflict

The persistent challenge of hallucination in Retrieval-Augmented Generation (RAG) systems represents perhaps the most fundamental obstacle to their reliable deployment. While RAG was conceived to mitigate the parametric limitations of LLMs—such as knowledge cutoff and fabricated content—the integration of external evidence introduces a new grammar of conflict that, if ungoverned, can produce outputs that are neither faithful to the retrieved context nor true to the model’s parametric knowledge. Building on the architectural and operational foundations established earlier, this section systematically revisits this central problem and details the reinforcing levers—from parameter conditioning, cumulative memory management, and decoding strategy calibration—that must be jointly orchestrated to ensure that the model selectively leverages external evidence without collapsing into maladaptive rejections or over-reliance on faulty priors.

The Dual Nature of Knowledge Conflicts

When a RAG system retrieves external evidence, it perpetually navigates a tug-of-war between its internal parametric memory and the provided non-parametric knowledge. The research reveals a kind of Dunning-Kruger effect: as LLMs become more capable, they may become more confident in their faulty intrinsic memories, even when presented with correctly relevant external evidence. This is not merely a numerical preference but a persistent behavior: stronger models frequently over-rely on their own (at times mistaken, at times merely outdated) parametric prior rather than updating their view based on retrieved facts [253]. In contrast, other findings indicate that they are quite receptive to external proof when it appears coherent and coherently structured, which implies that the way we craft and frame evidence is as important as the evidence itself [63]. These seemingly contradictory results highlight that the central struggle is not usually about the validity of a document alone, but about the conditions *under which* that document will challenge an established parametric memory. The study of *Adaptive Chameleon or Stubborn Sloth* [63] elaborates on this dynamic, affirming that LLMs can be highly receptive to coherent external evidence, while also possessing a robust bias when that evidence partially aligns with their parametric memory. The model does not “simply” choose between internal and external knowledge; it balances its prior with the specificity and consistency of the provided signal. This demonstrates that the knowledge conflict is not something that can be “solved” with a single preprocessing step that can be “removed” in a reinforcement loop—it is a persistent feature of the inference process.

The Challenge of Superfluous, Conflicting, and Majority Evidence

The challenge only mounts when the retrieval module returns a mixture of truthful, irrelevant, and positively misleading documents. Evidence indicates that the language model employs a “majority rule” in such contexts: it disproportionately focuses on the evidence pathway that appears most frequently, regardless of its factual accuracy [253]. Such a bias proves dangerous in realistic

RAG settings where the document pool contains a variety of viewpoints (thus plaguing many AI applications with confirmation bias). The same mechanism explains the confirmation bias: presented with a claim that is close to (but not identical to) its parametric memory, the LLM will tend to accept the plausible pieces as evidence, ignoring conflicting signals [63]. This curious phenomenon stipulated in [438] underlines that factual recall is an additive mechanism—multiple independent contributions accumulate in the logit space—and the final answer is a function of the interplay between the internal contributions and the external consistency signals that are shifted by frequency. This means the retrieved signal acts as a “majority” factor, and the final answer cannot be fully controlled by solely filtering the retrieved *documents*; we also need to control their *arrangement*.

Exploring the Levers of Control: Parameter, Memory, and Decoding Calibration

The fight against the hallucination pattern under retrieved evidence’s influence forces us to think of three necessary but mutually reinforcing levers. The first is the prompt actuator: the architecture of the LLM as a causal language model is not agnostic to ordering. In the context of RAG, the order of the retrieved documents and their arrangement in the prompt can be seen as a (fine) determined of control. In-context demonstration ordering is known to be influential [140], and this sensitivity is especially pronounced in causal language models. The feature of such sensitivity naturally maps to the retrieval setting: if we trim or reorder the evidence, the perceptual impact on the reasoning outcome is noticeable, and the update pathways that guard against stale output may need to specifically accommodate this bias in order to align.

The second lever is the *type of knowledge and memory state*. A critical question is whether we need to *re-inject* the parametric knowledge to reweight a piece of conflicting evidence, or *inject* it altogether. Recent work clarifies this issue by demonstrating that compressed LLMs typically suffer from an impaired ability to recall facts, whereas their in-context processing capabilities remain largely intact [439]. This implies that, in RAG, the internal parametric recall may be selectively unreliable—especially after compression—while the contextual processing of external information is less affected. Therefore, rather than re-training the entire network, “re-routing” through the inserted context—while treating the model’s inherent memory as a *candidate* rather than an verifiable source—might be a viable remedy. This aligns with the suggestion that “re-routing” with input-side augmentation, such as dynamic prompting, can recover performance more efficiently than re-learning the weights [440]. In this light, the question at hand becomes not “how do we avoid a catastrophic clash between internal memory and external evidence” but rather “how do we *position* a bias event where the *memory path* does not automatically debase evidence.”

Finally, there is the introspective, self-critical lever: we need mechanisms that allow the model to re-think its use of evidence if that evidence leads to a path of contradiction. Techniques such as Conflict-Disentangle Contrastive Decoding [253] offer a constructive calibration: they recenter the model’s confidence to favor more reliable pathways in the gradient of the “majority,” but requires the model to *know* which evidence it has relied on. The true difficulty, however, for hallucination that arises from the model’s flawed reasoning, as examined in [189], lies in the case where the LLM resolves a task but produces reasoning that does not support the correct answer. This occurs often in contextual reasoning and question-answering tasks; even when accuracy is high, the model may “skip” evidence or produce a *self-contradictory* explanation that is not just about retrieval conflict. To counter this, an introspective layer that monitors both internal uncertainty and external evidence alignment becomes essential.

The Path Forward

This “persistent grammar” of the problem involves tackling, not only conflicting evidence itself, but the *instability of model confidence* when the evidence is syntactically coherent. The grammar is *syntactic*: the model’s belief formation follows a weighted combination of frequent and memorized evidence pathways. But it is also *behavioral*: the model is stubborn in its prior knowledge, adaptive

in structured external proof, and fragile under majority and minority signals. Thus, the most robust solutions cannot let internal parametric memory *dominate* without checking against the external evidence, and cannot allow the retriever to force a context without validation against that parametric prior. At its core, the field is shifting toward engineering solutions that separate the *state* of uncertainty: when a model faces conflicting evidence, its acceptance should be the result of a conscious interaction between self-consistency and external validity, including a “backtracking” pathway for self-contradiction [189]. Only then can we effectively mitigate—rather than sidestep—the persistent reality of decision-making under uncertainty, where biased or contradictory evidence is the norm rather than the exception.

We conclude this subsection by suggesting the core next-generation work is to construct frameworks that separate *representing* knowledge from *believing* knowledge: the LLM treats its parametric memory as a candidate, and the retrieved context must be a calibrating trial, not law. The reinforcing framework—composed of context-aware prompting, memory-conditioned retrieval, and contrastive decoding—is the sharpest weapon yet to mitigate the influence of dominant-yet-faulty evidence. In the next subsection, we expand this internal-control perspective to the broader security, privacy, and integrity challenges that arise when RAG systems are deployed in real-world environments, further linking the robustness of generation to the durability of the entire system.

10.2 Security, Privacy, and Ethical Considerations

The integration of Retrieval-Augmented Generation (RAG) into real-world systems introduces a complex and expanded threat surface that goes well beyond the traditional vulnerabilities of standalone large language models (LLMs). While RAG enhances factual accuracy and knowledge currency, it concurrently inherits the security weaknesses of its constituent parts—the parametric model, the retrieval index, and the external knowledge sources—while creating novel vulnerabilities unique to their interaction. This security dimension is not merely an ancillary concern; it is a foundational constraint that shapes whether RAG can be deployed responsibly. Just as the previous subsection emphasized the need to separate *representing* knowledge from *believing* knowledge to counter biased and contradictory evidence, here we confront a complementary challenge: how to ensure that the system *trusts* the right knowledge in adversarial environments, and how that trust interacts with the performance and scalability demands discussed in the next subsection. The stability of the generative component established through internal controls is of little value if the retrieved context itself is compromised, and any scalable solution must account for both these axes of reliability simultaneously.

A critical security concern highlighted in recent research is the poisoning of knowledge bases themselves. The efficacy of RAG is fundamentally reliant on the integrity of its external data store, making it a prime target for malicious actors. Attackers can manipulate data sources with the intent of subtly influencing all relevant retrieval rankings or injecting biased, harmful, or outright false contexts into the generation pipeline. This is a particularly dangerous attack vector because the LLM is then used as a vehicle to legitimize the attacker’s malicious content, and its outputs become extremely difficult to audit [441]. The risk of such poisoning is compounded by the challenges in detecting these attacks, as analyses of poisoning attacks on large language models have exposed the difficulty of providing safe responses when the model has been unknowingly trained on adversarial examples, highlighting the need for robust system-level defenses to ensure the integrity of the model’s final output.

This persistent threat of malicious or simply untrusted context mirrors the internal conflicts described earlier, but now moves the locus of conflict from the model’s parametric memory to the external embedding space itself. The system needs a governance and access-control layer to ensure

that what it retrieves is not the poisoned majority, but the verifiably correct signal. Attacks can also occur not just at the data level but at the retrieval mechanism level. When the attacker controls the retrieval module, they can force the user to retrieve a specific document regardless of its relevance or the user’s query. Such retrieval poisoning exploits the *structural* reliance of the system on dense embeddings and similarity search, making this vulnerability a persistent design flaw that must be anticipated from the ground up.

While poisoning attacks compromise the *integrity* of the knowledge base, a different set of vulnerabilities surfaces when examining the *confidentiality* of the data that flows through the pipeline. In the context of the RAG system, several severe risks emerging from the very nature of its role as a retrieval and packaging mechanism. Retrieving and packaging information from a data store to answer user queries introduces a dimension of external data leakage, a term we now define to include a wider vector of risk. A primary concern is the system’s failure to correctly manage privileges, which is not merely a software bug but part of the wider challenge of data access control. Indeed, these risks often stem from control and management gaps, underlying a rigorous security architecture for data access control. The failures of this component can allow an attacker to violate encrypted data via a denial-of-service. When these security boundaries fail, the leakage of information can be exploited, quite possibly by the models themselves, which can be attacked through targeted extraction or other methods to *purposely* leak information.

In an organizational setting, a RAG system is expected to enforce compliance and data policies. It must incorporate guardrails that prevent or minimize the leakage of sensitive information that might be embedded in the model and vector database, especially in high-stakes domains. For instance, within a healthcare RAG system, it is critical to protect the confidentiality of medical data and also ensure its security during retrieval. The guarantee that a RAG system will securely “retrieve” relevant data is often rendered void by failures in these areas.

Beyond data access and the threat of leakage, a different set of challenges arises from the *fairness* of the RAG system itself. Proponents of RAG hold it to be a tool to reduce bias, but in practice it can still be victim to biases through its data and retrieval mechanisms. The promise of RAG also hinges on its ability to access the correct information, though lack of fairness can lead to biased outcomes.

The scalability of RAG in modern systems further complicates its security concerns, as the ability to retrieve from a widely distributed source introduces new power. Secure, decentralized architectures built on blockchain have been proposed to securely log data exchanges and provide a foundation for managing privacy in high-regulation data sharing. However, even in these diverse deployment types, the integrity of the knowledge store must be protected. The inability to fully trust an external data source or a decentralized platform to be free from malicious content is a significant Nothing, which holds for RAG as well. It is also essential to design RAG with the *understanding* and the *realistic* assertion that the system must not rely on the unrealistic assumption that the retrieved data is continuously clean.

The deployment of such RAG systems, therefore, requires robust safeguards, meticulous, organizational approval, and the design of secure retrieval mechanisms. In adversarial RL chapter and secure methods are paramount. In considering the ethical and responsible principles, we must consider that a *data* breach is an authorized party that could extract confidential and private user information. This reinforces the need to validate that the participants of a RAG system are not acting adversarially and to equip this knowledge with robust, immutable, and effectively verifiable access, so that the whole history of the knowledge is not malleable.

It is worth highlighting that *other* perspective of internal computational operation, where the data exfiltration happens from a malicious or a compromised administrator for the purpose of model exploitation. In this regard, Data Leak Prevention (DLP) techniques, often leveraging machine

learning, can predict and mitigate such insider threats by detecting adversarial patterns, ensuring that the system does not leak state and storage in case of malicious exfiltration. To actually enforce security, we need specialized methods for privacy-preserving computations, which rely on homomorphic or other cryptographic primitives, to protect the confidentiality of user data even when the system is hosted on a cloud platform.

These internal and external control decisions now return to the main idea of the RAG system: defining what is “trustworthy” for it to use as context. Given the persistent presence of poisoning and identity risks, we must incorporate security validation both as a retrieval-time and generation-time decorative step. In sum, RAG systems view their retrieved context as a strong, reliable source of information, required to establish a correctness check to counter adversarial manipulation and establish a trustworthy, secure RAG. We cannot *only* rely on the fact that retrieval is “more grounded”. The system needs multiple stages of verification—not just the internal parametric procedure—but also a precise access control layer on its initial data, to combine trust with utility.

Without control of the context, future and outlook are impossible. There are important steps to achieve a robust RAG: need to anonymize the data, and a RAG system that has built-in safety to robustly respond when relying on a private and unexplored model. This needs a redesign, not just another patch of robust ML, but perhaps a systems approach in which upstream system security and token-privacy with the data owner side and their intent of retrieval concern is taken into consideration.

These recommendations must be built through a multi-party approach combining both policy (human) and technical (machine) mechanisms. This entails not simply using strict firewalls or blocklisting but instead implementing privacy-preserving data sharing agreements for both compute and value, at the external knowledge or employer, and in turn making a design that produces an output which is both transparent, and not concentrating on a single source, capable of embedding high-level privacy considerations and that it is allowed to ask for *more* and be resilient to poisoning and contradictory data.

Crucially, this security architecture cannot be treated as an isolated perimeter. It must integrate with the cost-aware inference mechanisms described in the following section. The security-relevant indirect analyses—such as encrypting queries, isolating sensitive context vectors, or validating document provenance in a hierarchical retrieval cache—have a measurable impact on latency and memory. A secure system is thus not necessarily a slower one, but it is a system that must plan for redundancy and attestation as part of its engineering budget. Fragile architectures that optimize for nothing but raw retrieval speed face a foundational dilemma: they risk neither correctness nor trust. The future of secure, trustworthy RAG therefore is not just more powerful search, but a coordinated, elastic, delegation of each component and the whole. This directly ties to the scalability discussion that follows, where reliability must be assessed not only in terms of throughput but in resilient throughput, meaning efficient processing within a security risk boundary.

The forthcoming economic lens thus expands the systems view of the RAG platform: the costs of latency and compute are bounded not only by hardware and model size but also by the persistence of security harms. Sustainable RAG requires pullbacks on its assumption of trust in the retrieval index. As we transition to scalable economics, sustainable RAG is not merely a cheaper implementation of the same process; it is a thoroughly different process that is indirectly more efficient because it *limits* the potential liabilities by retrieving less—when possible—and routing on cost, complexity, and sensitivity, thereby maintaining both quality and security. In that sense, the active defense of security inherently aligns with the principles of sustainable RAG, reducing the cost of reasoning by trusting sources and not contaminating the models.

10.3 —

The integration of Retrieval-Augmented Generation (RAG) into real-world systems introduces a complex and expanded threat surface that goes well beyond the traditional vulnerabilities of standalone large language models (LLMs). While RAG enhances factual accuracy and knowledge currency, it concurrently inherits the security weaknesses of its constituent parts—the parametric model, the retrieval index, and the external knowledge sources—while creating novel vulnerabilities unique to their interaction. A critical security concern highlighted in recent research is the poisoning of knowledge bases themselves. The efficacy of RAG is fundamentally reliant on the integrity of its external data store, making it a prime target for malicious actors. Attackers can manipulate data sources with the intent of subtly influencing all relevant retrieval rankings or injecting biased, harmful, or outright false contexts into the generation pipeline. This is a particularly dangerous attack vector because the LLM is then used as a vehicle to legitimize the attacker’s malicious content, and its outputs become extremely difficult to audit [441]. The risk of such poisoning is compounded by the challenges in detecting it, as demonstrated by analyses of poisoning attacks on large language models that expose the difficulty of providing safe responses and highlight the need for robust system-level defenses to ensure the integrity of the model’s final output [442].

These controls can be fragile or misconfigured, leading to data leakage. A breakdown in these mechanisms can lead to the exposure of confidential data, not just directly through the retrieval store but also indirectly through authorized inference. Decision support includes both data confidentiality and data integrity of the data being processed. Because these risks often stem from control and management gaps, a rigid security architecture for data access control becomes necessary. The privacy implications of RAG are equally severe and have been highlighted by its very nature as a knowledge system that retrieves and packages information. The process of retrieving and storing information in a database and using it for user queries introduces a dimension of external data leaks, which must be treated carefully and protected, potentially employing secure searchable encryption and data sharing techniques to protect the confidentiality of documents stored in vector databases. The failures of encryption or the use of the resulting weakness can allow an attacker to compromise the confidentiality of data [443, 444]. Excessively strong assumptions could risk the security of the infrastructure that handles encrypted and private data [445, 446].

However, security goes beyond just preventing external attacks; it also introduces the values of information leakage of sensitive data that might be embedded in the vector database. In an organizational setting, a system must comply with internal governance and data policies to often incorporate guardrails that prevent or minimize the leakage of sensitive information. In particular in highly vulnerable high-stakes domains, such as a healthcare RAG system, it is critical to protect against the confidentiality of medical data and also ensure its security during retrieval [447]. Alongside the risk of illicit access, fairness concerns emerge from the very information available to the RAG system. The promise of RAG also hinges on its ability to access the correct information, though lack of fairness can lead to biased outcomes, reinforcing system-level biases and reinforcing the already important considerations of the generation process [140].

The scalability of RAG in modern systems further complicates its concerns, as the ability to retrieve from a widely distributed source introduces new power. Secure, decentralized architectures, often building on a blockchain, have been proposed to securely log data exchanges and provide a foundation for managing privacy in high-regulated data sharing [448, 449]. However, even in these varied deployment types, the integrity of the knowledge store must be protected. The inability to fully trust an external data source or a decentralized platform to be free from malicious content is a significant problem, which holds for RAG as well [450]. Furthermore, the intelligent system can avoid the risk of storing the entire database by using an external store; however, the design must

not rely on the unrealistic assumption that the retrieved data is clean.

The deployment of RAG requires robust safeguards, meticulous, organizational approval, and the design of secure retrieval mechanisms [451]. In considering the ethical and responsible principles, researchers must consider that a data breach is an authorized party that could extract confidential and private user information. This reinforces the need to validate that the participants of a RAG system are not acting adversarially and to equip this knowledge with robust, append-only logs and cryptographic verification to ensure that knowledge is not manipulated invisibly [446, 452].

There is also an internal computational operation from a malicious and a compromised administrator to model exploitation. This could be prevented by integrating Data Leak Prevention-based machine learning to ensure that the system does not leak in cases of data exfiltration [453]. To ensure security, various methods for privacy-preserving must protect the confidentiality of user data even if encrypted [454].

In sum, RAG systems view their retrieved context as a strong, reliable source of information, required to establish a correctness check to counter adversarial manipulation and establish a trustworthy, secure, trustworthy RA. The system can not only monitor the user but also monitor fine access control on its initial data, to combine trust. Without control of the context, future and outlook are impossible. There are important steps to achieve a robust RAG: need to anonymize the data, and a RAG system that has built-in safety to robustly respond when relying on the private and unexplored model. This needs a redesign, not just another patch of robust ML, but perhaps a systems approach in which distinguishing system security and robustness with the data owner side and their intent of retrieval concern is taken into consideration.

These recommendations must be built through a multi-party approach combining both policy (human) and technical (machine) mechanisms. This entails not simply using strict firewalls or blocklisting but instead implementing privacy-preserving data sharing agreements for both compute and value, at the external knowledge or employer, and in turn making a design that produces an output which is both transparent, and not concentrating on a single source, capable of embedding high-level privacy considerations and that it is allowed to ask for *more* and be resilient to poisoning “corrupt” or contradictory data.

“Without a system that is contextually grounded in the world.

The future of secure, trustworthy RAG therefore is not just more powerful search, but a coordinated, robust, deletion of each component and the whole. This means a world in which both the system and its creators ensure that it **must** have the control and trust of the system and must use cryptographic attestation and be flexible enough to incorporate newly untrusted data in a secure way.

10.4 Scalable and Bottom-Line Sustainability

Balancing the growing demands for output quality with the escalating costs of computation, latency, and energy is one of the most critical challenges to the sustained deployment of Retrieval-Augmented Generation (RAG) systems. As RAG frameworks move from research prototypes to large-scale industrial applications, the ambition to retrieve ever-larger corpora and pass longer contexts to increasingly massive language models confronts hard economic and physical realities. The bottom-line sustainability of such systems hinges on a strategic reconsideration of when, what, and how to compute, moving beyond a paradigm of unconditional retrieval and generation towards an intelligent, cost-aware architecture.

The foundational problem is that retrieving and processing external knowledge inherently incurs a significant computational penalty. Current RAG systems often default to retrieving a fixed set of documents for every query, regardless of its complexity. This can lead to unnecessarily high latency

and energy consumption for simple requests that a model could answer from its own parametric memory. Conversely, failing to retrieve for complex questions sacrifices answer quality. Therefore, a first-order principle of a sustainable system is knowing **when not to retrieve**. As introduced in the concept of frame-aware routing, a system can be trained to predict the complexity of an incoming query and adaptively choose its strategy [455]. A high-performing approach involves a classifier that first evaluates whether a query requires external augmentation at all; it can automatically retrieve non-parametric memories *only when necessary*, significantly improving performance while reducing inference costs, as generating for questions about high-popularity entities often does not require fetching documents [456]. This “selective retrieval” approach acknowledges that in some cases, the model’s internalized knowledge handles a query efficiently, while in others it is insufficient, demanding augmentation.

Once the decision to retrieve is made, there remains significant potential to save resources in the generation phase. A major computational bottleneck of standard RAG is the necessary encoding of a large number of retrieved passages, which can be far more expensive for the model than generating the answer itself [457]. To address this, **hybrid and hierarchical strategies** for memory representation are valuable. Instead of feeding all retrieved passages directly into the model as an unprocessed stream, a system could pre-encode a corpus into a memory and retrieve dense representations directly. While this can save on-the-fly encoding, it can incur a severe quality penalty due to a lack of contextualization. A more cost-effective approach is a hybrid model, such as LUMEN, which pre-computes the majority of the retrieval representation while using a live encoder conditioned on the query for only a few layers [457]. This hybrid approach balances the computational bottleneck while preserving quality, even outperforming full on-the-fly encoding for a given computational budget, and this advantage grows with model size.

Beyond adaptive retrieval, a sustainable RAG system should incorporate **query and task routing** to manage latency and cost. Modern electronic platforms handle queries of varying difficulty with a mix of available models. A meta-level strategy for cost reduction is to use a lightweight classifier to route queries between a cheaper, smaller model and a much larger, more capable one, which can maintain quality while significantly reducing operational costs [458]. The desired quality level can be tuned dynamically at test time to seamlessly trade quality for cost as per scenario requirements. Similarly, the same principle is applied to scaling LLM-based content moderation, where heuristics filter and deduplicate clusters before sending only representative samples to an expensive LLM, creating a substantial reduction in the number of required model calls [459]. These strategies are analogous to a **hierarchical model tiering** approach, where a cheap, local algorithm handles the majority of data samples while a larger, cloud-based model processes a minority of difficult cases [460].

Given that a large portion of an RAG system’s cost lies in long-sequence generation due to an ever-expanding key-value (KV) cache, system-level **caching** and **context compression** are essential to reduce memory footprint and latency. Serving repeated or semantically similar queries directly from a cache can bypass both the generator and retriever altogether [461]. When caching is unavailable, an alternative is to compress the context itself. The memory footprint of the KV cache—which grows linearly with sequence length and batch size—can be reduced through methods that compress it, such as GEAR, which uses quantization combined with error correction to achieve near-lossless inference [171]. More radically, systems can manage the intermediate states of retrieved knowledge in a knowledge tree and cache it across GPU and host memory, dynamically overlapping retrieval and inference steps, reducing time-to-first-token (TTFT) by up to 4 times and significantly improving throughput [462]. In addition to KV caching, caching the actual LLM response can be a **privacy-aware** method to reduce cost by identifying semantically similar user queries and retrieving those answers from memory, leading to a major reduction in compute and energy demand,

while achieving higher F1-scores than naive caching methods by reducing the false hit/miss ratio [461].

Finally, to further support scalable sustainability, we must invest in **resource-aware fine-tuning** and efficient system design. Fine-tuning an entire model with retrieved contexts is expensive; parameter-efficient methods like LoRA, implemented in a distributed JAX framework, can slash memory usage and training time for retrieval-augmented generation tasks, making it feasible on limited hardware [463]. System-level optimizations should also target memory and the memory bandwidth bottlenecks. For instance, running LLM inference with 4-bit quantization (AWQ) allows deployment on low-cost edge devices, improving on-device runtime on mobile GPUs by 3x while reducing energy consumption [464]. This dedication to quantized models also helps bridge the **memory bandwidth** gap, and, via parallelism, can enable a multitude of consumer-level GPUs to collectively train and generate, reducing costs for a given system budget. More advanced approaches use a hierarchical speculative decoding system, capable of achieving up to 2.31x throughput on an A100 GPU, while showcasing scalability in handling longer contexts [328]. An intelligent system can dynamically dispatch the most appropriate strategies, moving from feeding the largest model with every token to actively forecasting and selecting which compute units to use based on what is available and necessary.

In conclusion, scalable sustainability of RAG relies not on brute-force compute for every query, but on designing a dynamic, cost-aware **ecosystem**. It is achieved by moving from “retrieve always” to “retrieve when necessary”, from one large model for all to a multi-tier system of cheap alternatives, and from storing and re-encoding everything to caching and compressing context intelligently. A network that selectively retrieves, routes to the right model, caches at the right levels, and compresses context efficiently bridges the gap between the rapid growth of information and the rigid constraints of latency and energy. Ultimately, this strategic vision is what allows RAG to push the quality envelope without exceeding the economic limits of its users and operators.

10.5 Toward Most-Next-Generation and Dynamic RAG: A Roadmap

The evolution of Retrieval-Augmented Generation (RAG) is poised to transcend its current paradigm of static, pipeline-based systems. The next generation of RAG will be characterized by dynamic, self-improving, and deeply integrated architectures that actively reason over heterogeneous knowledge sources, adapt to feedback, and continuously refine their own capabilities. Drawing from advances in active learning, multi-agent collaboration, and hierarchical reasoning, we outline a roadmap for building these most-next-generation RAG systems.

First, a central tenet of this next wave is the shift from passive retrieval to **active learning-driven mixture refinement**. In contrast to the static, cost-aware routing discussed in the previous section—where queries are dispatched to a fixed set of components based on complexity—future RAG systems will make this entire selection and fusion process *adaptive* at the instance or task level. Inspired by active learning frameworks that address data selection challenges and overconfidence issues, future RAG models will intelligently query for the most informative evidence from a mixture of sources, including dense vectors, knowledge graphs, and structured databases [465, 466]. By using predictive uncertainty as a signal, the system can decide whether to decompose a query for a knowledge graph, fetch a full passage from a corpus, or abstain from retrieval altogether and rely on its own parametric memory. This dynamic routing can be formalized as a control task: the RAG system, acting as an agent, learns to balance exploration and exploitation not only across semantic states but across the *type* of external knowledge base it perturbs. This pushes the agent beyond merely responding based on a fixed index, allowing it to actively manipulate its own learning process. The emergent behavior is a fine-grained, uncertainty-driven adaptation of the internal

mixture-of-experts across sources, a direct evolution of the “retrieve when necessary” principle introduced earlier.

Second, the roadmap should integrate **cross-modal and graph-aware latent planning** to overcome the limitations of text-only retrieval and compression studied in the previous section. The current RAG architecture primarily handles textual, unstructured information, often losing fine-grained details by flattening structured data into chunks or relying on KV-cache compression to manage context length. Future systems will be built upon a latent planning layer that operates over a unified semantic space where outputs from visual encoders, textual chunks, and knowledge-graph nodes coexist cohesively. In this model, the generation process is no longer monolithic but is guided by a dynamic plan of linked knowledge nodes, enabling reasoning over complex relationships in a graph or an adaptive zoom into a salient spatial region. This is supported by systems that compose plans of multimodal procedural steps, leveraging code-style scaffolds for adaptive refinement [467, 468]. Each step of the reasoning sequence can trigger *dynamically* a retrieval event, allowing the model to traverse modality boundaries and integrate evidence effectively. This graph/visual fusion enables the “reasoner” to not just read text but literally *inspect* the evidence, thereby mitigating the information loss suffered when converting visual structures into sequential token streams—an improvement over the context-compression techniques described in the previous section.

Third, the roadmap incorporates **multi-agent feedback systems** to achieve more robust, consultative evaluation. Future RAG will not be a single monolithic module but an ensemble of specialists that can collaborate, challenge, and correct each other, much like an academic peer-review process where multiple LLM agents produce candidates and engage in iterative review and revision [469]. Instead of relying on majority voting, the system will engage in debate, with actors using attention mechanisms to capture inter-agent dynamics and identify common ground [470, 471]. This interaction allows for decentralized decision-making under uncertainty, where the mixture of sources is evaluated iteratively. This concept aligns with research that fosters interactive discussion between models, enhancing inference with adversarial remarks and robust training [472]. Within RAG, these agents might specialize in detecting factual inconsistencies, profiling user intent, or searching for fine-grained state information, with a super-set controller using end-to-end collaborative metrics to filter noise. This enhancement reframes RAG from a pipeline into a dynamical cognitive process—if one component is challenged, the entire system converges to more trustworthy and self-aware predictions, complementing the multi-tier routing and caching strategies of the previous section by adding a layer of interactive verification.

Fourth, a potential leap is in enabling **continuous self-supervised, self-survival update** of both the index and parameters without catastrophic forgetting. The system will operate under a belief-updating engine that stores relevant information and *dynamically updates its model* on the fly. A central challenge is reconciling newly retrieved evidence with previously “stable” beliefs. Here, solutions will be inspired by hybrid models with multiple workflows, including specific modules for specialized state, such as an Active Inference agent that learns to attract relevant state space via observation, modality, and goal [473, 474]. Instead of full recomputation, we propose a self-optimizing algorithm based on “rotating neurons” and strategic sampling via mutual information mechanisms [475]. As time evolves, the model improves its accumulated understanding of both the external world and user preferences; this closes the loop with a contextual bandit that determines whether the retriever or generator should be capitalized on delayed outcomes. Such a living system will maintain uncertainty as an epistemic state, not only to reason about its own bounds but also to drive experiments with new data chunks—a significant step forward from the static-fusion methods in earlier sections, moving towards “living documents” where the system continuously assimilates and distills knowledge, reminiscent of extraction and generation loops [476].

Finally, the roadmap crystallizes with the development of **efficient and calibrated deep**

learning and verification for safety-oriented real-world settings. Multi-agent RAG systems with active hyperparameter updates will require enormous KV-caches and more detailed model architectures. To be deployable in daily life, all slices of computation need to be handled through the design of *frugal exploration* and *adaptive solution searching*. The system might select not to retrieve or generate at all when retrieval cost outweighs the expected performance gain—echoing the adaptive cost-mitigation principles from the earlier subsection, but now extended to a fully integrated, uncertainty-driven context. Then, for final trustworthy output, we lay on post-hoc verification: rather than relying solely on autoregressive output, the RAG system employs a *semantic checking process* to compare encoded predictions against existing facts, aligning the internal state with both human comprehension and preferences.

In conclusion, the road toward next-generation dynamic RAG is not a single invention but a cumulative architectural reformation—a direct continuation of the economic and efficiency-driven principles toward adaptive retrieval and resource-aware system design discussed in the preceding section. The path moves from “retriever-reader” to a multi-modal, multi-agent specialist ensemble. By actively designing its own routing policies, using latent planning that includes both local and global dependencies, receiving multi-agent feedback to tune internal beliefs through iterative workflows, and implementing continuous update and verification, **we aim to devise a co-adaptive artificial intelligence**. The synthesis of these points—**active learning with mixture refinement, cross-modal graph planning, multi-agent feedback, and self-survival updates**—will drive RAG from a simple pipeline towards an extended, robust, and trustworthy generative framework.

References

- [1] Puzzle Solving using Reasoning of Large Language Models A Survey. arXiv:2402.11291v2, 2024. <https://arxiv.org/abs/2402.11291v2>
- [2] On Exploring the Reasoning Capability of Large Language Models with Knowledge Graphs. arXiv:2312.00353v1, 2023. <https://arxiv.org/abs/2312.00353v1>
- [3] NPHardEval Dynamic Benchmark on Reasoning Ability of Large Language Models via Complexity Classes. arXiv:2312.14890v4, 2023. <https://arxiv.org/abs/2312.14890v4>
- [4] Evaluating Consistency and Reasoning Capabilities of Large Language Models. arXiv:2404.16478v1, 2024. <https://arxiv.org/abs/2404.16478v1>
- [5] Journey of Hallucination-minimized Generative AI Solutions for Financial Decision Makers. arXiv:2311.10961v1, 2023. <https://arxiv.org/abs/2311.10961v1>
- [6] Hallucination is Inevitable An Innate Limitation of Large Language Models. arXiv:2401.11817v1, 2024. <https://arxiv.org/abs/2401.11817v1>
- [7] Can Knowledge Graphs Reduce Hallucinations in LLMs A Survey. arXiv:2311.07914v2, 2023. <https://arxiv.org/abs/2311.07914v2>
- [8] Why LLMs Hallucinate, and How to Get (Evidential) Closure Perceptual, Intensional, and Extensional Learning for Faithful Natural Language Generation. arXiv:2310.15355v1, 2023. <https://arxiv.org/abs/2310.15355v1>
- [9] (A)I Am Not a Lawyer, But... Engaging Legal Experts towards Responsible LLM Policies for Legal Advice. arXiv:2402.01864v1, 2024. <https://arxiv.org/abs/2402.01864v1>

- [10] Large Legal Fictions Profiling Legal Hallucinations in Large Language Models. arXiv:2401.01301v1, 2024. <https://arxiv.org/abs/2401.01301v1>
- [11] Towards Mitigating Hallucination in Large Language Models via Self-Reflection. arXiv:2310.06271v1, 2023. <https://arxiv.org/abs/2310.06271v1>
- [12] Deficiency of Large Language Models in Finance An Empirical Examination of Hallucination. arXiv:2311.15548v1, 2023. <https://arxiv.org/abs/2311.15548v1>
- [13] When Do LLMs Need Retrieval Augmentation Mitigating LLMs’ Overconfidence Helps Retrieval Augmentation. arXiv:2402.11457v1, 2024. <https://arxiv.org/abs/2402.11457v1>
- [14] Knowledge of Knowledge Exploring Known-Unknowns Uncertainty with Large Language Models. arXiv:2305.13712v1, 2023. <https://arxiv.org/abs/2305.13712v1>
- [15] RAGged Edges The Double-Edged Sword of Retrieval-Augmented Chatbots. arXiv:2403.01193v2, 2024. <https://arxiv.org/abs/2403.01193v2>
- [16] A Principled Framework for Knowledge-enhanced Large Language Model. arXiv:2311.11135v1, 2023. <https://arxiv.org/abs/2311.11135v1>
- [17] A collection of principles for guiding and evaluating large language models. arXiv:2312.10059v1, 2023. <https://arxiv.org/abs/2312.10059v1>
- [18] An Enhanced Prompt-Based LLM Reasoning Scheme via Knowledge Graph-Integrated Collaboration. arXiv:2402.04978v1, 2024. <https://arxiv.org/abs/2402.04978v1>
- [19] Deceptive Semantic Shortcuts on Reasoning Chains How Far Can Models Go without Hallucination. arXiv:2311.09702v3, 2023. <https://arxiv.org/abs/2311.09702v3>
- [20] Towards Uncovering How Large Language Model Works An Explainability Perspective. arXiv:2402.10688v2, 2024. <https://arxiv.org/abs/2402.10688v2>
- [21] Large Language Models Cannot Self-Correct Reasoning Yet. arXiv:2310.01798v2, 2023. <https://arxiv.org/abs/2310.01798v2>
- [22] Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401v4, 2020. <https://arxiv.org/abs/2005.11401v4>
- [23] Retrieval-Augmented Generation for Large Language Models A Survey. arXiv:2312.10997v5, 2023. <https://arxiv.org/abs/2312.10997v5>
- [24] Augmenting LLMs with Knowledge A survey on hallucination prevention. arXiv:2309.16459v1, 2023. <https://arxiv.org/abs/2309.16459v1>
- [25] Fine Tuning vs. Retrieval Augmented Generation for Less Popular Knowledge. arXiv:2403.01432v2, 2024. <https://arxiv.org/abs/2403.01432v2>
- [26] RAGAS Automated Evaluation of Retrieval Augmented Generation. arXiv:2309.15217v1, 2023. <https://arxiv.org/abs/2309.15217v1>
- [27] Benchmarking Retrieval-Augmented Generation for Medicine. arXiv:2402.13178v2, 2024. <https://arxiv.org/abs/2402.13178v2>

- [28] Studying Large Language Model Behaviors Under Realistic Knowledge Conflicts. arXiv:2404.16032v1, 2024. <https://arxiv.org/abs/2404.16032v1>
- [29] Improving Retrieval for RAG based Question Answering Models on Financial Documents. arXiv:2404.07221v1, 2024. <https://arxiv.org/abs/2404.07221v1>
- [30] Fine-Tuning or Retrieval Comparing Knowledge Injection in LLMs. arXiv:2312.05934v3, 2023. <https://arxiv.org/abs/2312.05934v3>
- [31] PaperQA Retrieval-Augmented Generative Agent for Scientific Research. arXiv:2312.07559v2, 2023. <https://arxiv.org/abs/2312.07559v2>
- [32] MedAgents Large Language Models as Collaborators for Zero-shot Medical Reasoning. arXiv:2311.10537v3, 2023. <https://arxiv.org/abs/2311.10537v3>
- [33] MemLLM Finetuning LLMs to Use An Explicit Read-Write Memory. arXiv:2404.11672v1, 2024. <https://arxiv.org/abs/2404.11672v1>
- [34] Benchmarking Large Language Models in Retrieval-Augmented Generation. arXiv:2309.01431v2, 2023. <https://arxiv.org/abs/2309.01431v2>
- [35] Not All Contexts Are Equal Teaching LLMs Credibility-aware Generation. arXiv:2404.06809v1, 2024. <https://arxiv.org/abs/2404.06809v1>
- [36] ActiveRAG Revealing the Treasures of Knowledge via Active Learning. arXiv:2402.13547v1, 2024. <https://arxiv.org/abs/2402.13547v1>
- [37] How faithful are RAG models Quantifying the tug-of-war between RAG and LLMs’ internal prior. arXiv:2404.10198v1, 2024. <https://arxiv.org/abs/2404.10198v1>
- [38] A Survey on Retrieval-Augmented Text Generation for Large Language Models. arXiv:2404.10981v1, 2024. <https://arxiv.org/abs/2404.10981v1>
- [39] LLMs Know What They Need Leveraging a Missing Information Guided Framework to Empower Retrieval-Augmented Generation. arXiv:2404.14043v1, 2024. <https://arxiv.org/abs/2404.14043v1>
- [40] Corrective Retrieval Augmented Generation. arXiv:2401.15884v2, 2024. <https://arxiv.org/abs/2401.15884v2>
- [41] Self-RAG Learning to Retrieve, Generate, and Critique through Self-Reflection. arXiv:2310.11511v1, 2023. <https://arxiv.org/abs/2310.11511v1>
- [42] RA-ISF Learning to Answer and Understand from Retrieval Augmentation via Iterative Self-Feedback. arXiv:2403.06840v1, 2024. <https://arxiv.org/abs/2403.06840v1>
- [43] A Survey of Visual Analytics Techniques for Machine Learning. arXiv:2008.09632v1, 2020. <https://arxiv.org/abs/2008.09632v1>
- [44] A Data-Centric Methodology and Task Typology for Time-Stamped Event Sequences. arXiv:2209.10181v1, 2022. <https://arxiv.org/abs/2209.10181v1>
- [45] Prompting or Fine-tuning A Comparative Study of Large Language Models for Taxonomy Construction. arXiv:2309.01715v1, 2023. <https://arxiv.org/abs/2309.01715v1>

- [46] Foundation Models for Time Series Analysis A Tutorial and Survey. arXiv:2403.14735v2, 2024. <https://arxiv.org/abs/2403.14735v2>
- [47] Empirical and Experimental Perspectives on Big Data in Recommendation Systems A Comprehensive Survey. arXiv:2402.03368v1, 2024. <https://arxiv.org/abs/2402.03368v1>
- [48] Understanding metric-related pitfalls in image analysis validation. arXiv:2302.01790v4, 2023. <https://arxiv.org/abs/2302.01790v4>
- [49] MultiHop-RAG Benchmarking Retrieval-Augmented Generation for Multi-Hop Queries. arXiv:2401.15391v1, 2024. <https://arxiv.org/abs/2401.15391v1>
- [50] The Power of Noise Redefining Retrieval for RAG Systems. arXiv:2401.14887v3, 2024. <https://arxiv.org/abs/2401.14887v3>
- [51] Prompt-RAG Pioneering Vector Embedding-Free Retrieval-Augmented Generation in Niche Domains, Exemplified by Korean Medicine. arXiv:2401.11246v1, 2024. <https://arxiv.org/abs/2401.11246v1>
- [52] From Local to Global A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130v1, 2024. <https://arxiv.org/abs/2404.16130v1>
- [53] Assessing generalization capability of text ranking models in Polish. arXiv:2402.14318v1, 2024. <https://arxiv.org/abs/2402.14318v1>
- [54] Improving language models by retrieving from trillions of tokens. arXiv:2112.04426v3, 2021. <https://arxiv.org/abs/2112.04426v3>
- [55] Bridging the Preference Gap between Retrievers and LLMs. arXiv:2401.06954v2, 2024. <https://arxiv.org/abs/2401.06954v2>
- [56] UniMS-RAG A Unified Multi-source Retrieval-Augmented Generation for Personalized Dialogue Systems. arXiv:2401.13256v1, 2024. <https://arxiv.org/abs/2401.13256v1>
- [57] PRCA Fitting Black-Box Large Language Models for Retrieval Question Answering via Pluggable Reward-Driven Contextual Adapter. arXiv:2310.18347v1, 2023. <https://arxiv.org/abs/2310.18347v1>
- [58] RAGGED Towards Informed Design of Retrieval Augmented Generation Systems. arXiv:2403.09040v1, 2024. <https://arxiv.org/abs/2403.09040v1>
- [59] Enhancing LLM Intelligence with ARM-RAG Auxiliary Rationale Memory for Retrieval Augmented Generation. arXiv:2311.04177v1, 2023. <https://arxiv.org/abs/2311.04177v1>
- [60] Think Before You Act Decision Transformers with Internal Working Memory. arXiv:2305.16338v1, 2023. <https://arxiv.org/abs/2305.16338v1>
- [61] Retrieval-Augmented Generation Is Dense Passage Retrieval Retrieving. arXiv:2402.11035v2, 2024. <https://arxiv.org/abs/2402.11035v2>
- [62] Intuitive or Dependent Investigating LLMs’ Behavior Style to Conflicting Prompts. arXiv:2309.17415v3, 2023. <https://arxiv.org/abs/2309.17415v3>

- [63] Adaptive Chameleon or Stubborn Sloth Revealing the Behavior of Large Language Models in Knowledge Conflicts. arXiv:2305.13300v4, 2023. <https://arxiv.org/abs/2305.13300v4>
- [64] An Artificial Neuron for Enhanced Problem Solving in Large Language Models. arXiv:2404.14222v1, 2024. <https://arxiv.org/abs/2404.14222v1>
- [65] Improving Medical Reasoning through Retrieval and Self-Reflection with Retrieval-Augmented Large Language Models. arXiv:2401.15269v2, 2024. <https://arxiv.org/abs/2401.15269v2>
- [66] FIT-RAG Black-Box RAG with Factual Information and Token Reduction. arXiv:2403.14374v1, 2024. <https://arxiv.org/abs/2403.14374v1>
- [67] Mafin Enhancing Black-Box Embeddings with Model Augmented Fine-Tuning. arXiv:2402.12177v4, 2024. <https://arxiv.org/abs/2402.12177v4>
- [68] T-RAG Lessons from the LLM Trenches. arXiv:2402.07483v1, 2024. <https://arxiv.org/abs/2402.07483v1>
- [69] A Fine-tuning Enhanced RAG System with Quantized Influence Measure as AI Judge. arXiv:2402.17081v1, 2024. <https://arxiv.org/abs/2402.17081v1>
- [70] JORA JAX Tensor-Parallel LoRA Library for Retrieval Augmented Fine-Tuning. arXiv:2403.11366v2, 2024. <https://arxiv.org/abs/2403.11366v2>
- [71] Enhancing Q&A with Domain-Specific Fine-Tuning and Iterative Reasoning A Comparative Study. arXiv:2404.11792v2, 2024. <https://arxiv.org/abs/2404.11792v2>
- [72] RAG vs Fine-tuning Pipelines, Tradeoffs, and a Case Study on Agriculture. arXiv:2401.08406v3, 2024. <https://arxiv.org/abs/2401.08406v3>
- [73] MASTISK. arXiv:1804.00912v1, 2018. <https://arxiv.org/abs/1804.00912v1>
- [74] SparTerm Learning Term-based Sparse Representation for Fast Text Retrieval. arXiv:2010.00768v1, 2020. <https://arxiv.org/abs/2010.00768v1>
- [75] BM25 Query Augmentation Learned End-to-End. arXiv:2305.14087v1, 2023. <https://arxiv.org/abs/2305.14087v1>
- [76] A Replication Study of Dense Passage Retriever. arXiv:2104.05740v1, 2021. <https://arxiv.org/abs/2104.05740v1>
- [77] Learning To Retrieve How to Train a Dense Retrieval Model Effectively and Efficiently. arXiv:2010.10469v1, 2020. <https://arxiv.org/abs/2010.10469v1>
- [78] Constructing Tree-based Index for Efficient and Effective Dense Retrieval. arXiv:2304.11943v1, 2023. <https://arxiv.org/abs/2304.11943v1>
- [79] A Thorough Examination on Zero-shot Dense Retrieval. arXiv:2204.12755v2, 2022. <https://arxiv.org/abs/2204.12755v2>
- [80] SPLADE v2 Sparse Lexical and Expansion Model for Information Retrieval. arXiv:2109.10086v1, 2021. <https://arxiv.org/abs/2109.10086v1>
- [81] A Unified Framework for Learned Sparse Retrieval. arXiv:2303.13416v1, 2023. <https://arxiv.org/abs/2303.13416v1>

- [82] Multimodal Learned Sparse Retrieval with Probabilistic Expansion Control. arXiv:2402.17535v1, 2024. <https://arxiv.org/abs/2402.17535v1>
- [83] ColBERT Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. arXiv:2004.12832v2, 2020. <https://arxiv.org/abs/2004.12832v2>
- [84] A comment on Guo et al. [arXiv 2206.11228]. arXiv:2208.01456v1, 2022. <https://arxiv.org/abs/2208.01456v1>
- [85] An Analysis on Matching Mechanisms and Token Pruning for Late-interaction Models. arXiv:2403.13291v1, 2024. <https://arxiv.org/abs/2403.13291v1>
- [86] On the Reverse Engineering of the Citadel Botnet. arXiv:1406.5569v1, 2014. <https://arxiv.org/abs/1406.5569v1>
- [87] SLIM Sparsified Late Interaction for Multi-Vector Retrieval with Inverted Indexes. arXiv:2302.06587v2, 2023. <https://arxiv.org/abs/2302.06587v2>
- [88] SPLATE Sparse Late Interaction Retrieval. arXiv:2404.13950v1, 2024. <https://arxiv.org/abs/2404.13950v1>
- [89] UnifieR A Unified Retriever for Large-Scale Retrieval. arXiv:2205.11194v2, 2022. <https://arxiv.org/abs/2205.11194v2>
- [90] Out-of-Domain Semantics to the Rescue! Zero-Shot Hybrid Retrieval Models. arXiv:2201.10582v1, 2022. <https://arxiv.org/abs/2201.10582v1>
- [91] Sparse Meets Dense A Hybrid Approach to Enhance Scientific Document Retrieval. arXiv:2401.04055v1, 2024. <https://arxiv.org/abs/2401.04055v1>
- [92] Perspectives on the State and Future of Deep Learning - 2023. arXiv:2312.09323v3, 2023. <https://arxiv.org/abs/2312.09323v3>
- [93] Are We There Yet A Decision Framework for Replacing Term Based Retrieval with Dense Retrieval Systems. arXiv:2206.12993v1, 2022. <https://arxiv.org/abs/2206.12993v1>
- [94] A White Box Analysis of ColBERT. arXiv:2012.09650v1, 2020. <https://arxiv.org/abs/2012.09650v1>
- [95] SPRINT A Unified Toolkit for Evaluating and Demystifying Zero-shot Neural Sparse Retrieval. arXiv:2307.10488v1, 2023. <https://arxiv.org/abs/2307.10488v1>
- [96] Predicting Efficiency Effectiveness Trade-offs for Dense vs. Sparse Retrieval Strategy Selection. arXiv:2109.10739v1, 2021. <https://arxiv.org/abs/2109.10739v1>
- [97] ARRIVAL Next Stop in CLS. arXiv:1802.07702v1, 2018. <https://arxiv.org/abs/1802.07702v1>
- [98] Pseudo-Relevance Feedback for Multiple Representation Dense Retrieval. arXiv:2106.11251v2, 2021. <https://arxiv.org/abs/2106.11251v2>
- [99] LoL A Comparative Regularization Loss over Query Reformulation Losses for Pseudo-Relevance Feedback. arXiv:2204.11545v1, 2022. <https://arxiv.org/abs/2204.11545v1>

- [100] BlendFilter Advancing Retrieval-Augmented Large Language Models via Query Generation Blending and Knowledge Filtering. arXiv:2402.11129v1, 2024. <https://arxiv.org/abs/2402.11129v1>
- [101] Query Expansion by Prompting Large Language Models. arXiv:2305.03653v1, 2023. <https://arxiv.org/abs/2305.03653v1>
- [102] Generative Relevance Feedback with Large Language Models. arXiv:2304.13157v1, 2023. <https://arxiv.org/abs/2304.13157v1>
- [103] Query2doc Query Expansion with Large Language Models. arXiv:2303.07678v2, 2023. <https://arxiv.org/abs/2303.07678v2>
- [104] GRM Generative Relevance Modeling Using Relevance-Aware Sample Estimation for Document Retrieval. arXiv:2306.09938v1, 2023. <https://arxiv.org/abs/2306.09938v1>
- [105] Ask Optimal Questions Aligning Large Language Models with Retriever’s Preference in Conversational Search. arXiv:2402.11827v1, 2024. <https://arxiv.org/abs/2402.11827v1>
- [106] Query Rewriting for Retrieval-Augmented Large Language Models. arXiv:2305.14283v3, 2023. <https://arxiv.org/abs/2305.14283v3>
- [107] Search-in-the-Chain Interactively Enhancing Large Language Models with Search for Knowledge-intensive Tasks. arXiv:2304.14732v7, 2023. <https://arxiv.org/abs/2304.14732v7>
- [108] ProTIP Progressive Tool Retrieval Improves Planning. arXiv:2312.10332v1, 2023. <https://arxiv.org/abs/2312.10332v1>
- [109] Leveraging Translation For Optimal Recall Tailoring LLM Personalization With User Profiles. arXiv:2402.13500v1, 2024. <https://arxiv.org/abs/2402.13500v1>
- [110] Augmenting Passage Representations with Query Generation for Enhanced Cross-Lingual Dense Retrieval. arXiv:2305.03950v1, 2023. <https://arxiv.org/abs/2305.03950v1>
- [111] Dense X Retrieval What Retrieval Granularity Should We Use. arXiv:2312.06648v2, 2023. <https://arxiv.org/abs/2312.06648v2>
- [112] Hi-Gen Generative Retrieval For Large-Scale Personalized E-commerce Search. arXiv:2404.15675v1, 2024. <https://arxiv.org/abs/2404.15675v1>
- [113] EHI End-to-end Learning of Hierarchical Index for Efficient Dense Retrieval. arXiv:2310.08891v1, 2023. <https://arxiv.org/abs/2310.08891v1>
- [114] LIDER An Efficient High-dimensional Learned Index for Large-scale Dense Passage Retrieval. arXiv:2205.00970v3, 2022. <https://arxiv.org/abs/2205.00970v3>
- [115] Query-Specific Knowledge Graphs for Complex Finance Topics. arXiv:2211.04142v1, 2022. <https://arxiv.org/abs/2211.04142v1>
- [116] ATLANTIC Structure-Aware Retrieval-Augmented Language Model for Interdisciplinary Science. arXiv:2311.12289v1, 2023. <https://arxiv.org/abs/2311.12289v1>

- [117] CorpusLM Towards a Unified Language Model on Corpus for Knowledge-Intensive Tasks. arXiv:2402.01176v2, 2024. <https://arxiv.org/abs/2402.01176v2>
- [118] How Does Generative Retrieval Scale to Millions of Passages. arXiv:2305.11841v1, 2023. <https://arxiv.org/abs/2305.11841v1>
- [119] Generative Retrieval as Dense Retrieval. arXiv:2306.11397v1, 2023. <https://arxiv.org/abs/2306.11397v1>
- [120] DSI++ Updating Transformer Memory with New Documents. arXiv:2212.09744v3, 2022. <https://arxiv.org/abs/2212.09744v3>
- [121] Continual Learning for Generative Retrieval over Dynamic Corpora. arXiv:2308.14968v1, 2023. <https://arxiv.org/abs/2308.14968v1>
- [122] Pre-training with Large Language Model-based Document Expansion for Dense Passage Retrieval. arXiv:2308.08285v1, 2023. <https://arxiv.org/abs/2308.08285v1>
- [123] Bridging the Gap Between Indexing and Retrieval for Differentiable Search Index with Query Generation. arXiv:2206.10128v3, 2022. <https://arxiv.org/abs/2206.10128v3>
- [124] Large Search Model Redefining Search Stack in the Era of LLMs. arXiv:2310.14587v2, 2023. <https://arxiv.org/abs/2310.14587v2>
- [125] Coverage-based Example Selection for In-Context Learning. arXiv:2305.14907v3, 2023. <https://arxiv.org/abs/2305.14907v3>
- [126] Comparable Demonstrations are Important in In-Context Learning A Novel Perspective on Demonstration Selection. arXiv:2312.07476v2, 2023. <https://arxiv.org/abs/2312.07476v2>
- [127] In-context Learning with Retrieved Demonstrations for Language Models A Survey. arXiv:2401.11624v5, 2024. <https://arxiv.org/abs/2401.11624v5>
- [128] Dr.ICL Demonstration-Retrieved In-context Learning. arXiv:2305.14128v1, 2023. <https://arxiv.org/abs/2305.14128v1>
- [129] $\$Se^2\$$ Sequential Example Selection for In-Context Learning. arXiv:2402.13874v2, 2024. <https://arxiv.org/abs/2402.13874v2>
- [130] Latent Skill Discovery for Chain-of-Thought Reasoning. arXiv:2312.04684v1, 2023. <https://arxiv.org/abs/2312.04684v1>
- [131] Active Prompting with Chain-of-Thought for Large Language Models. arXiv:2302.12246v3, 2023. <https://arxiv.org/abs/2302.12246v3>
- [132] Rectifying Demonstration Shortcut in In-Context Learning. arXiv:2403.09488v3, 2024. <https://arxiv.org/abs/2403.09488v3>
- [133] How Many Demonstrations Do You Need for In-context Learning. arXiv:2303.08119v3, 2023. <https://arxiv.org/abs/2303.08119v3>
- [134] Self-ICL Zero-Shot In-Context Learning with Self-Generated Demonstrations. arXiv:2305.15035v2, 2023. <https://arxiv.org/abs/2305.15035v2>

- [135] Self-Demos Eliciting Out-of-Demonstration Generalizability in Large Language Models. arXiv:2404.00884v1, 2024. <https://arxiv.org/abs/2404.00884v1>
- [136] Demonstrate-Search-Predict Composing retrieval and language models for knowledge-intensive NLP. arXiv:2212.14024v2, 2022. <https://arxiv.org/abs/2212.14024v2>
- [137] Instruction Tuning for Large Language Models A Survey. arXiv:2308.10792v5, 2023. <https://arxiv.org/abs/2308.10792v5>
- [138] Exploring the Relationship between In-Context Learning and Instruction Tuning. arXiv:2311.10367v1, 2023. <https://arxiv.org/abs/2311.10367v1>
- [139] How Does In-Context Learning Help Prompt Tuning. arXiv:2302.11521v1, 2023. <https://arxiv.org/abs/2302.11521v1>
- [140] Addressing Order Sensitivity of In-Context Demonstration Examples in Causal Language Models. arXiv:2402.15637v1, 2024. <https://arxiv.org/abs/2402.15637v1>
- [141] Instruction Tuned Models are Quick Learners. arXiv:2306.05539v1, 2023. <https://arxiv.org/abs/2306.05539v1>
- [142] A Unified Causal View of Instruction Tuning. arXiv:2402.06220v1, 2024. <https://arxiv.org/abs/2402.06220v1>
- [143] Instructed to Bias Instruction-Tuned Language Models Exhibit Emergent Cognitive Bias. arXiv:2308.00225v2, 2023. <https://arxiv.org/abs/2308.00225v2>
- [144] Active Instruction Tuning Improving Cross-Task Generalization by Training on Prompt Sensitive Tasks. arXiv:2311.00288v1, 2023. <https://arxiv.org/abs/2311.00288v1>
- [145] Instruction Diversity Drives Generalization To Unseen Tasks. arXiv:2402.10891v1, 2024. <https://arxiv.org/abs/2402.10891v1>
- [146] CodecLM Aligning Language Models with Tailored Synthetic Data. arXiv:2404.05875v1, 2024. <https://arxiv.org/abs/2404.05875v1>
- [147] Ensemble-Instruct Generating Instruction-Tuning Data with a Heterogeneous Mixture of LMs. arXiv:2310.13961v1, 2023. <https://arxiv.org/abs/2310.13961v1>
- [148] Harnessing the Power of David against Goliath Exploring Instruction Data Generation without Using Closed-Source Models. arXiv:2308.12711v1, 2023. <https://arxiv.org/abs/2308.12711v1>
- [149] Context-dependent Instruction Tuning for Dialogue Response Generation. arXiv:2311.07006v1, 2023. <https://arxiv.org/abs/2311.07006v1>
- [150] EntGPT Linking Generative Large Language Models with Knowledge Bases. arXiv:2402.06738v1, 2024. <https://arxiv.org/abs/2402.06738v1>
- [151] Do Models Really Learn to Follow Instructions An Empirical Study of Instruction Tuning. arXiv:2305.11383v2, 2023. <https://arxiv.org/abs/2305.11383v2>
- [152] Guideline Learning for In-context Information Extraction. arXiv:2310.05066v2, 2023. <https://arxiv.org/abs/2310.05066v2>

- [153] Differentiable Instruction Optimization for Cross-Task Generalization. arXiv:2306.10098v1, 2023. <https://arxiv.org/abs/2306.10098v1>
- [154] OPT-IML Scaling Language Model Instruction Meta Learning through the Lens of Generalization. arXiv:2212.12017v3, 2022. <https://arxiv.org/abs/2212.12017v3>
- [155] Instruction-tuning Aligns LLMs to the Human Brain. arXiv:2312.00575v1, 2023. <https://arxiv.org/abs/2312.00575v1>
- [156] Hexa Self-Improving for Knowledge-Grounded Dialogue System. arXiv:2310.06404v3, 2023. <https://arxiv.org/abs/2310.06404v3>
- [157] Automatic Evaluation of Generative Models with Instruction Tuning. arXiv:2310.20072v1, 2023. <https://arxiv.org/abs/2310.20072v1>
- [158] TRUE Re-evaluating Factual Consistency Evaluation. arXiv:2204.04991v3, 2022. <https://arxiv.org/abs/2204.04991v3>
- [159] A Controllable Model of Grounded Response Generation. arXiv:2005.00613v2, 2020. <https://arxiv.org/abs/2005.00613v2>
- [160] Are Pre-trained Language Models Knowledgeable to Ground Open Domain Dialogues. arXiv:2011.09708v1, 2020. <https://arxiv.org/abs/2011.09708v1>
- [161] A Glitch in the Matrix Locating and Detecting Language Model Grounding with Fakepedia. arXiv:2312.02073v2, 2023. <https://arxiv.org/abs/2312.02073v2>
- [162] Compressing Context to Enhance Inference Efficiency of Large Language Models. arXiv:2310.06201v1, 2023. <https://arxiv.org/abs/2310.06201v1>
- [163] Deleter Leveraging BERT to Perform Unsupervised Successive Text Compression. arXiv:1909.03223v1, 2019. <https://arxiv.org/abs/1909.03223v1>
- [164] RECOMP Improving Retrieval-Augmented LMs with Compression and Selective Augmentation. arXiv:2310.04408v1, 2023. <https://arxiv.org/abs/2310.04408v1>
- [165] In-context Autoencoder for Context Compression in a Large Language Model. arXiv:2307.06945v3, 2023. <https://arxiv.org/abs/2307.06945v3>
- [166] LLoCO Learning Long Contexts Offline. arXiv:2404.07979v1, 2024. <https://arxiv.org/abs/2404.07979v1>
- [167] LongLLMLingua Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. arXiv:2310.06839v1, 2023. <https://arxiv.org/abs/2310.06839v1>
- [168] Found in the Middle How Language Models Use Long Contexts Better via Plug-and-Play Positional Encoding. arXiv:2403.04797v1, 2024. <https://arxiv.org/abs/2403.04797v1>
- [169] Make Your LLM Fully Utilize the Context. arXiv:2404.16811v2, 2024. <https://arxiv.org/abs/2404.16811v2>
- [170] Training With Paraphrasing the Original Text” Improves Long-Context Performance. arXiv:2312.11193v8, 2023. <https://arxiv.org/abs/2312.11193v8>

- [171] GEAR An Efficient KV Cache Compression Recipe for Near-Lossless Generative Inference of LLM. arXiv:2403.05527v2, 2024. <https://arxiv.org/abs/2403.05527v2>
- [172] No Token Left Behind Reliable KV Cache Compression via Importance-Aware Mixed Precision Quantization. arXiv:2402.18096v1, 2024. <https://arxiv.org/abs/2402.18096v1>
- [173] SnapKV LLM Knows What You are Looking for Before Generation. arXiv:2404.14469v1, 2024. <https://arxiv.org/abs/2404.14469v1>
- [174] SqueezeAttention 2D Management of KV-Cache in LLM Inference via Layer-wise Optimal Budget. arXiv:2404.04793v1, 2024. <https://arxiv.org/abs/2404.04793v1>
- [175] LoMA Lossless Compressed Memory Attention. arXiv:2401.09486v2, 2024. <https://arxiv.org/abs/2401.09486v2>
- [176] StreamingDialogue Prolonged Dialogue Learning via Long Context Compression with Minimal Losses. arXiv:2403.08312v1, 2024. <https://arxiv.org/abs/2403.08312v1>
- [177] Prompt Cache Modular Attention Reuse for Low-Latency Inference. arXiv:2311.04934v2, 2023. <https://arxiv.org/abs/2311.04934v2>
- [178] GMAT Global Memory Augmentation for Transformers. arXiv:2006.03274v1, 2020. <https://arxiv.org/abs/2006.03274v1>
- [179] Attendre Wait To Attend By Retrieval With Evicted Queries in Memory-Based Transformers for Long Context Processing. arXiv:2401.04881v1, 2024. <https://arxiv.org/abs/2401.04881v1>
- [180] Dialectical Alignment Resolving the Tension of 3H and Security Threats of LLMs. arXiv:2404.00486v1, 2024. <https://arxiv.org/abs/2404.00486v1>
- [181] DoLa Decoding by Contrasting Layers Improves Factuality in Large Language Models. arXiv:2309.03883v2, 2023. <https://arxiv.org/abs/2309.03883v2>
- [182] Sequence-Level Certainty Reduces Hallucination In Knowledge-Grounded Dialogue Generation. arXiv:2310.18794v3, 2023. <https://arxiv.org/abs/2310.18794v3>
- [183] Chain-of-Verification Reduces Hallucination in Large Language Models. arXiv:2309.11495v2, 2023. <https://arxiv.org/abs/2309.11495v2>
- [184] CONFLARE CONFormal Large language model RETrieval. arXiv:2404.04287v1, 2024. <https://arxiv.org/abs/2404.04287v1>
- [185] TRAQ Trustworthy Retrieval Augmented Question Answering via Conformal Prediction. arXiv:2307.04642v2, 2023. <https://arxiv.org/abs/2307.04642v2>
- [186] C-RAG Certified Generation Risks for Retrieval-Augmented Language Models. arXiv:2402.03181v3, 2024. <https://arxiv.org/abs/2402.03181v3>
- [187] On Linking Level Segments. arXiv:2203.05057v2, 2022. <https://arxiv.org/abs/2203.05057v2>
- [188] RCOT Detecting and Rectifying Factual Inconsistency in Reasoning by Reversing Chain-of-Thought. arXiv:2305.11499v2, 2023. <https://arxiv.org/abs/2305.11499v2>

- [189] Self-Contradictory Reasoning Evaluation and Detection. arXiv:2311.09603v2, 2023. <https://arxiv.org/abs/2311.09603v2>
- [190] Can LLMs Produce Faithful Explanations For Fact-checking Towards Faithful Explainable Fact-Checking via Multi-Agent Debate. arXiv:2402.07401v1, 2024. <https://arxiv.org/abs/2402.07401v1>
- [191] Fine-Grained Self-Endorsement Improves Factuality and Reasoning. arXiv:2402.15631v1, 2024. <https://arxiv.org/abs/2402.15631v1>
- [192] Improving the Reliability of Large Language Models by Leveraging Uncertainty-Aware In-Context Learning. arXiv:2310.04782v1, 2023. <https://arxiv.org/abs/2310.04782v1>
- [193] Focus on Your Question! Interpreting and Mitigating Toxic CoT Problems in Commonsense Reasoning. arXiv:2402.18344v1, 2024. <https://arxiv.org/abs/2402.18344v1>
- [194] INSIDE LLMs’ Internal States Retain the Power of Hallucination Detection. arXiv:2402.03744v1, 2024. <https://arxiv.org/abs/2402.03744v1>
- [195] The Category CNOT. arXiv:1707.02348v2, 2017. <https://arxiv.org/abs/1707.02348v2>
- [196] Ari The Automated R Instructor. arXiv:2007.13477v2, 2020. <https://arxiv.org/abs/2007.13477v2>
- [197] RIFLE Backpropagation in Depth for Deep Transfer Learning through Re-Initializing the Fully-connected LayEr. arXiv:2007.03349v1, 2020. <https://arxiv.org/abs/2007.03349v1>
- [198] Revisiting Fine-tuning for Few-shot Learning. arXiv:1910.00216v2, 2019. <https://arxiv.org/abs/1910.00216v2>
- [199] A Baseline for Few-Shot Image Classification. arXiv:1909.02729v5, 2019. <https://arxiv.org/abs/1909.02729v5>
- [200] Scaled Prompt-Tuning for Few-Shot Natural Language Generation. arXiv:2309.06759v1, 2023. <https://arxiv.org/abs/2309.06759v1>
- [201] Unsupervised Information Refinement Training of Large Language Models for Retrieval-Augmented Generation. arXiv:2402.18150v1, 2024. <https://arxiv.org/abs/2402.18150v1>
- [202] Unsupervised Finetuning. arXiv:2110.09510v1, 2021. <https://arxiv.org/abs/2110.09510v1>
- [203] LoBaSS Gauging Learnability in Supervised Fine-tuning Data. arXiv:2310.13008v1, 2023. <https://arxiv.org/abs/2310.13008v1>
- [204] Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning. arXiv:2205.05638v2, 2022. <https://arxiv.org/abs/2205.05638v2>
- [205] On Task Performance and Model Calibration with Supervised and Self-Ensembled In-Context Learning. arXiv:2312.13772v2, 2023. <https://arxiv.org/abs/2312.13772v2>
- [206] SuryaKiran at MEDIQA-Sum 2023 Leveraging LoRA for Clinical Dialogue Summarization. arXiv:2307.05162v1, 2023. <https://arxiv.org/abs/2307.05162v1>

- [207] Comparison between parameter-efficient techniques and full fine-tuning A case study on multilingual news article classification. arXiv:2308.07282v2, 2023. <https://arxiv.org/abs/2308.07282v2>
- [208] S-LoRA Serving Thousands of Concurrent LoRA Adapters. arXiv:2311.03285v2, 2023. <https://arxiv.org/abs/2311.03285v2>
- [209] LoraHub Efficient Cross-Task Generalization via Dynamic LoRA Composition. arXiv:2307.13269v2, 2023. <https://arxiv.org/abs/2307.13269v2>
- [210] Does Combining Parameter-efficient Modules Improve Few-shot Transfer Accuracy. arXiv:2402.15414v1, 2024. <https://arxiv.org/abs/2402.15414v1>
- [211] ALoRA Allocating Low-Rank Adaptation for Fine-tuning Large Language Models. arXiv:2403.16187v2, 2024. <https://arxiv.org/abs/2403.16187v2>
- [212] Sparse Low-rank Adaptation of Pre-trained Language Models. arXiv:2311.11696v1, 2023. <https://arxiv.org/abs/2311.11696v1>
- [213] A Single Linear Layer Yields Task-Adapted Low-Rank Matrices. arXiv:2403.14946v1, 2024. <https://arxiv.org/abs/2403.14946v1>
- [214] Multimodal Instruction Tuning with Conditional Mixture of LoRA. arXiv:2402.15896v1, 2024. <https://arxiv.org/abs/2402.15896v1>
- [215] MoELoRA Contrastive Learning Guided Mixture of Experts on Parameter-Efficient Fine-Tuning for Large Language Models. arXiv:2402.12851v1, 2024. <https://arxiv.org/abs/2402.12851v1>
- [216] Higher Layers Need More LoRA Experts. arXiv:2402.08562v1, 2024. <https://arxiv.org/abs/2402.08562v1>
- [217] LLaVA-MoLE Sparse Mixture of LoRA Experts for Mitigating Data Conflicts in Instruction Finetuning MLLMs. arXiv:2401.16160v2, 2024. <https://arxiv.org/abs/2401.16160v2>
- [218] Sparse is Enough in Fine-tuning Pre-trained Large Language Model. arXiv:2312.11875v1, 2023. <https://arxiv.org/abs/2312.11875v1>
- [219] RoSA Accurate Parameter-Efficient Fine-Tuning via Robust Adaptation. arXiv:2401.04679v6, 2024. <https://arxiv.org/abs/2401.04679v6>
- [220] LoTR Low Tensor Rank Weight Adaptation. arXiv:2402.01376v2, 2024. <https://arxiv.org/abs/2402.01376v2>
- [221] LoRA-FA Memory-efficient Low-rank Adaptation for Large Language Models Fine-tuning. arXiv:2308.03303v1, 2023. <https://arxiv.org/abs/2308.03303v1>
- [222] ResLoRA Identity Residual Mapping in Low-Rank Adaption. arXiv:2402.18039v1, 2024. <https://arxiv.org/abs/2402.18039v1>
- [223] AFLoRA Adaptive Freezing of Low Rank Adaptation in Parameter Efficient Fine-Tuning of Large Models. arXiv:2403.13269v3, 2024. <https://arxiv.org/abs/2403.13269v3>

- [224] GaLore Memory-Efficient LLM Training by Gradient Low-Rank Projection. arXiv:2403.03507v1, 2024. <https://arxiv.org/abs/2403.03507v1>
- [225] ZipLoRA Any Subject in Any Style by Effectively Merging LoRAs. arXiv:2311.13600v1, 2023. <https://arxiv.org/abs/2311.13600v1>
- [226] Memory-Efficient Fine-Tuning of Compressed Large Language Models via sub-4-bit Integer Quantization. arXiv:2305.14152v2, 2023. <https://arxiv.org/abs/2305.14152v2>
- [227] An Invitation to Deep Reinforcement Learning. arXiv:2312.08365v1, 2023. <https://arxiv.org/abs/2312.08365v1>
- [228] Summary of 2nd International Workshop on Requirements Engineering and Testing (RET). arXiv:2308.01933v1, 2023. <https://arxiv.org/abs/2308.01933v1>
- [229] ReFT Reasoning with Reinforced Fine-Tuning. arXiv:2401.08967v1, 2024. <https://arxiv.org/abs/2401.08967v1>
- [230] Fine-tuning Language Models with Generative Adversarial Reward Modelling. arXiv:2305.06176v3, 2023. <https://arxiv.org/abs/2305.06176v3>
- [231] Reward Model Ensembles Help Mitigate Overoptimization. arXiv:2310.02743v2, 2023. <https://arxiv.org/abs/2310.02743v2>
- [232] Self-Play Fine-Tuning Converts Weak Language Models to Strong Language Models. arXiv:2401.01335v2, 2024. <https://arxiv.org/abs/2401.01335v2>
- [233] Augmenting Unsupervised Reinforcement Learning with Self-Reference. arXiv:2311.09692v1, 2023. <https://arxiv.org/abs/2311.09692v1>
- [234] Distillation Strategies for Proximal Policy Optimization. arXiv:1901.08128v2, 2019. <https://arxiv.org/abs/1901.08128v2>
- [235] Chain of LoRA Efficient Fine-tuning of Language Models via Residual Learning. arXiv:2401.04151v1, 2024. <https://arxiv.org/abs/2401.04151v1>
- [236] Dynamic Corrective Self-Distillation for Better Fine-Tuning of Pretrained Models. arXiv:2312.07028v1, 2023. <https://arxiv.org/abs/2312.07028v1>
- [237] A Self-Tuning Actor-Critic Algorithm. arXiv:2002.12928v5, 2020. <https://arxiv.org/abs/2002.12928v5>
- [238] QFT Quantized Full-parameter Tuning of LLMs with Affordable Resources. arXiv:2310.07147v1, 2023. <https://arxiv.org/abs/2310.07147v1>
- [239] Quantized Side Tuning Fast and Memory-Efficient Tuning of Quantized Large Language Models. arXiv:2401.07159v1, 2024. <https://arxiv.org/abs/2401.07159v1>
- [240] QLoRA Efficient Finetuning of Quantized LLMs. arXiv:2305.14314v1, 2023. <https://arxiv.org/abs/2305.14314v1>
- [241] LQ-LoRA Low-rank Plus Quantized Matrix Decomposition for Efficient Language Model Finetuning. arXiv:2311.12023v2, 2023. <https://arxiv.org/abs/2311.12023v2>

- [242] INT2.1 Towards Fine-Tunable Quantized Large Language Models with Error Correction through Low-Rank Adaptation. arXiv:2306.08162v1, 2023. <https://arxiv.org/abs/2306.08162v1>
- [243] ApiQ Finetuning of 2-Bit Quantized Large Language Model. arXiv:2402.05147v2, 2024. <https://arxiv.org/abs/2402.05147v2>
- [244] QUIK Towards End-to-End 4-Bit Inference on Generative Large Language Models. arXiv:2310.09259v2, 2023. <https://arxiv.org/abs/2310.09259v2>
- [245] SqueezeLLM Dense-and-Sparse Quantization. arXiv:2306.07629v3, 2023. <https://arxiv.org/abs/2306.07629v3>
- [246] Do Emergent Abilities Exist in Quantized Large Language Models An Empirical Study. arXiv:2307.08072v2, 2023. <https://arxiv.org/abs/2307.08072v2>
- [247] Improving Sharpness-Aware Minimization with Fisher Mask for Better Generalization on Language Models. arXiv:2210.05497v1, 2022. <https://arxiv.org/abs/2210.05497v1>
- [248] HyperJump Accelerating HyperBand via Risk Modelling. arXiv:2108.02479v5, 2021. <https://arxiv.org/abs/2108.02479v5>
- [249] Hyperparameter Optimization for Large Language Model Instruction-Tuning. arXiv:2312.00949v2, 2023. <https://arxiv.org/abs/2312.00949v2>
- [250] Knowledge Conflicts for LLMs A Survey. arXiv:2403.08319v1, 2024. <https://arxiv.org/abs/2403.08319v1>
- [251] Untangle the KNOT Interweaving Conflicting Knowledge and Reasoning Skills in Large Language Models. arXiv:2404.03577v1, 2024. <https://arxiv.org/abs/2404.03577v1>
- [252] Entity-Based Knowledge Conflicts in Question Answering. arXiv:2109.05052v2, 2021. <https://arxiv.org/abs/2109.05052v2>
- [253] Tug-of-War Between Knowledge Exploring and Resolving Knowledge Conflicts in Retrieval-Augmented Language Models. arXiv:2402.14409v1, 2024. <https://arxiv.org/abs/2402.14409v1>
- [254] Context-faithful Prompting for Large Language Models. arXiv:2303.11315v2, 2023. <https://arxiv.org/abs/2303.11315v2>
- [255] Robust and Scalable Model Editing for Large Language Models. arXiv:2403.17431v1, 2024. <https://arxiv.org/abs/2403.17431v1>
- [256] Human-Imperceptible Retrieval Poisoning Attacks in LLM-Powered Applications. arXiv:2404.17196v1, 2024. <https://arxiv.org/abs/2404.17196v1>
- [257] PoisonedRAG Knowledge Poisoning Attacks to Retrieval-Augmented Generation of Large Language Models. arXiv:2402.07867v1, 2024. <https://arxiv.org/abs/2402.07867v1>
- [258] Typos that Broke the RAG’s Back Genetic Attack on RAG Pipeline by Simulating Documents in the Wild via Low-level Perturbations. arXiv:2404.13948v1, 2024. <https://arxiv.org/abs/2404.13948v1>

- [259] Prompt Perturbation in Retrieval-Augmented Generation based Large Language Models. arXiv:2402.07179v1, 2024. <https://arxiv.org/abs/2402.07179v1>
- [260] PRADA Practical Black-Box Adversarial Attacks against Neural Ranking Models. arXiv:2204.01321v3, 2022. <https://arxiv.org/abs/2204.01321v3>
- [261] Black-box Adversarial Attacks against Dense Retrieval Models A Multi-view Contrastive Learning Method. arXiv:2308.09861v1, 2023. <https://arxiv.org/abs/2308.09861v1>
- [262] How Should Pre-Trained Language Models Be Fine-Tuned Towards Adversarial Robustness. arXiv:2112.11668v1, 2021. <https://arxiv.org/abs/2112.11668v1>
- [263] Perturbation-Invariant Adversarial Training for Neural Ranking Models Improving the Effectiveness-Robustness Trade-Off. arXiv:2312.10329v1, 2023. <https://arxiv.org/abs/2312.10329v1>
- [264] Certified Robustness to Word Substitution Ranking Attack for Neural Ranking Models. arXiv:2209.06691v1, 2022. <https://arxiv.org/abs/2209.06691v1>
- [265] NoMIRACL Knowing When You Don’t Know for Robust Multilingual Retrieval-Augmented Generation. arXiv:2312.11361v2, 2023. <https://arxiv.org/abs/2312.11361v2>
- [266] Towards a Robust Retrieval-Based Summarization System. arXiv:2403.19889v1, 2024. <https://arxiv.org/abs/2403.19889v1>
- [267] Pandora Jailbreak GPTs by Retrieval Augmented Generation Poisoning. arXiv:2402.08416v1, 2024. <https://arxiv.org/abs/2402.08416v1>
- [268] Follow My Instruction and Spill the Beans Scalable Data Extraction from Retrieval-Augmented Generation Systems. arXiv:2402.17840v1, 2024. <https://arxiv.org/abs/2402.17840v1>
- [269] Investigating the prompt leakage effect and black-box defenses for multi-turn LLM interactions. arXiv:2404.16251v2, 2024. <https://arxiv.org/abs/2404.16251v2>
- [270] RecUP-FL Reconciling Utility and Privacy in Federated Learning via User-configurable Privacy Defense. arXiv:2304.05135v1, 2023. <https://arxiv.org/abs/2304.05135v1>
- [271] Whispers in the Machine Confidentiality in LLM-integrated Systems. arXiv:2402.06922v1, 2024. <https://arxiv.org/abs/2402.06922v1>
- [272] Algorand. arXiv:1607.01341v9, 2016. <https://arxiv.org/abs/1607.01341v9>
- [273] The Good and The Bad Exploring Privacy Issues in Retrieval-Augmented Generation (RAG). arXiv:2402.16893v1, 2024. <https://arxiv.org/abs/2402.16893v1>
- [274] Follow the Leader If You Can, Hedge If You Must. arXiv:1301.0534v2, 2013. <https://arxiv.org/abs/1301.0534v2>
- [275] Privacy Implications of Retrieval-Based Language Models. arXiv:2305.14888v1, 2023. <https://arxiv.org/abs/2305.14888v1>
- [276] Quantifying Membership Privacy via Information Leakage. arXiv:2010.05965v2, 2020. <https://arxiv.org/abs/2010.05965v2>

- [277] Search and Rescue on a Poset. arXiv:2312.06622v2, 2023. <https://arxiv.org/abs/2312.06622v2>
- [278] Source Matching and Rewriting. arXiv:2202.04153v1, 2022. <https://arxiv.org/abs/2202.04153v1>
- [279] Effects of Differential Privacy and Data Skewness on Membership Inference Vulnerability. arXiv:1911.09777v1, 2019. <https://arxiv.org/abs/1911.09777v1>
- [280] DPMLBench Holistic Evaluation of Differentially Private Machine Learning. arXiv:2305.05900v2, 2023. <https://arxiv.org/abs/2305.05900v2>
- [281] Flexible Accuracy for Differential Privacy. arXiv:2110.09580v1, 2021. <https://arxiv.org/abs/2110.09580v1>
- [282] Private Private Information. arXiv:2112.14356v3, 2021. <https://arxiv.org/abs/2112.14356v3>
- [283] Data. arXiv:1801.04992v2, 2017. <https://arxiv.org/abs/1801.04992v2>
- [284] Does Differential Privacy Prevent Backdoor Attacks in Practice. arXiv:2311.06227v1, 2023. <https://arxiv.org/abs/2311.06227v1>
- [285] Reconstruction of training samples from loss functions. arXiv:1805.07337v1, 2018. <https://arxiv.org/abs/1805.07337v1>
- [286] Client-specific Property Inference against Secure Aggregation in Federated Learning. arXiv:2303.03908v2, 2023. <https://arxiv.org/abs/2303.03908v2>
- [287] Privacy Threats Analysis to Secure Federated Learning. arXiv:2106.13076v1, 2021. <https://arxiv.org/abs/2106.13076v1>
- [288] MemGuard Defending against Black-Box Membership Inference Attacks via Adversarial Examples. arXiv:1909.10594v3, 2019. <https://arxiv.org/abs/1909.10594v3>
- [289] Uncertainty Sets for Image Classifiers using Conformal Prediction. arXiv:2009.14193v5, 2020. <https://arxiv.org/abs/2009.14193v5>
- [290] Distribution-Free, Risk-Controlling Prediction Sets. arXiv:2101.02703v3, 2021. <https://arxiv.org/abs/2101.02703v3>
- [291] Non-Exchangeable Conformal Risk Control. arXiv:2310.01262v2, 2023. <https://arxiv.org/abs/2310.01262v2>
- [292] Adaptive Conformal Inference Under Distribution Shift. arXiv:2106.00170v3, 2021. <https://arxiv.org/abs/2106.00170v3>
- [293] Non-Exchangeable Conformal Language Generation with Nearest Neighbors. arXiv:2402.00707v1, 2024. <https://arxiv.org/abs/2402.00707v1>
- [294] Conformal Prediction with Missing Values. arXiv:2306.02732v1, 2023. <https://arxiv.org/abs/2306.02732v1>
- [295] Conformal Prediction with Large Language Models for Multi-Choice Question Answering. arXiv:2305.18404v3, 2023. <https://arxiv.org/abs/2305.18404v3>

- [296] API Is Enough Conformal Prediction for Large Language Models Without Logit-Access. arXiv:2403.01216v2, 2024. <https://arxiv.org/abs/2403.01216v2>
- [297] COLEP Certifiably Robust Learning-Reasoning Conformal Prediction via Probabilistic Circuits. arXiv:2403.11348v1, 2024. <https://arxiv.org/abs/2403.11348v1>
- [298] KIVI A Tuning-Free Asymmetric 2bit Quantization for KV Cache. arXiv:2402.02750v1, 2024. <https://arxiv.org/abs/2402.02750v1>
- [299] Efficient Memory Management for Large Language Model Serving with PagedAttention. arXiv:2309.06180v1, 2023. <https://arxiv.org/abs/2309.06180v1>
- [300] LLM-QAT Data-Free Quantization Aware Training for Large Language Models. arXiv:2305.17888v1, 2023. <https://arxiv.org/abs/2305.17888v1>
- [301] SpQR A Sparse-Quantized Representation for Near-Lossless LLM Weight Compression. arXiv:2306.03078v1, 2023. <https://arxiv.org/abs/2306.03078v1>
- [302] IntactKV Improving Large Language Model Quantization by Keeping Pivot Tokens Intact. arXiv:2403.01241v1, 2024. <https://arxiv.org/abs/2403.01241v1>
- [303] Shortened LLaMA A Simple Depth Pruning for Large Language Models. arXiv:2402.02834v1, 2024. <https://arxiv.org/abs/2402.02834v1>
- [304] WKVQuant Quantizing Weight and Key Value Cache for Large Language Models Gains More. arXiv:2402.12065v2, 2024. <https://arxiv.org/abs/2402.12065v2>
- [305] Scissorhands Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. arXiv:2305.17118v2, 2023. <https://arxiv.org/abs/2305.17118v2>
- [306] H\$_2\$O Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. arXiv:2306.14048v3, 2023. <https://arxiv.org/abs/2306.14048v3>
- [307] Keyformer KV Cache Reduction through Key Tokens Selection for Efficient Generative Inference. arXiv:2403.09054v2, 2024. <https://arxiv.org/abs/2403.09054v2>
- [308] Sequence can Secretly Tell You What to Discard. arXiv:2404.15949v1, 2024. <https://arxiv.org/abs/2404.15949v1>
- [309] Dynamic Memory Compression Retrofitting LLMs for Accelerated Inference. arXiv:2403.09636v1, 2024. <https://arxiv.org/abs/2403.09636v1>
- [310] Context Compression for Auto-regressive Transformers with Sentinel Tokens. arXiv:2310.08152v2, 2023. <https://arxiv.org/abs/2310.08152v2>
- [311] T3 Transparent Tracking & Triggering for Fine-grained Overlap of Compute & Collectives. arXiv:2401.16677v1, 2024. <https://arxiv.org/abs/2401.16677v1>
- [312] Accelerating Heterogeneous Tensor Parallelism via Flexible Workload Control. arXiv:2401.11469v1, 2024. <https://arxiv.org/abs/2401.11469v1>
- [313] FastDecode High-Throughput GPU-Efficient LLM Serving using Heterogeneous Pipelines. arXiv:2403.11421v1, 2024. <https://arxiv.org/abs/2403.11421v1>

- [314] HeteGen Heterogeneous Parallel Inference for Large Language Models on Resource-Constrained Devices. arXiv:2403.01164v1, 2024. <https://arxiv.org/abs/2403.01164v1>
- [315] NeuPIMs NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing. arXiv:2403.00579v3, 2024. <https://arxiv.org/abs/2403.00579v3>
- [316] GPU Domain Specialization via Composable On-Package Architecture. arXiv:2104.02188v1, 2021. <https://arxiv.org/abs/2104.02188v1>
- [317] DistServe Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. arXiv:2401.09670v2, 2024. <https://arxiv.org/abs/2401.09670v2>
- [318] Modeling Data Movement Performance on Heterogeneous Architectures. arXiv:2010.10378v3, 2020. <https://arxiv.org/abs/2010.10378v3>
- [319] Iris Automatic Generation of Efficient Data Layouts for High Bandwidth Utilization. arXiv:2211.04361v1, 2022. <https://arxiv.org/abs/2211.04361v1>
- [320] The Role of Idle Waves, Desynchronization, and Bottleneck Evasion in the Performance of Parallel Programs. arXiv:2205.04190v1, 2022. <https://arxiv.org/abs/2205.04190v1>
- [321] Mage Online Interference-Aware Scheduling in Multi-Scale Heterogeneous Systems. arXiv:1804.06462v1, 2018. <https://arxiv.org/abs/1804.06462v1>
- [322] Accelerating HPC codes on Intel(R) Omni-Path Architecture networks From particle physics to Machine Learning. arXiv:1711.04883v1, 2017. <https://arxiv.org/abs/1711.04883v1>
- [323] Beyond the Speculative Game A Survey of Speculative Execution in Large Language Models. arXiv:2404.14897v1, 2024. <https://arxiv.org/abs/2404.14897v1>
- [324] Accelerating Large Language Model Decoding with Speculative Sampling. arXiv:2302.01318v1, 2023. <https://arxiv.org/abs/2302.01318v1>
- [325] DistillSpec Improving Speculative Decoding via Knowledge Distillation. arXiv:2310.08461v2, 2023. <https://arxiv.org/abs/2310.08461v2>
- [326] Sequoia Scalable, Robust, and Hardware-aware Speculative Decoding. arXiv:2402.12374v2, 2024. <https://arxiv.org/abs/2402.12374v2>
- [327] Recursive Speculative Decoding Accelerating LLM Inference via Sampling Without Replacement. arXiv:2402.14160v2, 2024. <https://arxiv.org/abs/2402.14160v2>
- [328] TriForce Lossless Acceleration of Long Sequence Generation with Hierarchical Speculative Decoding. arXiv:2404.11912v2, 2024. <https://arxiv.org/abs/2404.11912v2>
- [329] SkipDecode Autoregressive Skip Decoding with Batching and Caching for Efficient LLM Inference. arXiv:2307.02628v1, 2023. <https://arxiv.org/abs/2307.02628v1>
- [330] Skip-Sliding Window Codes. arXiv:1711.09494v2, 2017. <https://arxiv.org/abs/1711.09494v2>
- [331] Accelerating Inference in Large Language Models with a Unified Layer Skipping Strategy. arXiv:2404.06954v1, 2024. <https://arxiv.org/abs/2404.06954v1>

- [332] Confident Adaptive Language Modeling. arXiv:2207.07061v2, 2022. <https://arxiv.org/abs/2207.07061v2>
- [333] You Need Multiple Exiting Dynamic Early Exiting for Accelerating Unified Vision Language Model. arXiv:2211.11152v2, 2022. <https://arxiv.org/abs/2211.11152v2>
- [334] Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. arXiv:2403.02310v1, 2024. <https://arxiv.org/abs/2403.02310v1>
- [335] AttentionStore Cost-effective Attention Reuse across Multi-turn Conversations in Large Language Model Serving. arXiv:2403.19708v2, 2024. <https://arxiv.org/abs/2403.19708v2>
- [336] Fast and Robust Early-Exiting Framework for Autoregressive Language Models with Synchronized Parallel Decoding. arXiv:2310.05424v1, 2023. <https://arxiv.org/abs/2310.05424v1>
- [337] Profiling-Assisted Decoupled Access-Execute. arXiv:1601.01722v1, 2016. <https://arxiv.org/abs/1601.01722v1>
- [338] SuperServe Fine-Grained Inference Serving for Unpredictable Workloads. arXiv:2312.16733v1, 2023. <https://arxiv.org/abs/2312.16733v1>
- [339] Linear, or Non-Linear, That is the Question!. arXiv:2111.07265v2, 2021. <https://arxiv.org/abs/2111.07265v2>
- [340] Intelligence Primer. arXiv:2008.07324v4, 2020. <https://arxiv.org/abs/2008.07324v4>
- [341] Event-B SLP. arXiv:1301.2368v1, 2013. <https://arxiv.org/abs/1301.2368v1>
- [342] Computing with P Systems. arXiv:1809.08664v1, 2018. <https://arxiv.org/abs/1809.08664v1>
- [343] New primitives for bounded degradation in network service. arXiv:2208.08429v1, 2022. <https://arxiv.org/abs/2208.08429v1>
- [344] Optimal Resource Allocation for Elastic and Inelastic Jobs. arXiv:2005.09745v1, 2020. <https://arxiv.org/abs/2005.09745v1>
- [345] Elastic Decision Transformer. arXiv:2307.02484v6, 2023. <https://arxiv.org/abs/2307.02484v6>
- [346] Energy-Aware Adaptive Offloading of Soft Real-Time Jobs in Mobile Edge Clouds. arXiv:2102.05504v2, 2021. <https://arxiv.org/abs/2102.05504v2>
- [347] Mitigating Cold Starts in Serverless Platforms A Pool-Based Approach. arXiv:1903.12221v1, 2019. <https://arxiv.org/abs/1903.12221v1>
- [348] The Serverless Scheduling Problem and NOAH. arXiv:1809.06100v1, 2018. <https://arxiv.org/abs/1809.06100v1>
- [349] A Deep Reinforcement Learning based Algorithm for Time and Cost Optimized Scaling of Serverless Applications. arXiv:2308.11209v1, 2023. <https://arxiv.org/abs/2308.11209v1>

- [350] Edge computing service deployment and task offloading based on multi-task high-dimensional multi-objective optimization. arXiv:2312.04101v1, 2023. <https://arxiv.org/abs/2312.04101v1>
- [351] Elasticutor Rapid Elasticity for Realtime Stateful Stream Processing. arXiv:1711.01046v1, 2017. <https://arxiv.org/abs/1711.01046v1>
- [352] CILP Co-simulation based Imitation Learner for Dynamic Resource Provisioning in Cloud Computing Environments. arXiv:2302.05630v2, 2023. <https://arxiv.org/abs/2302.05630v2>
- [353] Locally Optimal Load Balancing. arXiv:1502.04511v1, 2015. <https://arxiv.org/abs/1502.04511v1>
- [354] iRAG An Incremental Retrieval Augmented Generation System for Videos. arXiv:2404.12309v1, 2024. <https://arxiv.org/abs/2404.12309v1>
- [355] REALM RAG-Driven Enhancement of Multimodal Electronic Health Records Analysis via Large Language Models. arXiv:2402.07016v1, 2024. <https://arxiv.org/abs/2402.07016v1>
- [356] Retrieval Augmented Chest X-Ray Report Generation using OpenAI GPT models. arXiv:2305.03660v1, 2023. <https://arxiv.org/abs/2305.03660v1>
- [357] MuRAG Multimodal Retrieval-Augmented Generator for Open Question Answering over Images and Text. arXiv:2210.02928v2, 2022. <https://arxiv.org/abs/2210.02928v2>
- [358] Unlocking Multi-View Insights in Knowledge-Dense Retrieval-Augmented Generation. arXiv:2404.12879v1, 2024. <https://arxiv.org/abs/2404.12879v1>
- [359] A review of deep learning-based information fusion techniques for multimodal medical image classification. arXiv:2404.15022v1, 2024. <https://arxiv.org/abs/2404.15022v1>
- [360] Variational Fusion for Multimodal Sentiment Analysis. arXiv:1908.06008v1, 2019. <https://arxiv.org/abs/1908.06008v1>
- [361] Adaptive Fusion Techniques for Multimodal Data. arXiv:1911.03821v2, 2019. <https://arxiv.org/abs/1911.03821v2>
- [362] Progressive Fusion for Multimodal Integration. arXiv:2209.00302v2, 2022. <https://arxiv.org/abs/2209.00302v2>
- [363] Deep Equilibrium Multimodal Fusion. arXiv:2306.16645v1, 2023. <https://arxiv.org/abs/2306.16645v1>
- [364] Multimodal Densenet. arXiv:1811.07407v1, 2018. <https://arxiv.org/abs/1811.07407v1>
- [365] MultiModN- Multimodal, Multi-Task, Interpretable Modular Networks. arXiv:2309.14118v2, 2023. <https://arxiv.org/abs/2309.14118v2>
- [366] Robust Multimodal Learning with Missing Modalities via Parameter-Efficient Adaptation. arXiv:2310.03986v3, 2023. <https://arxiv.org/abs/2310.03986v3>
- [367] Borrowing Treasures from Neighbors In-Context Learning for Multimodal Learning with Missing Modalities and Data Scarcity. arXiv:2403.09428v2, 2024. <https://arxiv.org/abs/2403.09428v2>

- [368] Retrieving Multimodal Information for Augmented Generation A Survey. arXiv:2303.10868v3, 2023. <https://arxiv.org/abs/2303.10868v3>
- [369] X-TRA Improving Chest X-ray Tasks with Cross-Modal Retrieval Augmentation. arXiv:2302.11352v1, 2023. <https://arxiv.org/abs/2302.11352v1>
- [370] Mr. TyDi A Multi-lingual Benchmark for Dense Retrieval. arXiv:2108.08787v2, 2021. <https://arxiv.org/abs/2108.08787v2>
- [371] LAReQA Language-agnostic answer retrieval from a multilingual pool. arXiv:2004.05484v1, 2020. <https://arxiv.org/abs/2004.05484v1>
- [372] Inducing Language-Agnostic Multilingual Representations. arXiv:2008.09112v2, 2020. <https://arxiv.org/abs/2008.09112v2>
- [373] One Question Answering Model for Many Languages with Cross-lingual Dense Passage Retrieval. arXiv:2107.11976v2, 2021. <https://arxiv.org/abs/2107.11976v2>
- [374] Improving Low-Resource Cross-lingual Document Retrieval by Reranking with Deep Bilingual Representations. arXiv:1906.03492v1, 2019. <https://arxiv.org/abs/1906.03492v1>
- [375] Zero-Shot Cross-Lingual Reranking with Large Language Models for Low-Resource Languages. arXiv:2312.16159v1, 2023. <https://arxiv.org/abs/2312.16159v1>
- [376] Cross-language Sentence Selection via Data Augmentation and Rationale Training. arXiv:2106.02293v1, 2021. <https://arxiv.org/abs/2106.02293v1>
- [377] MIA 2022 Shared Task Evaluating Cross-lingual Open-Retrieval Question Answering for 16 Diverse Languages. arXiv:2207.00758v1, 2022. <https://arxiv.org/abs/2207.00758v1>
- [378] Pivot Through English Reliably Answering Multilingual Questions without Document Retrieval. arXiv:2012.14094v2, 2020. <https://arxiv.org/abs/2012.14094v2>
- [379] Cross-Lingual Open-Domain Question Answering with Answer Sentence Generation. arXiv:2110.07150v3, 2021. <https://arxiv.org/abs/2110.07150v3>
- [380] Pre-training Cross-lingual Open Domain Question Answering with Large-scale Synthetic Supervision. arXiv:2402.16508v1, 2024. <https://arxiv.org/abs/2402.16508v1>
- [381] ARAGOG Advanced RAG Output Grading. arXiv:2404.01037v1, 2024. <https://arxiv.org/abs/2404.01037v1>
- [382] Path to Medical AGI Unify Domain-specific Medical LLMs with the Lowest Cost. arXiv:2306.10765v1, 2023. <https://arxiv.org/abs/2306.10765v1>
- [383] IryoNLP at MEDIQA-CORR 2024 Tackling the Medical Error Detection & Correction Task On the Shoulders of Medical Agents. arXiv:2404.15488v1, 2024. <https://arxiv.org/abs/2404.15488v1>
- [384] How well do LLMs cite relevant medical references An evaluation framework and analyses. arXiv:2402.02008v1, 2024. <https://arxiv.org/abs/2402.02008v1>
- [385] Development and Testing of a Novel Large Language Model-Based Clinical Decision Support Systems for Medication Safety in 12 Clinical Specialties. arXiv:2402.01741v2, 2024. <https://arxiv.org/abs/2402.01741v2>

- [386] From RAGs to riches Using large language models to write documents for clinical trials. arXiv:2402.16406v1, 2024. <https://arxiv.org/abs/2402.16406v1>
- [387] Evaluating and Enhancing Large Language Models Performance in Domain-specific Medicine Osteoarthritis Management with DocOA. arXiv:2401.12998v1, 2024. <https://arxiv.org/abs/2401.12998v1>
- [388] CBR-RAG Case-Based Reasoning for Retrieval Augmented Generation in LLMs for Legal Question Answering. arXiv:2404.04302v1, 2024. <https://arxiv.org/abs/2404.04302v1>
- [389] From RAG to QA-RAG Integrating Generative AI for Pharmaceutical Regulatory Compliance Process. arXiv:2402.01717v1, 2024. <https://arxiv.org/abs/2402.01717v1>
- [390] Retrieval augmented text-to-SQL generation for epidemiological question answering using electronic health records. arXiv:2403.09226v1, 2024. <https://arxiv.org/abs/2403.09226v1>
- [391] Augmenting Black-box LLMs with Medical Textbooks for Clinical Question Answering. arXiv:2309.02233v2, 2023. <https://arxiv.org/abs/2309.02233v2>
- [392] Assessing the Usability of GutGPT A Simulation Study of an AI Clinical Decision Support System for Gastrointestinal Bleeding Risk. arXiv:2312.10072v1, 2023. <https://arxiv.org/abs/2312.10072v1>
- [393] The Science of Detecting LLM-Generated Texts. arXiv:2303.07205v3, 2023. <https://arxiv.org/abs/2303.07205v3>
- [394] InspectorRAGet An Introspection Platform for RAG Evaluation. arXiv:2404.17347v1, 2024. <https://arxiv.org/abs/2404.17347v1>
- [395] Evaluating Very Long-Term Conversational Memory of LLM Agents. arXiv:2402.17753v1, 2024. <https://arxiv.org/abs/2402.17753v1>
- [396] From LLM to Conversational Agent A Memory Enhanced Architecture with Fine-Tuning of Large Language Models. arXiv:2401.02777v2, 2024. <https://arxiv.org/abs/2401.02777v2>
- [397] RAM Towards an Ever-Improving Memory System by Learning from Communications. arXiv:2404.12045v1, 2024. <https://arxiv.org/abs/2404.12045v1>
- [398] Enhancing Multilingual Information Retrieval in Mixed Human Resources Environments A RAG Model Implementation for Multicultural Enterprise. arXiv:2401.01511v1, 2024. <https://arxiv.org/abs/2401.01511v1>
- [399] CRUD-RAG A Comprehensive Chinese Benchmark for Retrieval-Augmented Generation of Large Language Models. arXiv:2401.17043v2, 2024. <https://arxiv.org/abs/2401.17043v2>
- [400] Retrieval Augmented Generation and Representative Vector Summarization for large unstructured textual data in Medical Education. arXiv:2308.00479v1, 2023. <https://arxiv.org/abs/2308.00479v1>
- [401] Learning to Sample. arXiv:1812.01659v2, 2018. <https://arxiv.org/abs/1812.01659v2>
- [402] Seven Failure Points When Engineering a Retrieval Augmented Generation System. arXiv:2401.05856v1, 2024. <https://arxiv.org/abs/2401.05856v1>

- [403] Reference-based Metrics Disprove Themselves in Question Generation. arXiv:2403.12242v1, 2024. <https://arxiv.org/abs/2403.12242v1>
- [404] The statistical advantage of automatic NLG metrics at the system level. arXiv:2105.12437v1, 2021. <https://arxiv.org/abs/2105.12437v1>
- [405] Spurious Correlations in Reference-Free Evaluation of Text Generation. arXiv:2204.09890v1, 2022. <https://arxiv.org/abs/2204.09890v1>
- [406] MoverScore Text Generation Evaluating with Contextualized Embeddings and Earth Mover Distance. arXiv:1909.02622v2, 2019. <https://arxiv.org/abs/1909.02622v2>
- [407] REAM\$\{\}\$sharp\$ An Enhancement Approach to Reference-based Evaluation Metrics for Open-domain Dialog Generation. arXiv:2105.14488v2, 2021. <https://arxiv.org/abs/2105.14488v2>
- [408] SLIDE Reference-free Evaluation for Machine Translation using a Sliding Document Window. arXiv:2309.08832v2, 2023. <https://arxiv.org/abs/2309.08832v2>
- [409] The Probabilities Also Matter A More Faithful Metric for Faithfulness of Free-Text Explanations in Large Language Models. arXiv:2404.03189v1, 2024. <https://arxiv.org/abs/2404.03189v1>
- [410] Testing robustness of predictions of trained classifiers against naturally occurring perturbations. arXiv:2204.10046v2, 2022. <https://arxiv.org/abs/2204.10046v2>
- [411] Collect, Measure, Repeat Reliability Factors for Responsible AI Data Collection. arXiv:2308.12885v2, 2023. <https://arxiv.org/abs/2308.12885v2>
- [412] A Comparison of Audio Preprocessing Techniques and Deep Learning Algorithms for Raga Recognition. arXiv:2212.05335v1, 2022. <https://arxiv.org/abs/2212.05335v1>
- [413] ARES An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. arXiv:2311.09476v2, 2023. <https://arxiv.org/abs/2311.09476v2>
- [414] ARES Adaptive, Reconfigurable, Erasure coded, atomic Storage. arXiv:1805.03727v2, 2018. <https://arxiv.org/abs/1805.03727v2>
- [415] Double Perturbation On the Robustness of Robustness and Counterfactual Bias Evaluation. arXiv:2104.05232v1, 2021. <https://arxiv.org/abs/2104.05232v1>
- [416] MRKE The Multi-hop Reasoning Evaluation of LLMs by Knowledge Edition. arXiv:2402.11924v2, 2024. <https://arxiv.org/abs/2402.11924v2>
- [417] AGIBench A Multi-granularity, Multimodal, Human-referenced, Auto-scoring Benchmark for Large Language Models. arXiv:2309.06495v1, 2023. <https://arxiv.org/abs/2309.06495v1>
- [418] MMBench Is Your Multi-modal Model an All-around Player. arXiv:2307.06281v3, 2023. <https://arxiv.org/abs/2307.06281v3>
- [419] A Benchmark for Multi-modal Foundation Models on Low-level Vision from Single Images to Pairs. arXiv:2402.07116v1, 2024. <https://arxiv.org/abs/2402.07116v1>

- [420] MME A Comprehensive Evaluation Benchmark for Multimodal Large Language Models. arXiv:2306.13394v4, 2023. <https://arxiv.org/abs/2306.13394v4>
- [421] MLLM-as-a-Judge Assessing Multimodal LLM-as-a-Judge with Vision-Language Benchmark. arXiv:2402.04788v1, 2024. <https://arxiv.org/abs/2402.04788v1>
- [422] How Abilities in Large Language Models are Affected by Supervised Fine-tuning Data Composition. arXiv:2310.05492v3, 2023. <https://arxiv.org/abs/2310.05492v3>
- [423] Efficient Benchmarking of Language Models. arXiv:2308.11696v5, 2023. <https://arxiv.org/abs/2308.11696v5>
- [424] Superposition Prompting Improving and Accelerating Retrieval-Augmented Generation. arXiv:2404.06910v1, 2024. <https://arxiv.org/abs/2404.06910v1>
- [425] LoRA Low-Rank Adaptation of Large Language Models. arXiv:2106.09685v2, 2021. <https://arxiv.org/abs/2106.09685v2>
- [426] A RAG Method for Source Code Inquiry Tailored to Long-Context LLMs. arXiv:2404.06082v1, 2024. <https://arxiv.org/abs/2404.06082v1>
- [427] Improving Wikipedia Verifiability with AI. arXiv:2207.06220v1, 2022. <https://arxiv.org/abs/2207.06220v1>
- [428] Human and technological infrastructures of fact-checking. arXiv:2205.10894v1, 2022. <https://arxiv.org/abs/2205.10894v1>
- [429] Generating Fact Checking Briefs. arXiv:2011.05448v1, 2020. <https://arxiv.org/abs/2011.05448v1>
- [430] Claim Check-Worthiness Detection How Well do LLMs Grasp Annotation Guidelines. arXiv:2404.12174v1, 2024. <https://arxiv.org/abs/2404.12174v1>
- [431] Assisting the Human Fact-Checkers Detecting All Previously Fact-Checked Claims in a Document. arXiv:2109.07410v3, 2021. <https://arxiv.org/abs/2109.07410v3>
- [432] Verifiable by Design Aligning Language Models to Quote from Pre-Training Data. arXiv:2404.03862v1, 2024. <https://arxiv.org/abs/2404.03862v1>
- [433] Fact-Saboteurs A Taxonomy of Evidence Manipulation Attacks against Fact-Verification Systems. arXiv:2209.03755v4, 2022. <https://arxiv.org/abs/2209.03755v4>
- [434] Synthetic Disinformation Attacks on Automated Fact Verification Systems. arXiv:2202.09381v1, 2022. <https://arxiv.org/abs/2202.09381v1>
- [435] ClaimVer Explainable Claim-Level Verification and Evidence Attribution of Text Through Knowledge Graphs. arXiv:2403.09724v1, 2024. <https://arxiv.org/abs/2403.09724v1>
- [436] Providing Assurance and Scrutability on Shared Data and Machine Learning Models with Verifiable Credentials. arXiv:2105.06370v1, 2021. <https://arxiv.org/abs/2105.06370v1>
- [437] Reproducibility Issues for BERT-based Evaluation Metrics. arXiv:2204.00004v3, 2022. <https://arxiv.org/abs/2204.00004v3>

- [438] Summing Up the Facts Additive Mechanisms Behind Factual Recall in LLMs. arXiv:2402.07321v1, 2024. <https://arxiv.org/abs/2402.07321v1>
- [439] The Cost of Down-Scaling Language Models Fact Recall Deteriorates before In-Context Learning. arXiv:2310.04680v1, 2023. <https://arxiv.org/abs/2310.04680v1>
- [440] Do Compressed LLMs Forget Knowledge An Experimental Study with Practical Implications. arXiv:2310.00867v3, 2023. <https://arxiv.org/abs/2310.00867v3>
- [441] How to Sift Out a Clean Data Subset in the Presence of Data Poisoning. arXiv:2210.06516v2, 2022. <https://arxiv.org/abs/2210.06516v2>
- [442] Competition Report Finding Universal Jailbreak Backdoors in Aligned LLMs. arXiv:2404.14461v1, 2024. <https://arxiv.org/abs/2404.14461v1>
- [443] Data Confidentiality In P2P Communication And Smart Contracts Of Blockchain In Industry 4.0. arXiv:2007.14195v1, 2020. <https://arxiv.org/abs/2007.14195v1>
- [444] A Distributed Framework for Scalable Search over Encrypted Documents. arXiv:1408.5539v1, 2014. <https://arxiv.org/abs/1408.5539v1>
- [445] Ciphertext-Only Attack on a Secure $\mathbb{Z}_N$ -NN Computation on Cloud. arXiv:2403.09080v2, 2024. <https://arxiv.org/abs/2403.09080v2>
- [446] It Takes Two to MeToo - Using Enclaves to Build Autonomous Trusted Systems. arXiv:1808.02708v1, 2018. <https://arxiv.org/abs/1808.02708v1>
- [447] A Solution for Commercializing, Decentralizing and Storing Electronic Medical Records by Integrating Proxy Re-Encryption, IPFS, and Blockchain. arXiv:2402.05498v1, 2024. <https://arxiv.org/abs/2402.05498v1>
- [448] Second layer data governance for permissioned blockchains the privacy management challenge. arXiv:2010.11677v1, 2020. <https://arxiv.org/abs/2010.11677v1>
- [449] Decentralized Inverse Transparency With Blockchain. arXiv:2304.11033v1, 2023. <https://arxiv.org/abs/2304.11033v1>
- [450] Mitigation of liveness attacks in DAG-based ledgers. arXiv:2305.01207v1, 2023. <https://arxiv.org/abs/2305.01207v1>
- [451] P-MOD Secure Privilege-Based Multilevel Organizational Data-Sharing in Cloud Computing. arXiv:1801.02685v1, 2018. <https://arxiv.org/abs/1801.02685v1>
- [452] Trust and Privacy in Knowledge Graphs. arXiv:1903.07673v1, 2019. <https://arxiv.org/abs/1903.07673v1>
- [453] A Forecasting-Based DLP Approach for Data Security. arXiv:2312.13704v1, 2023. <https://arxiv.org/abs/2312.13704v1>
- [454] Semantic, Efficient, and Secure Search over Encrypted Cloud Data. arXiv:2002.10294v1, 2020. <https://arxiv.org/abs/2002.10294v1>
- [455] Adaptive-RAG Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. arXiv:2403.14403v2, 2024. <https://arxiv.org/abs/2403.14403v2>

- [456] When Not to Trust Language Models Investigating Effectiveness of Parametric and Non-Parametric Memories. arXiv:2212.10511v4, 2022. <https://arxiv.org/abs/2212.10511v4>
- [457] Pre-computed memory or on-the-fly encoding A hybrid approach to retrieval augmentation makes the most of your compute. arXiv:2301.10448v2, 2023. <https://arxiv.org/abs/2301.10448v2>
- [458] Hybrid LLM Cost-Efficient and Quality-Aware Query Routing. arXiv:2404.14618v1, 2024. <https://arxiv.org/abs/2404.14618v1>
- [459] Scaling Up LLM Reviews for Google Ads Content Moderation. arXiv:2402.14590v1, 2024. <https://arxiv.org/abs/2402.14590v1>
- [460] Unsupervised Regenerative Learning of Hierarchical Features in Spiking Deep Networks for Object Recognition. arXiv:1602.01510v1, 2016. <https://arxiv.org/abs/1602.01510v1>
- [461] Privacy-Aware Semantic Cache for Large Language Models. arXiv:2403.02694v2, 2024. <https://arxiv.org/abs/2403.02694v2>
- [462] Active Retrieval Augmented Generation. arXiv:2305.06983v2, 2023. <https://arxiv.org/abs/2305.06983v2>
- [463] Heterogeneous Programming with Single Operation Multiple Data. arXiv:1312.4993v2, 2013. <https://arxiv.org/abs/1312.4993v2>
- [464] AWQ Activation-aware Weight Quantization for LLM Compression and Acceleration. arXiv:2306.00978v4, 2023. <https://arxiv.org/abs/2306.00978v4>
- [465] Overcoming Overconfidence for Active Learning. arXiv:2308.10571v1, 2023. <https://arxiv.org/abs/2308.10571v1>
- [466] Perturbation-Based Two-Stage Multi-Domain Active Learning. arXiv:2306.10700v1, 2023. <https://arxiv.org/abs/2306.10700v1>
- [467] Multimodal Procedural Planning via Dual Text-Image Prompting. arXiv:2305.01795v1, 2023. <https://arxiv.org/abs/2305.01795v1>
- [468] AdaPlanner Adaptive Planning from Feedback with Language Models. arXiv:2305.16653v1, 2023. <https://arxiv.org/abs/2305.16653v1>
- [469] Towards Reasoning in Large Language Models via Multi-Agent Peer Review Collaboration. arXiv:2311.08152v2, 2023. <https://arxiv.org/abs/2311.08152v2>
- [470] Actor-Attention-Critic for Multi-Agent Reinforcement Learning. arXiv:1810.02912v2, 2018. <https://arxiv.org/abs/1810.02912v2>
- [471] Intent-based Deep Reinforcement Learning for Multi-agent Informative Path Planning. arXiv:2303.05351v2, 2023. <https://arxiv.org/abs/2303.05351v2>
- [472] A new approach of designing Multi-Agent Systems. arXiv:1204.1581v1, 2012. <https://arxiv.org/abs/1204.1581v1>
- [473] Online Sparse Reinforcement Learning. arXiv:2011.04018v4, 2020. <https://arxiv.org/abs/2011.04018v4>

- [474] A Human-Centered Data-Driven Planner-Actor-Critic Architecture via Logic Programming. arXiv:1909.09209v1, 2019. <https://arxiv.org/abs/1909.09209v1>
- [475] Meta-Reinforcement Learning with Self-Modifying Networks. arXiv:2202.02363v3, 2022. <https://arxiv.org/abs/2202.02363v3>
- [476] A Unified Continuous Learning Framework for Multi-modal Knowledge Discovery and Pre-training. arXiv:2206.05555v1, 2022. <https://arxiv.org/abs/2206.05555v1>